

OptKit: A Compositional Optimization Framework for Compound AI Systems

Architecture, mathematical foundations, implementation design, and migration guide for Coinvault and rag-ttc

Status: Proposed design **Audience:** Engineers and interns implementing the first production version **Primary language:** Go **Reference systems inspected:** Coinvault, rag-ttc, RagOpt, JudgeKit, RagKit, and FlowKit **Design date:** 2026-08-20

Abstract

Coinvault and rag-ttc already contain most of the mechanisms needed for disciplined self-optimization: immutable candidate snapshots, one-change interventions, real agent execution, rich traces, deterministic contracts, LLM judging, paired comparisons, resource ceilings, resumable artifact custody, and conservative promotion gates. Those mechanisms are currently distributed across product code, RagOpt, JudgeKit, RagKit, FlowKit, configuration bundles, and human operating procedures.

This document proposes **OptKit**, a domain-neutral framework that makes the optimization process itself a first-class, persistent object. OptKit represents what may change, why a change was proposed, how a candidate was materialized, which complete agent trajectories were executed, what instruments measured them, which constraints were violated, how candidates relate to one another, what an optimizer learned, and why a release was or was not promoted. It provides one event-sourced campaign model from which a pre-run plan, a live execution view, and a historical optimization ledger can all be derived.

The design deliberately does not reduce a compound AI system to a prompt and a scalar score. The optimized object is a parameterized executable system. Its behavior is a distribution over multi-turn trajectories that may include model calls, retrieval, SQL, tools, citations, widgets, errors, and human review. Measurements are typed and protocol-identified. Hard constraints remain non-compensable. Search and release promotion remain separate algorithms. RL, GEPA-like reflection, coordinate search, random search, Bayesian optimization, and manual engineering become interchangeable proposal policies over the same substrate.

The document proceeds pedagogically. Each important term is introduced by motivation, given a precise definition, and then applied to Coinvault or rag-ttc. Later chapters specify Go APIs, artifact schemas, runtime algorithms, UI projections, command-line tools, migration steps, tests, and intern-sized work packages.

1. How to read this document

A reader new to the codebase should first read Chapters 2 through 7. They establish the running example, identify the reusable parts of the current systems, motivate OptKit, and introduce the mathematical model. Chapters 8 through 15 define the core domain vocabulary in increasing detail. Chapters 16 through 24 specify persistence, state machines, package boundaries, and APIs. Chapters 25 through 32 design the plan, live, and historical user interfaces. Chapters 33 through 37 give the migration path for RagOpt, rag-ttc, and Coinvault. Chapters 38 through 41 turn the design into concrete tools and intern-sized work packages. Chapters 42 through 46 provide worked examples. Chapters 47 through 50 cover testing, safety, operations, and framework observability. Part IX records the major design decisions and rejected alternatives; the appendices provide schemas, event catalogs, equations, checklists, a glossary, and a source map.

A reader already familiar with RagOpt can begin with Chapters 4, 8, 17, 34, 36, and 37. An intern assigned to one package should still read Chapters 5, 6, 8, 17, 19, 20, and 48 before coding; those chapters contain the laws that prevent apparently local changes from corrupting experiment semantics.

The central claim is simple:

OptKit should be a versioned experimental control system for optimizing stochastic compound programs, not a prompt-tuning library and not an arbitrary workflow engine.

That claim determines almost every boundary in the design.

1.1 Navigation map

Reader goal	Recommended path
Understand the mathematical idea	Chapters 2, 5, 8, 13, and 14
Understand package boundaries	Chapters 3, 6, 7, 16, and 22
Implement persistence and recovery	Chapters 17 through 20 and Appendix B
Build the UI	Chapters 25 through 32
Port RAG-TTC	Chapters 33 through 35, then Worked Example B
Port Coinvault	Chapters 33, 34, and 36, then Worked Example A
Add an optimizer	Chapters 15, 21, 24, 39.15, and Worked Example C
Review safety and correctness	Chapters 47 through 50 and Appendix E

The document uses one recurring pipeline:

snapshot → patch → candidate → trial → episode → trajectory
→ checks → measurements → estimates → decisions → lineage

When a later section introduces an interface, trace each argument back to one object in this pipeline. That prevents API details from obscuring the experiment being represented.

Part I — Existing systems, motivation, and mathematical foundations

2. The running example: Coinvault’s grounded-answer candidate

Abstract architecture is easier to understand when every term can be tied to a real experiment. This document therefore uses Coinvault’s grounded-answer-v2 candidate as its principal example.

2.1 The observed problem

Coinvault received comparison questions such as:

Compare Morgan dollars with Peace dollars.

The retrieval system found evidence for both subjects, but the answer model sometimes produced comparison clauses that were not directly entailed by the admitted evidence. The earliest failing stage was therefore not retrieval. It was answer grounding.

A human engineer formed the following hypothesis:

With comparison-intent retrieval held fixed, requiring direct clause-level entailment and adjacent citations will reduce unsupported comparison claims without degrading answer relevance.

The intervention changed one asset, `answer_grounding_prompt`, while keeping the runtime, corpus, retrieval policy, evidence ledger, evaluation suite, judge protocol, and other assets fixed.

2.2 What happened in the experiment

For the Morgan-versus-Peace case, faithfulness changed from approximately

0.4595 → 1.0000,

so the paired delta was

$$\Delta F = 1.0000 - 0.4595 = 0.5405.$$

For the gold-coins-versus-bars case, faithfulness changed from approximately

$$0.3778 \rightarrow 0.9615,$$

so

$$\Delta F \approx 0.5838.$$

The treatment strongly improved the cases it was intended to improve. Nevertheless, the whole feedback run had five other route, retrieval, or projection failures. The frozen release gate required zero failures, so the candidate was rejected for release.

This is not contradictory. It produced two different decisions:

1. **Component evidence:** the grounding mechanism was strongly supported on its target cases.
2. **Release evidence:** the complete candidate was not safe to promote as a whole-system release.

OptKit must represent both decisions. Conflating them loses useful learning.

2.3 The loop already present in Coinvault

The existing process can be written as:

```
inspect failures
↓
identify the earliest responsible component
↓
propose one bounded change and a causal hypothesis
↓
freeze identities, data, budgets, and gate policy
↓
execute incumbent and challenger on matched cases
↓
verify that the treatment was actually exercised
↓
run deterministic product contracts
↓
run judge instruments when eligible
↓
compute paired deltas and costs
↓
apply lexicographic gates
↓
record component and release decisions
↓
use the evidence to choose the next change
```

The inner evaluation half is mostly automated. The outer diagnosis-and-proposal half is currently human-driven. A GEPA-like optimizer would automate the proposal arrow, not replace the execution, custody, measurement, or promotion machinery.

3. What exists today

OptKit should extract and connect proven mechanisms. It should not discard them and start again.

3.1 FlowKit: bounded execution, not campaign control

FlowKit is a domain-neutral execution library. It provides ordered bounded mapping, content-addressed caches, batching, retries, rate admission, finite resource budgets, metering, reports, ledgers, and typed pipelines. It correctly separates semantic identity from execution policy: changing worker count or retry policy does not change the identity of the semantic work.

FlowKit explicitly is not a workflow server or persistent DAG scheduler. That boundary should remain. OptKit will use FlowKit to execute bounded campaign work, but campaign state, candidate lineage, adaptive planning, and promotion decisions belong above it.

3.2 JudgeKit: measurement instruments, not optimization policy

JudgeKit supplies the vocabulary and mechanics for constructs, measurement contracts, evidence, protocols, assessments, evaluator suites, calibration, panels, and audits. It treats a judge as a measurement instrument rather than as truth.

OptKit should preserve that distinction. It consumes typed measurement facts and protocol identities. It does not own the meaning of faithfulness, the prompts that define a Coinvault judge, or the calibration evidence that qualifies a judge protocol.

3.3 RagKit: RAG semantics, not campaign state

RagKit owns RAG-specific concepts: documents, chunks, evidence, retrieval, ranking, index bundles, and grounded-answer contracts. It should not import OptKit. Product adapters and a future RagOpt specialization can connect RAG concepts to OptKit.

3.4 RagOpt: a strong paired-experiment kernel

RagOpt currently provides:

- immutable snapshots and candidate bundles;
- independent verification that exactly one mutable asset changed;
- opaque product-defined evaluation cases;
- incumbent/challenger execution through an `Arm` interface;
- durable one-cell-per-case/repeat/arm artifacts;
- append-only active runs and immutable terminal runs;
- exact resume against frozen input digests;
- strict paired comparison with explicit missingness;
- lexicographic gates;
- blinded human review support;
- promotion review and apply plans that still require a human.

These are valuable semantics. OptKit generalizes them from a single paired run to a persistent adaptive campaign. Existing RagOpt artifacts must remain readable.

3.5 Coinvault: the richest product adapter

Coinvault's current adapter does much more than call a model. For each cell it:

1. loads a locked treatment contract;
2. materializes the arm-specific configuration;
3. runs the canonical production-like admin chat boundary;
4. captures model, tool, retrieval, evidence, citation, token, and terminal events;
5. verifies runtime semantic identities;

6. proves whether the treatment was exercised;
7. applies deterministic answer and route contracts;
8. calls the judge only when deterministic checks permit it;
9. writes a rich native artifact;
10. projects a small generic RagOpt outcome.

The weakness is not the execution path. The weakness is that treatment families are concentrated in a growing central switch, metrics cross the generic boundary as `map[string]float64`, candidate lineage is implicit, and the full optimization process is not persisted as a campaign.

3.6 rag-ttc: a simpler second product proof

rag-ttc wraps its real tool-loop runtime as a RagOpt arm, materializes a candidate tool description, records the native session, runs answer-quality judging, and projects faithfulness, relevance, and product costs. It already has an Experiments TUI that can browse completed run directories and generic metric columns.

Its earlier GEPA-inspired design also identified useful principles that remain correct:

- native JSONL or session artifacts are authoritative;
- SQLite is a rebuildable analysis projection;
- candidate bundles are immutable;
- reflection should consume selected diagnostic trajectories;
- one-change experiments improve causal attribution;
- hidden evaluation and the judge must remain outside the mutable set;
- automatic deployment should be deferred.

OptKit turns these local principles into shared framework contracts.

4. Why a new layer is needed

4.1 A run is not a campaign

A RagOpt run answers:

Did this challenger beat this incumbent on this frozen paired evaluation?

An optimization campaign answers a larger sequence of questions:

What can change? Why was each change proposed? Which candidates descend from which snapshots? Which evidence did the optimizer see? Which trials ran? Which candidates were abandoned, combined, or advanced? Which measurement protocols were used? Why did the campaign stop? Which result was reviewed and promoted?

A campaign may contain dozens of candidates and many trials. It may be adaptive: later work depends on earlier results. Treating every run as an isolated directory makes the optimization history difficult to understand and impossible to drive generically.

4.2 A metric map is not a measurement model

The current generic boundary stores:

Metrics `map[string]float64`

This cannot express:

- which construct the number operationalizes;
- which measurement contract defined it;
- which judge protocol produced it;
- whether it was applicable;
- which evidence supports it;

- whether it is a point value, category, interval, distribution, or count;
- whether two values are comparable across evaluator revisions.

An optimizer that ignores those distinctions will eventually optimize accidental evaluator behavior.

4.3 A treatment switch is not a search space

Coinvault’s treatment switch mixes four concerns:

1. the semantic variable that may change;
2. validation of candidate values;
3. application of the value to a runtime configuration;
4. observation proving that the runtime used the value.

As new treatment families are added, the switch becomes the framework. This prevents composition and makes multi-variable or generated candidates awkward.

OptKit separates those concerns into registered variables, materializers, and intervention verifiers.

4.4 A completed-run browser is not a live optimization UI

The existing rag-ttc experiment browser is useful, but it begins after run directories exist and primarily compares metric columns. The desired UI must also show:

- the adaptive plan before execution;
- candidate lineage and proposal rationale;
- currently active episodes and tool calls;
- resource use and remaining budget;
- intervention checks, contracts, and judge stages;
- data exposure and hidden-set boundaries;
- Pareto status and optimizer decisions;
- promotion review and production ancestry.

Building three separate data paths for planning, live operation, and history would create drift. OptKit therefore uses one event journal and multiple projections.

5. Mathematical foundations

This chapter introduces the mathematical model in the same order that an engineer encounters the system.

5.1 Parameterized executable system

Motivation A prompt optimizer assumes the object being changed is a string. Coinvault and rag-ttc contain prompts, but also retrieval depth, query transformation, reranking, tool schemas, model identities, runtime policies, and multi-turn control behavior. The optimized object must therefore be the entire executable system configuration.

Definition Let Θ be a **search space**. A point $\theta \in \Theta$ is one complete semantic configuration of a system. Let X be the space of cases or inputs and $\mathcal{T}$ the space of possible trajectories.

The system is represented as a stochastic map

$$K : \Theta \times X \rightsquigarrow \mathcal{T}.$$

The squiggly arrow means that a configuration and input determine a probability distribution over trajectories rather than one guaranteed output. In probability theory, this is a **Markov kernel**.

For fixed θ ,

$$\tau \sim K_{\theta}(x)$$

means: execute system configuration θ on input x , producing trajectory τ .

[!NOTE] A deterministic program is a special case: its distribution places probability one on a single trajectory. Using a stochastic model does not force every component to be random; it simply admits model sampling, provider variation, time-dependent data, concurrency, and tool behavior.

Coinvault example A simplified Coinvault configuration is

$$\theta = (d_{\text{default}}, d_{\text{forced}}, q_{\text{comparison}}, i_{\text{comparison}}, p_{\text{grounding}}, p_{\text{routing}}, p_{\text{policy}}, \rho_{\text{reranker}}, d_{\text{tool}}).$$

This is a heterogeneous product space:

$$\Theta = \Theta_1 \times \Theta_2 \times \dots \times \Theta_k,$$

where one coordinate may be an integer, another prompt text, another structured YAML plan, and another model configuration.

5.2 Variable and domain

Motivation To optimize a system, the framework must know what may change and what values are valid. A plain asset path does not state whether the asset is an integer setting, an arbitrary prompt, an enum, or a structured graph.

Definition A **variable** is a named semantic coordinate of the system. Its **domain** is the set of values that coordinate may take.

Examples:

- `knowledge.default_results` has a bounded integer domain;
- `answer.grounding_prompt` has an artifact/text domain;
- `retrieval.reranker` has a structured configuration domain;
- `agent.flow_topology` could have a graph domain;
- `widget.policy` could have a structured policy domain.

A variable also specifies how a value is canonically encoded. Canonical encoding is required because semantic identity must not depend on YAML key order or incidental whitespace unless whitespace is itself semantically meaningful.

Worked example

```
id: answer.grounding_prompt
kind: artifact
media_type: text/plain
mutable: true
constraints:
  max_bytes: 20000
application_adapter: coinvault.answer_prompt_suffix/v1
intervention_verifier: coinvault.prompt_suffix_applied/v1
```

The domain describes admissible prompt artifacts. The application adapter describes where the prompt enters Coinvault. The verifier later proves that the expected digest was present in the executed trajectory.

5.3 Snapshot

Motivation A candidate must be reproducible months later. A label such as `grounded-answer-v2` is not enough because external files, model profiles, corpus bundles, and locked policies may change.

Definition A **snapshot** is an immutable, content-identified description of one complete semantic system configuration. It references all mutable and locked semantic inputs required to reproduce that configuration.

Write the snapshot as $s(\theta)$. Its digest is

$$D_s = H(\text{canonical}(s)),$$

where H is a cryptographic hash and `canonical` is a stable serialization.

Execution policy such as worker count or UI refresh rate must not enter the semantic snapshot unless it changes observable system behavior.

Existing example RagOpt snapshots already distinguish locked and mutable assets and include dimensions. OptKit preserves this idea but allows variable values to be structured rather than only file assets.

5.4 Patch and change support

Motivation An optimizer proposes a change relative to a parent, not a free-floating complete copy. The framework needs a composable representation of that intervention.

Definition A **patch** δ is an immutable set of typed changes applied to a parent snapshot:

$$\theta' = \theta \oplus \delta.$$

The **support** of a patch is the set of variables it changes:

$$\text{supp}(\delta) = \{j : \theta'_j \neq \theta_j\}.$$

Current RagOpt candidates enforce

$$|\text{supp}(\delta)| = 1.$$

OptKit does not make that a universal law. It provides it as a candidate policy named, for example, `ExactlyKChanges(1)`. GEPA-like or MIPRO-like strategies may eventually propose multi-variable patches.

Worked example

```
api_version: optkit-patch/v1
parent_snapshot: sha256:...
changes:
- variable: answer.grounding_prompt
  before:
    artifact_digest: sha256:old
  after:
    artifact_digest: sha256:new
```


5.5 Proposal and candidate

Motivation An LLM or human may propose an invalid change. The framework must distinguish an intention from a successfully materialized, independently verified configuration.

Definitions A **proposal** contains:

- a parent snapshot;
- a requested patch;
- a hypothesis;
- target objectives;
- anticipated regression risks;
- diagnostic evidence references;
- proposer identity;
- optional influence references to prior candidates.

A **candidate** is the result of validating and materializing a proposal. It contains the independently computed child snapshot and verified patch.

This gives a lifecycle:

```
proposal
  ↓ static validation
materialization attempt
  ↓ independent semantic diff
candidate or rejection
```

A candidate has one materialization parent in v1. It may be *influenced by* many prior candidates. This supports GEPA-style combination of lessons without introducing ambiguous multi-parent snapshot merge semantics.

5.6 Case, dataset, and role

Motivation Optimization data is not homogeneous. Diagnostic examples can be inspected freely; development examples can be queried repeatedly; hidden promotion examples must not leak into proposals.

Definitions A **case** is an immutable input instance with a stable ID, payload, groups, and optional product contract metadata.

A **dataset snapshot** is an ordered, content-identified collection of cases.

A **dataset role** determines how data may be used. OptKit defines at least:

- diagnostic: used to understand failures and form hypotheses;
- development: repeatedly visible to the optimizer;
- calibration: used to qualify measurement instruments;
- selection: used to select among finalists with constrained feedback;
- promotion: hidden release qualification data;
- shadow: production-like observation without user-visible deployment.

An **exposure policy** states which actor may see case inputs, trajectories, per-case measurements, aggregate measurements, and judge feedback.

Mathematical interpretation Let $\mathcal{F}_n^{allowed}$ denote all information the optimizer is authorized to know at iteration n . A valid proposal must depend only on that information:

$$\delta_{n+1} \text{ is measurable with respect to } \mathcal{F}_n^{allowed}.$$

In plain language: the next patch may depend on development evidence, but not on hidden promotion answers that the optimizer was never authorized to inspect.

[!IMPORTANT] When a hidden case is revealed and fed back into reflection, it is no longer hidden. OptKit records this as an exposure event and requires a new promotion dataset or a new validation epoch.

5.7 Episode and trajectory

Motivation A final answer omits the behavior that caused it. For multi-turn agents, optimization needs tool calls, retrieval results, intermediate model turns, state transitions, widget choices, errors, and costs.

Definitions An **episode** is one execution of one candidate on one case under one frozen runtime lock, repeat index, and optional random seed.

A **trajectory** is the ordered and causally linked sequence of observations and actions produced during that episode.

A generic trajectory event may be one of:

```
input.received  
model.call.started  
model.message  
model.call.completed  
tool.call  
tool.result  
retrieval.query  
retrieval.evidence  
state.transition  
widget.intent  
widget.rendered  
constraint.check  
error  
episode.completed
```

Each event has a total sequence number for replay and optional parent/span identifiers for causal structure.

POMDP interpretation For a multi-turn agent, let h_t be the visible history at time t . The agent chooses an action

$$a_t \sim \pi_\theta(a \mid h_t),$$

and the environment returns an observation

$$o_{t+1} \sim P(o \mid h_t, a_t).$$

Actions can include text, retrieval, SQL, tool calls, widget intents, or termination. This is why RL terminology is useful for trajectory representation even when the outer optimizer is not doing policy-gradient training.

5.8 Intervention check

Motivation Changing a configuration file does not prove that runtime behavior used it. Coinvault learned this when a changed default result depth was overridden by an explicit model-supplied tool argument.

Definition An **intervention check**, also called a manipulation check, determines whether a proposed change was applicable and actually exercised in a specific episode.

It returns at least:

applicable	yes/no
exercised	yes/no/unknown
checks	named check results
evidence_refs	supporting trajectory/artifact references

For patch δ and trajectory τ , write

$$I(\delta, \tau) \in \{\text{exercised, not exercised, not applicable, unknown}\}.$$

Worked example: default result depth Suppose the candidate changes the default from 5 to 8. The model calls:

```
{"tool": "knowledge_search", "limit": 5}
```

The configured default is 8, but the effective limit source is `model_argument`. The intervention report is:

```
applicable: true
exercised: false
checks:
  configured_default_matches_candidate: pass
  fallback_default_selected: fail
  effective_limit_equals_candidate: fail
```

The correct outcome is `treatment_not_exercised`, not “the candidate produced no quality gain.”

5.9 Product contract

Motivation Some behaviors are invalid regardless of a judge score. A forbidden route, unauthorized evidence, unresolved citation, or malformed terminal state should not be compensated by fluent prose.

Definition A **product contract** is a deterministic or auditable predicate over an episode and its artifacts. It produces a named check result with status, severity, and evidence.

For checks $c_1, \dots, c_q$, an episode contract may be

$$C(x, \tau) = \bigwedge_{j=1}^q c_j(x, \tau).$$

OptKit distinguishes:

- **candidate rules**, evaluated before execution;
- **intervention checks**, confirming the patch was exercised;
- **episode contracts**, validating runtime behavior;
- **selection gates**, deciding what evidence permits advancement or release.

This prevents the overloaded word “constraint” from hiding when and where a rule applies.

5.10 Construct, instrument, measurement, and epoch

Motivation A number has no stable meaning without a property being measured and a procedure that produced it.

Definitions A **construct** is the abstract property of interest, such as evidence faithfulness, answer relevance, route compliance, cost, or widget usefulness.

A **measurement instrument** is a procedure that consumes evidence and produces an observation about a construct. It may be deterministic code, an LLM judge, a human panel, or a composition of these.

A **measurement** is one persisted result from an instrument. It includes:

- construct identity;
- subject identity;
- value and value kind;
- applicability;
- measurement contract digest;
- protocol/instrument digest;
- uncertainty where available;
- evidence references;
- diagnostics and failure state.

A **measurement epoch** is the identity of the construct operationalization and procedure under which values are comparable. A practical epoch key is:

$$e = H(\text{construct contract, instrument protocol, parser, aggregator}).$$

Measurements from different epochs are not aggregated by default.

Worked example

```
construct_id: faithfulness
subject: episode:coinvault/.../feedback-compare-morgan-peace/challenger/0
value:
  kind: ratio
  number: 1.0
applicability: applicable
measurement_contract_digest: sha256:...
protocol_digest: sha256:evaluator-v8...
evidence_refs:
  - artifact:admitted-evidence-ledger
  - artifact:final-answer
uncertainty: null
```

If evaluator-v10 changes claim extraction or zero-claim behavior, it creates a new epoch. OptKit either re-measures both candidates under v10 or stores an explicit bridge/calibration result. It does not silently mix the values.

5.11 Objective vector and feasible set

Motivation Production systems have multiple desired properties and non-negotiable constraints. A single weighted score makes it too easy for one gain to hide a catastrophic regression.

Definitions Let the expected measurement vector be

$$\mu(\theta) = \mathbb{E}_{x,\tau}[M(\tau)].$$

An **objective** identifies a construct, direction, aggregation, population, and role in search. The objective vector may include:

$$J(\theta) = (\text{faithfulness, relevance, } -\text{cost, } -\text{latency}).$$

The **feasible set** contains configurations satisfying required rules:

$$\mathcal{F} = \{\theta \in \Theta : g_j(\theta) \leq 0, j = 1, \dots, p\}.$$

Optimization is then

$$\max_{\theta \in \mathcal{F}} J(\theta),$$

where J is generally vector-valued.

Coinvault’s zero-failure, contract-valid, and faithfulness-floor requirements define feasibility. Target faithfulness improvement is considered only after feasibility.

5.12 Trial and experimental design

Motivation The same candidate can be evaluated with different experimental designs: paired A/B, repeated paired runs, a tournament, an ablation, a shadow deployment, or a judge-only replay.

Definition A **trial** is a frozen experimental design that expands candidates, cases, repeats, seeds, ordering rules, instruments, and budgets into episodes and comparisons.

A paired trial matches incumbent and challenger on case and repeat:

$$\Delta_{i,r}^{(m)} = M_m(\tau_{i,r}^{challenger}) - M_m(\tau_{i,r}^{incumbent}).$$

The group mean is

$$\bar{\Delta}_G^{(m)} = \frac{1}{|G|} \sum_{i \in G} \Delta_i^{(m)}.$$

Missing pairs remain explicit. They do not disappear from denominators.

[!NOTE] Pairing can be understood as a coupling: two stochastic executions are deliberately linked by the same case, runtime lock, and repeat index so nuisance variation is reduced and the comparison has a clearer interpretation.

5.13 Optimizer and history

Motivation The framework must support human proposals today and automated reflection later without changing the execution model.

Definition Let H_n be the authorized campaign history after n observations. An **optimizer** is a proposal policy

$$Q_A(d\delta \mid H_n)$$

that returns one or more proposed patches.

Examples include:

- a human operator;
- one-coordinate search;
- random or grid search;
- Bayesian optimization;
- OPRO-like prompting over prior solutions and scores;
- MIPRO-like multi-module proposal and surrogate search;
- TextGrad-like textual credit assignment;
- GEPA-like reflection with a Pareto archive;

- an RL learner producing a new policy snapshot.

The optimizer receives a redacted `OptimizerView`, not unrestricted storage access. Exposure policy is enforced by construction.

5.14 Pareto archive

Motivation Two candidates may improve different objectives. Discarding every candidate except the one with a scalar score loses complementary information.

Definition Candidate a dominates candidate b when it is no worse on every search objective and strictly better on at least one:

$$a \succ b \iff (\forall k, J_k(a) \geq J_k(b)) \wedge (\exists k, J_k(a) > J_k(b)).$$

The **Pareto frontier** is the set of non-dominated candidates.

The archive is a search structure, not an automatic deployment rule. A candidate on the development Pareto frontier may still fail hidden promotion constraints.

5.15 Search decision versus promotion decision

Motivation The candidate most useful for further exploration is not necessarily safe to deploy.

Definitions A **search decision** determines which candidates receive more evaluation or influence future proposals.

A **promotion decision** determines whether a candidate may advance to another data role or production.

They must be implemented by separate policies and may use different evidence. Coinvault’s grounding candidate illustrates why: it supplied strong component evidence for search but failed the release gate.

5.16 Campaign

Motivation All prior concepts need a durable container that persists across proposals, trials, failures, restarts, reviews, and promotions.

Definition An **optimization campaign** is a versioned process

$$\mathcal{C} = (S, \Theta, D, M, O, A, P, B, H),$$

where:

- S is the parameterized system adapter;
- Θ is the search space;
- D is the role-tagged data;
- M is the instrument and contract catalog;
- O is the objective and gate catalog;
- A is the optimizer policy;
- P is the adaptive phase plan;
- B is the budget policy;
- H is the append-only history.

A campaign contains candidates, trials, episodes, measurements, comparisons, optimizer observations, reviews, and decisions.

6. OptKit design principles

6.1 Optimize executable systems, not strings

Prompts are values of variables. They are not the universal system model.

6.2 Preserve native product artifacts

OptKit stores standardized envelopes and references. Coinvault session traces and rag-ttc session archives remain authoritative for product detail. Generic projections must never discard evidence required for audit.

6.3 Separate semantic identity from execution policy

Candidate identity includes behavior-changing inputs. Workers, UI settings, and ordinary retry mechanics are recorded but do not alter candidate identity unless they affect semantics.

6.4 Separate search from measurement

Optimizers propose. Instruments measure. Promotion policies decide. No component silently assumes all three authorities.

6.5 Separate component evidence from release qualification

A mechanism can be supported while a release is rejected. Both facts must be preserved.

6.6 Hard constraints do not trade against soft gains

Authorization, route legality, evidence integrity, contract validity, protected-case safety, and hidden-data policy are not terms in a weighted sum.

6.7 Make adaptive plans honest

An adaptive optimizer cannot enumerate future candidates before observing results. The plan should display known work, conditional transitions, bounds, and stop policies rather than pretending the future is a static DAG.

6.8 One journal, many views

Plan, live, and historical UIs are projections of one event history. They are not separate reporting systems.

6.9 Local-first, rebuildable projections

The first implementation uses a local append-only journal and content-addressed artifacts. SQLite or another database may index the history for UI and analysis, but it is rebuildable and not the authority.

6.10 No automatic production apply in v1

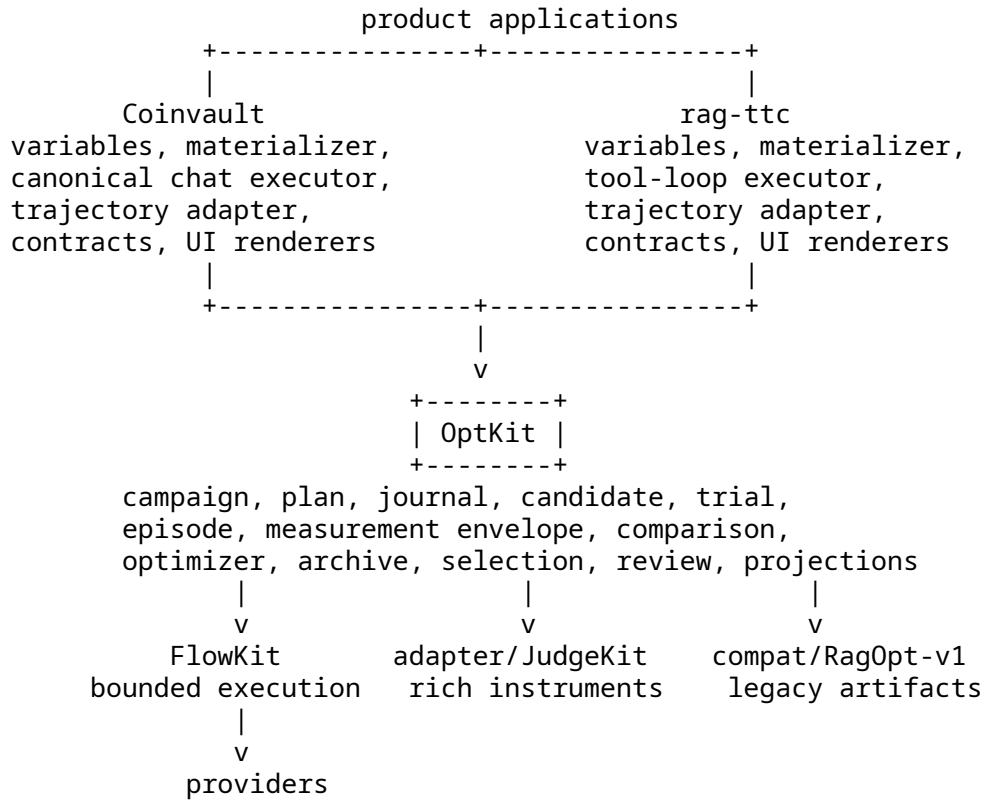
OptKit may produce a signed or digested promotion plan. Product deployment remains an explicit human-authorized action outside the campaign runtime.

6.11 Laws before extension points

Interfaces can change. Invariants such as replay determinism, epoch isolation, and hidden-set non-interference are the durable core.

7. Target repository and dependency architecture

7.1 Module boundaries



RagKit remains beside this stack and is imported by product/RagOpt adapters, not by OptKit core.

7.2 Dependency rules

1. judgekit must not import OptKit or product packages.
2. flowkit must not import OptKit.
3. ragkit must not import OptKit.
4. OptKit core imports no product package.
5. optkit/adapter/judgekit may import JudgeKit.
6. optkit/adapter/flowkit may import FlowKit.
7. Coinvault and rag-ttc import OptKit and register product adapters.
8. A compatibility reader may import RagOpt v1 schemas, but OptKit artifacts do not require RagOpt at runtime.
9. Product-native trajectory payloads remain opaque to generic core code.
10. UI product extensions consume stable projection interfaces rather than internal store structs.

7.3 Proposed package tree

The first release should keep package count comprehensible:

```
optkit/  
  pkg/  
    identity/    canonical IDs, digests, semantic/runtime locks  
    artifact/    content-addressed artifact references and stores  
    space/       variables, domains, values, snapshots, patches  
    candidate/   proposals, materialization, lineage, candidate rules
```


dataset/	cases, snapshots, roles, exposure policies and ledger
episode/	episode specs, trajectory events, terminal results
measurement/	generic measurement envelope and instrument interface
experiment/	trials, designs, pairing, comparison and statistics
optimizer/	proposal policies, optimizer views, reflection packets
archive/	candidate archive and Pareto projections
selection/	stage gates, component decisions, promotion decisions
review/	blinded review and human decision artifacts
campaign/	campaign spec, plan, state, commands and coordinator
journal/	append-only events, replay, leases and reconciliation
projection/	rebuildable read models for CLI, TUI and web
adapter/	
flowkit/	
judgekit/	
compat/	
ragoptv1/	
cmd/	
optkit/	
ui/	
api/	
web/	optional generic static application
internal/	
canonicaljson/	
fstxn/	
testlab/	

[!NOTE] Package names describe stable responsibilities, not every possible concept. Do not create one package per struct. During implementation, identity and artifact may be internal until more than one public package needs them.

7.4 What remains in RagOpt

The existing RagOpt repository should remain readable and buildable during migration. The recommended end state is:

- RagOpt v1 is frozen as the legacy paired-experiment format;
- OptKit contains compat / ragoptv1 readers and importers;
- a future lightweight ragopt layer may provide RAG-specific variables, objectives, and trial presets using OptKit and RagKit;
- old run directories are referenced, not rewritten.

This avoids a high-risk module rename and preserves evidence custody.

Part II — The OptKit domain model

8. Vocabulary: the terms an implementation must keep distinct

The easiest way to produce an unusable optimization framework is to call every number a metric, every run an experiment, and every changed file a candidate. This section defines the terms used throughout OptKit.

8.1 System definition

Motivation. A snapshot needs to say which product knows how to interpret it.

Definition. A **system definition** identifies a product adapter and the schema of configurations it accepts.

```

type SystemRef struct {
    Name          string // e.g. "coinvault.admin-chat"
    AdapterSchema string // e.g. "coinvault.opt-adapter/v1"
}

```

Example. Coinvault and RAG-TTC can both expose a variable called `tool.search.description`, but their materializers and runtime semantics differ. `SystemRef` prevents accidental cross-materialization.

8.2 Variable

Motivation. A treatment switch that grows one branch per experiment does not scale and obscures the search space.

Definition. A **variable** is a named coordinate in the system configuration together with its value domain, validation rules, documentation, and sensitivity metadata.

```

type VariableSpec struct {
    ID          VariableID
    Name        string
    Description  string
    Domain      DomainSpec
    Tags        []string
    Sensitivity  Sensitivity
}

```

A variable describes *what values are valid*. Product code separately binds those values to runtime construction and intervention probes.

Examples.

<code>coinvault.knowledge.default_results</code>	integer in [1, 8]
<code>coinvault.answer.grounding_prompt</code>	text artifact
<code>coinvault.knowledge.reranker</code>	structured configuration
<code>rag_ttc.tool.search.description</code>	YAML/text artifact

8.3 Domain

Motivation. Integers, enums, prompts, model references, and structured plans cannot share one naive type.

Definition. A **domain** defines the admissible values of a variable and their canonical encoding.

Common domain kinds:

```

boolean
bounded integer
bounded real
finite choice
enum with metadata
text artifact
JSON/YAML schema value
model/profile reference
subprogram reference
opaque product artifact

```

A domain may support enumeration or sampling, but those capabilities are optional. A free-form prompt domain cannot be exhaustively enumerated.

```

type Domain interface {
    Kind() string
    Canonicalize(raw json.RawMessage) (Value, error)
    Validate(value Value) error
}

```

```
Describe() DomainSpec
}
```

8.4 Value and value reference

Motivation. Small values can be inline; prompts or schemas should be content-addressed artifacts.

Definition. A **value** is the canonical semantic value assigned to a variable. A **value reference** stores either canonical inline JSON or an artifact digest.

```
type ValueRef struct {
    Kind      string
    Inline    json.RawMessage `json:",omitempty"`
    Artifact  *artifact.Ref      `json:",omitempty"`
    Digest    Digest
    MediaType string
}
```

The digest must be computed from semantic canonical bytes, not a Go map's accidental iteration order.

Fundamentals — What “content-addressed” means.

An object is content-addressed when its identity is derived from canonical bytes representing its meaning. If two independently created prompt artifacts have identical canonical bytes, they receive the same digest. If one semantic byte changes, the digest changes. This makes equality, caching, provenance, and replay checks explicit. Canonicalization is part of the contract: unstable map ordering, timestamps, and display-only labels must not leak into semantic digests.

8.5 Snapshot

Motivation. An experiment needs an immutable identity for the complete system configuration.

Definition. A **snapshot** is a total assignment from declared variable IDs to canonical values for one system definition.

$$\theta : V \rightarrow \text{Value}.$$

```
type Snapshot struct {
    APIVersion string
    ID         SnapshotID
    System     SystemRef
    Values     map[VariableID]ValueRef
    Labels     map[string]string
    Digest     Digest
}
```

Mutability does not belong intrinsically to the snapshot. A campaign's search-space declaration says which variables may change. This is more general than current RagOpt snapshots, whose assets are permanently divided into locked and mutable lists.

Example. `answer.grounding_prompt` may be mutable in a grounding campaign and locked in a later routing campaign. The same snapshot can participate in both.

8.6 Environment fingerprint

Motivation. The same snapshot can behave differently after a code, model, corpus, or index change.

Definition. An **environment fingerprint** is a content-addressed declaration of execution-affecting external identities that are not represented as snapshot variables.

```

type EnvironmentFingerprint struct {
    System      SystemRef
    CodeRevision string
    Dimensions  map[string]string
    ArtifactRefs []artifact.Ref
    Digest      Digest
}

```

For Coinvault, dimensions include the answer/judge runtime identities, bundle identity, source locks, authorizer/evidence-ledger identities, and canonical runtime adapter version. For RAG-TTC, they include the selected profile digest, model identity, source digests, index manifest, and corpus digest.

8.7 Search space

Motivation. A snapshot describes one point. An optimizer needs to know which directions it may move.

Definition. A **search space** selects variables from a system schema, supplies domains or local restrictions, and states structural candidate rules.

```

type SearchSpace struct {
    System      SystemRef
    Variables   []VariableSpec
    CandidatePolicy CandidatePolicy
    Digest      Digest
}

type CandidatePolicy struct {
    MinimumChangedVariables int
    MaximumChangedVariables int
    AllowedCombinations     []CombinationRule
}

```

Current RagOpt behavior is expressed as:

```

minimum changed variables = 1
maximum changed variables = 1

```

A future MIPRO-like campaign may permit multiple prompt coordinates, while a safety-sensitive Coinvault campaign can continue to require one.

8.8 Assignment and patch

Motivation. A candidate should record the exact intervention, not just the resulting snapshot.

Definition. An **assignment** changes one variable from an expected old value to a new value. A **patch** is an ordered, content-addressed set of assignments against one base snapshot.

```

type Assignment struct {
    Variable VariableID
    Before   ValueRef
    After    ValueRef
}

type Patch struct {
    APIVersion  string
    ID          PatchID
    BaseSnapshot SnapshotID
    BaseDigest  Digest
    Assignments []Assignment
}

```

```

    Digest          Digest
}

```

Applying a patch is written

$$\theta' = \theta \oplus \delta.$$

The operation must fail if any Before digest differs from the base snapshot. This prevents applying a stale patch to a changed root.

8.9 Candidate

Motivation. A patch alone does not state why it exists or what evidence motivated it.

Definition. A **candidate** binds a validated patch to its materialized child snapshot, proposer identity, hypothesis, expected improvements, regression risks, and diagnostic evidence.

```

type Candidate struct {
    APIVersion    string
    ID            CandidateID
    Parent        SnapshotRef
    Patch         PatchRef
    Child         SnapshotRef
    Proposer      ActorRef
    Hypothesis    string
    Targets       []ObjectiveRef
    RegressionRisks []string
    Evidence       []artifact.Ref
    CreatedAt     time.Time
    Digest        Digest
}

```

A candidate is an immutable proposal. Its evaluation status belongs to campaign history, not inside the candidate bytes.

8.10 Case and dataset manifest

Motivation. A hidden validation example, a diagnostic example, and an online shadow request have different exposure and statistical roles.

Definition. A **case** is a stable product input. A **dataset manifest** is a content-addressed collection of cases plus declared roles, groups, provenance, and access policy.

```

type Case struct {
    ID        CaseID
    Groups    []string
    Payload   artifact.Ref
    Metadata  map[string]string
    Digest    Digest
}

```

```

type DatasetRole string

```

```

const (
    RoleDiagnostic  DatasetRole = "diagnostic"
    RoleDevelopment DatasetRole = "development"
    RoleCalibration DatasetRole = "calibration"
    RoleSelection   DatasetRole = "selection"
    RolePromotion   DatasetRole = "promotion"
)

```

```

    RoleShadow      DatasetRole = "shadow"
)

```

Example. Coinvault’s twelve feedback cases are development cases. A closed held-out corpus is a promotion dataset. Once its detailed failures are shown to the proposer, it is no longer hidden and must be reclassified or replaced.

8.11 Stage

Motivation. Optimization does not use all data with the same visibility or acceptance rule.

Definition. A **stage** is one policy-bounded phase of a campaign. It binds a data role, trial design, evaluator suite, resource budget, reuse policy, exposure policy, and transition rule.

```

type StagePlan struct {
    ID             StageID
    Dataset        DatasetSelector
    Design         TrialDesignSpec
    Measurements   MeasurementPlanRef
    Selection      SelectionPolicyRef
    Resources      ResourcePolicy
    Reuse          ReusePolicy
    Exposure       ExposurePolicy
    OnPass         Transition
    OnFail         Transition
}

```

8.12 Arm

Motivation. “Incumbent” and “challenger” are roles inside a specific trial, not permanent properties of snapshots.

Definition. An **arm** is a named system snapshot/environment pairing participating in a trial.

```

type Arm struct {
    ID           ArmID
    Role         string
    Snapshot     SnapshotRef
    Environment  EnvironmentRef
}

```

A snapshot can be challenger in one trial and incumbent in a later trial.

8.13 Trial design and trial

Motivation. Paired A/B evaluation is one design among several.

Definition. A **trial design** specifies how arms, cases, repeats, order, randomization, and reuse are converted into episode specifications and how resulting measurements are compared. A **trial** is one concrete, content-addressed instantiation of that design.

Built-in designs should eventually include:

```

paired
multi-arm paired
independent samples
sequential/adaptive allocation
online shadow
reproduction

```

The first release needs paired and reproduction designs.

8.14 Episode specification and episode result

Motivation. Planning and execution must be separable so the UI can show future work and resume safely.

Definition. An **episode specification** is the exact planned unit of system execution. An **episode result** records its terminal status, native trajectory reference, resource usage, and failures.

```
type EpisodeSpec struct {
    ID           EpisodeID
    Trial         TrialID
    Stage        StageID
    Case         CaseRef
    Arm          ArmRef
    RepeatIndex  int
    OrderIndex   int
    ExecutionProtocol Digest
    Key          Digest
}
```

```
type EpisodeResult struct {
    Episode      EpisodeID
    Status       EpisodeStatus
    Trajectory    *TrajectoryRef
    Usage        []UsageRecord
    Failure       *Failure
    StartedAt    time.Time
    FinishedAt    time.Time
    Digest       Digest
}
```

8.15 Trajectory

Motivation. A universal transcript schema would either discard product meaning or become an unbounded union of every product event.

Definition. A **trajectory** is the authoritative product-native execution artifact plus a small generic manifest and optional normalized observation streams.

```
type TrajectoryManifest struct {
    Episode      EpisodeID
    NativeRoot    artifact.Ref
    Streams       []StreamRef
    SummaryFacts map[string]ValueRef
    Terminal      TerminalStatus
    StartedAt     time.Time
    FinishedAt     time.Time
    Digest       Digest
}
```

OptKit defines a small optional event vocabulary—`model.call`, `tool.call`, `tool.result`, `retrieval`, `state.transition`, `widget.intent`, `widget.rendered`, `message`, `error`—but native Coinvault time-line/turn databases and RAG-TTC session records remain authoritative.

8.16 Intervention check

Motivation. A changed configuration does not prove that the changed mechanism affected runtime behavior.

Definition. An **intervention check** compares the expected effect of a patch with observed trajectory facts.

```

type InterventionReport struct {
    Episode      EpisodeID
    Patch        PatchID
    Checks       []CheckResult
    Applicable   Applicability
    Exercised    bool
    Artifact     artifact.Ref
    Digest       Digest
}

```

This generalizes Coinvault’s treatment-exercise report.

8.17 Construct, instrument, measurement, and metric

These four terms must remain distinct.

- A **construct** is the abstract property of interest, such as faithfulness.
- An **instrument** is the procedure used to observe it, such as a specific JudgeKit contract and protocol.
- A **measurement** is one result produced by that instrument for one episode or comparison unit.
- A **metric** is a convenient named numeric projection. Metrics are useful, but they are not the foundational record because a float alone loses instrument identity, applicability, and uncertainty.

8.18 Measurement epoch

Motivation. Coinvault candidate histories currently encode evaluator revisions in proposer identities. This allows values from different judge definitions to appear deceptively comparable.

Definition. A **measurement epoch** is the compatibility identity under which measurements may be aggregated:

$$\eta = H(\text{construct contract, protocol, adapter, aggregation, calibration policy}).$$

Changing a material component creates a new epoch. Competing candidates must either be remeasured in the same epoch or compared through an explicitly defined bridge/calibration study.

8.19 Constraint

Motivation. Route validity or authorization should not be represented as a tiny negative reward.

Definition. A **constraint** is a typed predicate whose failure affects feasibility or selection.

```

type ConstraintResult struct {
    ConstraintID string
    Subject      EntityRef
    Status       CheckStatus // pass/fail/NA/unknown/error
    Severity     Severity   // hard/soft/diagnostic
    Evidence     []artifact.Ref
    Instrument    Digest
}

```

8.20 Estimand and estimate

Motivation. “Mean faithfulness” is ambiguous until the population, contrast, missing-data rule, and measurement epoch are stated.

Definition. An **estimand** is the population-level quantity the campaign intends to learn. An **estimate** is a statistical result computed from observed measurements.


```

type Estimand struct {
    ID            EstimandID
    Construct      ConstructRef
    EpochPolicy    EpochPolicy
    Population      SliceSelector
    Contrast        ContrastSpec
    Aggregation     AggregationSpec
    MissingPolicy   MissingPolicy
    Direction       Direction
}

type Estimate struct {
    Estimand      EstimandID
    Point          float64
    Interval       *Interval
    N              int
    Method          string
    Diagnostics    map[string]ValueRef
    Digest          Digest
}

```

For current paired RagOpt evaluation, the estimand is often:

$$\mathbb{E}[m(candidate) - m(incumbent) \mid x \in G].$$

Fundamentals — Define the estimand before the estimator.

The estimand states what question is being asked. The estimator states how observed data answer it. This distinction lets OptKit later switch from a simple mean to a bootstrap interval or hierarchical model without silently changing the scientific question.

8.21 Objective, selection policy, and promotion

An **objective** says what search should improve. A **selection policy** decides whether evidence makes a candidate eligible, ineligible, or reviewable. **Promotion** is an authorized decision to adopt a candidate as a new baseline or deployment input.

Search and promotion must not be the same interface. The strategy that proposes candidates is incentivized to exploit its evaluation signal; the promotion authority must remain independently constrained.

8.22 Archive and Pareto frontier

Motivation. One global “best” candidate can discard specialized but complementary improvements.

Definition. An **archive** retains candidates according to a declared policy. A Pareto archive retains non-dominated candidates. Candidate a dominates candidate b for an objective vector when

$$a \succ b \iff \forall k, J_k(a) \geq J_k(b) \wedge \exists k, J_k(a) > J_k(b).$$

OptKit should support multiple archive policies. A simple objective Pareto frontier and a GEPA-style case-coverage/elites archive are related but not identical and should not be conflated.

8.23 Search strategy

Motivation. Human proposals, coordinate search, random search, and reflective evolution all need the same evidence substrate.

Definition. A **search strategy** consumes an authorized view of history and returns a declarative next decision: propose candidates, allocate trials, wait for input, or stop.

It does not execute product code directly and does not mutate campaign storage.

8.24 Projection

Motivation. Raw event journals are excellent authorities and poor UI query models.

Definition. A **projection** is a rebuildable read model derived from the campaign plan, control journal, and referenced artifacts. Examples include campaign overview, live episode matrix, candidate lineage, measurement table, Pareto archive, exposure ledger, and resource ledger.

8.25 Review

Motivation. A passing gate still needs human/product interpretation.

Definition. A **review** is an attributable annotation or decision made over a frozen evidence packet. Blind review, rubric dimensions, comments, conflicts, and adjudication remain explicit artifacts.

8.26 Supporting terms used throughout the APIs

The previous definitions describe the scientific structure of an optimization campaign. The implementation also needs a small operational vocabulary. These terms are grouped here so later API sketches can use them without silently changing meaning.

Artifact and entity reference **Motivation.** Events and database rows should not embed every prompt, trace, judge report, or review packet. They need stable links to immutable evidence and to typed campaign objects.

Definitions. An **artifact** is immutable content-addressed bytes plus media type, size, and optional sensitivity metadata. An **entity reference** is a typed identifier for a campaign object such as a candidate, trial, episode, instrument, or decision. An artifact answers “where are the bytes?” An entity reference answers “which domain object is this?”

```
type EntityRef struct {
    Kind string
    ID    string
}

type ArtifactRef struct {
    Digest      Digest
    MediaType   string
    Size        int64
    Sensitivity Sensitivity
}
```

Example. A Coinvault episode entity can reference a native session archive, a normalized trajectory, and a provider-usage report as three artifacts. The episode ID remains stable even though those artifacts have different formats.

Actor and authorized view **Motivation.** A reflective optimizer, judge, human reviewer, UI user, and deployment operator should not automatically see the same data.

Definitions. An **actor** is an attributable principal—human or software—with a role and identity. An **authorized view** is the exact campaign history and artifact subset a policy permits that actor to inspect. Constructing the view records exposure when protected data or derived feedback is disclosed.

Example. A GEPA-like reflector may receive development trajectories and critiques but only aggregate pass/fail information from a promotion stage. A human release reviewer may receive the complete promotion packet.

Prepared system and execution protocol Motivation. Applying configuration values is a different operation from running a case. A trial also needs to state how repeats, seeds, ordering, retries, and reuse work.

Definitions. A **prepared system** is the validated, immutable runtime materialization of a snapshot under an environment fingerprint. An **execution protocol** is the semantic rule for expanding a trial into episode specifications, including arm ordering, repeat assignment, seed policy, and reuse policy.

```
type PreparedSystem struct {  
    Snapshot    SnapshotID  
    Environment Digest  
    Manifest    ArtifactRef  
    Digest      Digest  
}
```

Example. Coinvault materialization resolves the grounding-prompt artifact, runtime profiles, bundle identity, and product options without issuing provider calls. The paired protocol then schedules incumbent and challenger for the same case and repeat.

Resource, budget, and reuse policy Motivation. Cost limits and cache reuse influence whether a campaign is safe and whether two measurements mean the same thing, but they are not all candidate semantics.

Definitions. A **resource** is a named consumable such as provider calls, input tokens, output tokens, dollars, or wall-clock slots. A **budget** is an admitted upper bound over a resource and scope. A **reuse policy** declares which existing episode, trajectory, or measurement artifacts may satisfy new work and under which exact identity rules.

Example. An unchanged episode key can permit reuse of a Coinvault native trajectory, while a new JudgeKit epoch can prohibit reuse of the old measurement and schedule remeasurement over the same trajectory.

Command, control event, observation, attempt, and failure Motivation. UI requests, durable campaign facts, high-volume runtime telemetry, retries, and errors have different semantics and must not share one undifferentiated event type.

Definitions. A **command** requests a state transition and may be rejected. A **control event** records an accepted durable fact in the campaign journal. An **observation** is a runtime fact emitted by a product episode, usually stored in a native or high-volume stream. An **attempt** is one physical execution try for an episode specification. A **failure** is a typed, attributable outcome stating its phase, retryability, cause, and referenced evidence.

Example. PauseCampaign is a command. CampaignPauseRequested is a control event. `tool.call.started` is a Coinvault observation. A retry after HTTP 429 creates a second attempt. Exhausted admission creates a resource failure rather than a low quality measurement.

Campaign history, case slice, and trial allocation Motivation. Search strategies need a bounded summary of what happened and a declarative way to request more evidence.

Definitions. **Campaign history** is the append-only sequence of control facts and referenced evidence. A **case slice** is a stable selector over cases and groups, not an eagerly copied list. A **trial allocation** is a strategy request assigning candidates, slices, repeats, instruments, and resources to future trials.

Example. A coordinate strategy can request one paired trial over the comparison slice. A reflective strategy can allocate three proposals to a small development slice before asking for a larger finalist trial.

9. Snapshot, patch, and candidate design in detail

9.1 Why OptKit replaces “treatment mechanism” with variables and bindings

Coinvault’s current treatment code selects one of several mechanisms and constructs runner options through a large branch. That code is correct for the current candidates, but every new optimization coordinate enlarges one central switch and duplicates three concerns:

1. value validation;
2. runtime materialization;
3. proof that the value affected execution.

OptKit separates them:

VariableSpec	→ what values are legal
VariableBinding	→ how a value changes product runtime construction
ExerciseProbe	→ how a trajectory proves the change was active

9.2 Recommended product binding API

The generic space package should not import product runtime types. Product repositories register erased bindings around their own typed configuration builder.

```
type Binding interface {
    VariableID() space.VariableID

    // Apply is deterministic and provider-free. It mutates only the in-memory
    // build model or writes content-addressed prepared artifacts.
    Apply(
        ctx context.Context,
        value space.Value,
        build *BuildContext,
    ) error

    // ExpectedEffect records facts later consumed by the exercise probe.
    ExpectedEffect(
        value space.Value,
        build BuildContext,
    ) (ExpectedEffect, error)
}

type ExerciseProbe interface {
    VariableID() space.VariableID
    Check(
        ctx context.Context,
        expected ExpectedEffect,
        trajectory trajectory.View,
    ) (InterventionReport, error)
}
```

A product can use generic typed helpers:

```
space.BindInt(
    "coinvault.knowledge.default_results",
    1, 8,
    func(cfg *CoinvaultBuild, value int) error {
        cfg.Runner.KnowledgeDefaultResults = value
        return nil
    },
),
```

```

    DefaultResultsProbe{},
)

```

9.3 Applying a patch

Patch application should be pure at the semantic level:

input:

```

    validated base snapshot
    validated patch whose base digest matches
    search-space candidate policy

```

algorithm:

```

    clone the base assignment map
    for each assignment in canonical variable-ID order:
        require current digest == assignment.before.digest
        validate assignment.after against variable domain
        replace value
    validate changed-variable count and combination rules
    compute child snapshot digest

```

output:

```

    immutable child snapshot

```

Pseudocode:

```

func ApplyPatch(
    base space.Snapshot,
    patch space.Patch,
    schema space.Schema,
    policy space.CandidatePolicy,
) (space.Snapshot, error) {
    if patch.BaseDigest != base.Digest {
        return space.Snapshot{}, ErrStalePatch
    }

    values := clone(base.Values)
    changed := map[space.VariableID]struct{}{}

    assignments := canonicalAssignments(patch.Assignments)
    for _, a := range assignments {
        current, ok := values[a.Variable]
        if !ok {
            return space.Snapshot{}, ErrUnknownVariable
        }
        if current.Digest != a.Before.Digest {
            return space.Snapshot{}, ErrBeforeValueMismatch
        }
        variable := schema.MustVariable(a.Variable)
        if err := variable.Domain.Validate(a.After); err != nil {
            return space.Snapshot{}, err
        }
        values[a.Variable] = a.After
        changed[a.Variable] = struct{}{}
    }

    if err := policy.ValidateChangedVariables(changed); err != nil {
        return space.Snapshot{}, err
    }
}

```

```

    }
    return space.NewSnapshot(base.System, values)
}

```

9.4 Candidate identity

The candidate digest should commit to:

```

parent snapshot digest
patch digest
child snapshot digest
proposer identity
hypothesis and expected objectives
regression-risk declarations
diagnostic evidence references
candidate schema version

```

It should not commit to later evaluation results. The same candidate can be evaluated in multiple stages or measurement epochs without changing its identity.

9.5 Worked example: grounding-prompt patch

```

api_version: optkit.patch/v1
id: patch-gec-grounding-v2
base_snapshot_digest: sha256:...
assignments:
  - variable: coinvault.answer.grounding_prompt
    before:
      artifact: sha256:old-prompt
    after:
      artifact: sha256:new-prompt
api_version: optkit.candidate/v1
id: gec-grounded-answer-v2
parent: sha256:parent-snapshot
patch: sha256:patch-gec-grounding-v2
child: sha256:child-snapshot
proposer:
  kind: human
  identity: ragopt-gec-phase5-evidence-only-answer
hypothesis: >-
  Requiring direct clause-level entailment and adjacent citations will reduce
  unsupported comparison claims without degrading answer relevance.
targets:
  - estimand: feedback-comparison-faithfulness-delta
regression_risks:
  - answers may become overly terse
  - useful synthesis may be omitted

```

9.6 Candidate policies as campaign policy

The one-mutation rule should no longer be hard-coded into candidate loading. It becomes a campaign rule:

```

candidate_policy:
  minimum_changed_variables: 1
  maximum_changed_variables: 1
  forbidden_variables:

```

- coinvault.authorization.policy
- coinvault.evidence_ledger.contract

A future reflective campaign may use:

```
candidate_policy:
  minimum_changed_variables: 1
  maximum_changed_variables: 3
  allowed_combinations:
    - all_of:
      - answer.grounding_prompt
      - answer.citation_prompt
```

The framework still computes the actual changed variables independently from the candidate declaration.

10. Execution identity, trials, and episodes

10.1 Episode key

A completed episode may be reused only under exact semantic identity. Define

$$K_e = H(P, \text{stage, trial design, case, arm snapshot, environment, execution protocol, repeat, reuse epoch}).$$

In Go:

```
type EpisodeKeyInput struct {
    PlanDigest          Digest
    StageID             StageID
    TrialDesignDigest    Digest
    CaseDigest          Digest
    SnapshotDigest      Digest
    EnvironmentDigest   Digest
    ExecutionProtocolDigest Digest
    RepeatIndex         int
    ReuseEpoch         string
}
```

ReuseEpoch permits a reproduction stage to forbid reuse while preserving all other identities.

10.2 Trial design interface

```
type Design interface {
    Name() string
    Digest() Digest

    Plan(
        ctx context.Context,
        req PlanRequest,
    ) ([]EpisodeSpec, error)

    Analyze(
        ctx context.Context,
        req AnalyzeRequest,
    ) (ComparisonSet, error)
}
```

PlanRequest includes arms, selected cases, repeats, order policy, random seed, and reuse policy. AnalyzeRequest includes terminal episode results and compatible typed measurements.

10.3 Paired design

For cases $i = 1, \dots, n$, repeats $r = 0, \dots, R - 1$, and two arms, paired design emits

$$2nR$$

episode specifications and pairs them by

$$(i, r).$$

```
func PlanPaired(req PlanRequest) []EpisodeSpec {
    specs := make([]EpisodeSpec, 0, len(req.Cases)*req.Repeats*2)
    order := req.OrderPolicy.Sequence(req.Seed)

    for r := 0; r < req.Repeats; r++ {
        for _, c := range req.Cases {
            for position, armIndex := range order.ForPair(c.ID, r) {
                arm := req.Arms[armIndex]
                specs = append(specs, NewEpisodeSpec(
                    req, c, arm, r, position,
                ))
            }
        }
    }
    return specs
}
```

Current RagOpt compatibility uses fixed incumbent $\rightarrow$ challenger. New campaigns should be able to choose alternating or seeded counterbalanced order to reduce temporal/provider drift.

10.4 Episode runner boundary

```
type SystemAdapter interface {
    System() model.SystemRef
    Schema(ctx context.Context) (space.Schema, error)

    ValidateEnvironment(
        ctx context.Context,
        expected model.EnvironmentFingerprint,
    ) error

    Materialize(
        ctx context.Context,
        req MaterializeRequest,
    ) (PreparedSystem, error)

    RunEpisode(
        ctx context.Context,
        req EpisodeRequest,
        observations ObservationSink,
    ) (trajectory.Manifest, error)
}
```


Materialize must be deterministic and provider-free. It verifies the environment, applies variable bindings, writes prepared configuration artifacts, and returns a manifest. RunEpisode owns product execution and native artifacts.

10.5 Why materialization is separate from execution

The separation supports:

- preflight without provider calls;
- plan visualization of exact candidate diffs;
- one-time validation before multiple episodes;
- deterministic comparison of prepared runtime identities;
- clearer failure attribution;
- caching prepared artifacts without caching stochastic episodes.

10.6 Native trajectory authority

The adapter returns a generic manifest that links to native data. For Coinvault:

```
native episode root
├─ timeline.db
├─ turns.db
├─ outcome.json
└─ optional exported sessionstream frames
```

For RAG-TTC:

```
native episode root
├─ session/
│   └─ turn records
│       └─ runtime artifacts
└─ outcome.json
```

OptKit stores references and digests rather than copying every native row into a universal transcript format.

10.7 Observation streams

A product adapter may emit normalized observations for live UI and generic analysis:

```
type Observation struct {
    Episode      EpisodeID
    Sequence     uint64
    Kind         string
    OccurredAt   time.Time
    SpanID       string
    ParentSpan   string
    Summary      map[string]ValueRef
    NativeRef    *artifact.Ref
}
```

Recommended kinds include:

```
model.call.started / completed
tool.call.started / completed
retrieval.completed
evidence.admitted
state.transition
message.completed
widget.intent
```

```
widget.rendered
constraint.observed
error
```

These events are optional projections, not a replacement for product-native schemas.

10.8 Resource usage scopes

OptKit should distinguish resource scopes so an invalid answer is not rewarded merely because its judge was skipped.

```
type UsageScope string

const (
    UsageSystem      UsageScope = "system"
    UsageEvaluation   UsageScope = "evaluation"
    UsageOptimizer    UsageScope = "optimizer"
    UsageControl      UsageScope = "control"
)

type UsageRecord struct {
    Scope      UsageScope
    Resource    string // provider_calls, input_tokens, USD, etc.
    Amount      decimal.Decimal
    Unit        string
    Provider    string
    Model       string
    Attempt     int
}
```

RAG-TTC's current decision to exclude judge overhead from product cost becomes an explicit UsageEvaluation record rather than discarded information.

11. Intervention checks and product constraints

11.1 Intervention checks are manipulation checks

An optimization result is uninterpretable when the patch was never exercised. OptKit therefore runs intervention checks before attributing an outcome to a candidate.

For a patch with assignments $v_1, \dots, v_k$, the report contains one check set per variable:

$$E(\tau, \delta) = \bigwedge_{j=1}^k E_j(\tau, v_j).$$

Applicability is separate. A routing prompt may apply to every case, while a knowledge-result limit may apply only to cases that actually invoke `knowledge_search`.

11.2 Worked example: default result depth

Expected effect:

```
{
  "variable": "coinvault.knowledge.default_results",
  "configured": 8,
```

```
"source": "default"
}
```

Observed facts:

```
{
  "knowledge_calls": 1,
  "requested_limit": null,
  "effective_limit": 8,
  "effective_limit_source": "default"
}
```

Checks:

knowledge call observed	PASS
configured value matches prepared runtime	PASS
effective value matches 8	PASS
effective source is default	PASS
retrieval result event observed	PASS

If the model explicitly requested `limit=5`, the candidate is `not_exercised`; it is not a quality tie.

11.3 Worked example: prompt digest

The materializer records the expected prompt-suffix digest. The trajectory collector records the runtime digest. The probe compares them:

```
func (PromptProbe) Check(expected ExpectedEffect, tr trajectory.View) Result {
    observed := tr.Fact("runtime.prompt_suffix_digest")
    return EqualDigestCheck(expected.Digest, observed)
}
```

11.4 Constraint pipeline

A product adapter should run cheap deterministic checks before expensive judges:

```
execution terminal?
↓
trajectory schema valid?
↓
intervention exercised?
↓
route/evidence/citation/authorization contracts?
↓
blocking issue?
├ yes → seal failure/constraint results; skip judge
└ no  → run measurement suite
```

Pseudocode:

```
func EvaluateEpisode(ctx context.Context, e EpisodeResult) ([]Measurement, []ConstraintResult) {
    constraints := deterministicConstraints(e)
    constraints = append(constraints, interventionChecks(e)...)

    if AnyBlockingFailure(constraints) {
        return nil, constraints, nil
    }

    measurements, err := measurementSuite.Run(ctx, BuildInstance(e))
    if err != nil {
```

```

        return nil, append(constraints, MeasurementFailure(err)), nil
    }
    return measurements, constraints, nil
}

```

11.5 Constraints are typed and attributable

Every result should include:

```

constraint identity
instrument/checker version
subject entity
status
severity
expected and observed values
evidence references
message suitable for UI

```

Do not place the only explanation inside a human-oriented string. Structured values support filtering and stable UI rendering.

12. Typed measurements and JudgeKit integration

12.1 Why map[string]float64 is insufficient

The current RagOpt projection is convenient for comparisons but loses:

- construct definition;
- unit and direction;
- protocol/evaluator identity;
- applicability;
- measurement status;
- uncertainty;
- evidence and assessment report;
- calibration information;
- compatibility epoch.

OptKit should retain a numeric projection for convenience, but typed measurements are authoritative.

12.2 Measurement record

```
type MeasurementStatus string
```

```

const (
    MeasurementObserved MeasurementStatus = "observed"
    MeasurementMissing  MeasurementStatus = "missing"
    MeasurementFailed   MeasurementStatus = "failed"
    MeasurementInvalid  MeasurementStatus = "invalid"
)

```

```
type Applicability string
```

```

const (
    Applicable      Applicability = "applicable"
    NotApplicable   Applicability = "not_applicable"
    Unknown         Applicability = "unknown"
)

```

```

type Measurement struct {
    APIVersion      string
    ID              MeasurementID
    Subject         EntityRef
    Construct       ConstructRef
    Instrument      InstrumentRef
    Epoch          EpochID
    Status          MeasurementStatus
    Applicability   Applicability
    Value           *MeasurementValue
    Uncertainty     *Uncertainty
    Evidence        []artifact.Ref
    Assessment      *artifact.Ref
    Diagnostics     map[string]ValueRef
    ProducedAt      time.Time
    Digest          Digest
}

```

MeasurementValue should support scalar, integer, Boolean, label, vector, distribution, and interval kinds. Statistical comparison initially requires a numeric scalar projection declared by the estimand.

12.3 JudgeKit adapter

The adapter translates a sealed JudgeKit assessment into one or more OptKit measurements while preserving identities.

```

type JudgeKitAdapter struct {
    Contract *spec.ContractDocument
    Protocol *protocol.Document
    Suite    judgekitSuite
    Version  string
}

```

```

func (a *JudgeKitAdapter) Measure(
    ctx context.Context,
    episode EpisodeView,
) ([]optmeasure.Measurement, error)

```

Mapping:

JudgeKit Construct.ID	→ OptKit ConstructRef
Contract digest	→ Instrument contract identity
Protocol digest	→ Instrument protocol identity
Assessment report digest	→ Assessment artifact reference
Dimension result	→ Measurement value
Evidence-set digest	→ Evidence reference
adapter + aggregation versions	→ measurement epoch

12.4 Deterministic measurements

Not all measurements use LLM judges. Route compliance, tool-call count, citation resolution, result count, latency, and cost are deterministic instruments with their own versioned identities.

A useful rule is:

Every reported value has an instrument identity, even when the instrument is a deterministic function.

12.5 Measurement epoch compatibility

```
type EpochPolicy struct {
    Mode      string // exact, bridge, descriptive-only
    Allowed   []EpochID
    Bridge    *artifact.Ref
}
```

The default is exact. If candidate A was measured by evaluator v8 and candidate B by v10, the comparison fails closed unless both are remeasured in one epoch or a declared bridge is provided.

12.6 Worked example: Coinvault faithfulness

```
construct:
  id: faithfulness
  definition: >-
    Fraction of answer claims supported by admitted evidence under the
    measurement contract's claim and exclusion rules.
  unit: fraction
  direction: maximize
  range: [0, 1]

instrument:
  contract_digest: sha256:...
  protocol_digest: sha256:...
  adapter_version: coinvault-judgekit-adapter/v1

epoch: sha256:...

measurement:
  subject: episode:...
  status: observed
  applicability: applicable
  value:
    kind: scalar
    number: 0.9615
  assessment: artifact:sha256:...
```

12.7 Calibration and judge reliability

JudgeKit's audit and calibration outputs should be linkable from the instrument/epoch. OptKit selection policies may impose requirements such as:

```
calibration report present
minimum extraction recall
maximum expected calibration error
position-swap reliability above threshold
protocol not marked retired
```

The optimizer may use a development judge that is cheap and diagnostic, while promotion uses a separately controlled instrument. OptKit should make that distinction explicit rather than pretending one judge is truth.

13. Estimands, comparisons, and statistics

13.1 Paired estimand

For construct m , group G , and compatible epoch η , define

$$\delta_{i,r}^{(m)} = m_{i,r}^{candidate} - m_{i,r}^{baseline}.$$

The target estimand is

$$\Delta_G^{(m)} = \mathbb{E}[\delta_{i,r}^{(m)} \mid x_i \in G].$$

The current RagOpt mean-delta estimator is

$$\widehat{\Delta}_G^{(m)} = \frac{1}{N_G} \sum_{(i,r) \in G} \delta_{i,r}^{(m)}.$$

OptKit should preserve wins, ties, losses, minimum/worst delta, presence counts, and failure/missingness diagnostics alongside the point estimate.

13.2 Missing-data policy

A comparison must declare what happens when an arm is missing, invalid, not applicable, or measured in the wrong epoch.

Recommended policies:

fail_closed	selection fails when required data are absent
complete_pairs	estimate only complete compatible pairs, but report loss
worst_case	substitute a declared conservative bound
not_applicable	remove only when estimand defines the case as NA

Production promotion should normally use fail_closed for required constructs.

13.3 Statistical intervals

The first release can preserve current deterministic thresholds and simple means, but the type system should allow intervals:

```
type Interval struct {
  Lower    float64
  Upper    float64
  Level    float64
  Method   string
}
```

Later estimators may include paired bootstrap, randomization tests, Bayesian hierarchical estimates, or sequential confidence sequences.

Fundamentals — Adaptive search changes inference.

Testing hundreds of candidates on the same development set and reporting the winner introduces selection bias. An ordinary interval around the selected winner does not account for the search process. Hidden promotion data, exposure accounting, fresh reproduction, and possibly sequential/multiple-testing procedures are part of the statistical design, not administrative ceremony.

13.4 Cost estimates

Costs should be estimands too. For example:

$$\Delta_G^{tokens} = \mathbb{E}[T_{candidate} - T_{baseline} \mid G].$$

Because usage has scopes, a selection policy can optimize system cost while separately enforcing an evaluator-budget ceiling.

13.5 Comparison API

```
type Analyzer interface {
    Analyze(
        ctx context.Context,
        trial TrialView,
        measurements MeasurementReader,
        constraints ConstraintReader,
        estimands []Estimand,
    ) (ComparisonReport, error)
}
```

A comparison report should carry exact subject IDs and input digests so every aggregate can be drilled down to episodes and native artifacts.

14. Selection, archives, review, and promotion

14.1 Lexicographic selection

OptKit should generalize RagOpt's gate ordering as an ordered list of phases. Each phase contains checks and a stop policy.

Fundamentals — Lexicographic does not mean weighted.

Dictionary order compares the first letter before considering the second. Lexicographic selection works the same way: identity and feasibility are decided before target gain; target gain is decided before regression tolerance; cost is considered only after the preceding phases permit it. A large quality improvement therefore cannot compensate for a forbidden route, corrupt custody, or a failed safety contract.

phases:

- id: identity
stop_on_failure: true
- id: feasibility
stop_on_failure: true
- id: target
stop_on_failure: true
- id: regressions
stop_on_failure: true
- id: cost
stop_on_failure: false

```
type SelectionPolicy interface {
    Evaluate(
        ctx context.Context,
        req SelectionRequest,
    ) (SelectionDecision, error)
}
```


14.2 Decision states

Use more than pass/fail:

eligible	all automatic requirements passed
ineligible	one or more blocking checks failed
needs_review	evidence is complete but policy requires human judgment
inconclusive	required evidence is missing or statistically insufficient
invalid	identity/epoch/custody problem

Current RagOpt pass/fail can map into these states during migration.

14.3 Worked example: grounded-answer policy

The current policy becomes:

identity:

- run terminal
- exact plan/policy identity
- complete pairing

hard feasibility:

- every challenger episode completed
- every challenger answer contract valid
- failure rate = 0
- every faithfulness measurement ≥ 0.80

primary target:

- mean paired faithfulness delta on feedback+comparison ≥ 0

regressions:

- per-case faithfulness delta ≥ -0.20
- per-case answer relevance delta ≥ -0.30
- overall mean faithfulness delta ≥ -0.05
- overall mean answer relevance delta ≥ -0.05

cost ordering:

- provider calls
- tool calls
- total system tokens
- duration

A candidate with a large target gain but route failures is ineligible.

14.4 Search archive versus promotion eligibility

A candidate can be useful to search without being deployable. For example, a grounding prompt may strongly improve two target cases but fail hard constraints elsewhere. A reflective strategy can learn from that candidate or merge its lesson, while the promotion policy rejects it.

Therefore:

search archive membership $\neq$ promotion eligibility

14.5 Archive interface

```
type ArchivePolicy interface {  
    Update(  
        ctx context.Context,
```

```

        current ArchiveView,
        observations []CandidateObservation,
    ) (ArchiveUpdate, error)
}

```

Built-ins:

- BestByEstimand for simple coordinate search;
- ObjectivePareto for multi-objective trade-offs;
- CaseElites for candidates that are best on different case subsets;
- AllEvidence for audit-only campaigns.

14.6 Review artifact

```

type ReviewRequest struct {
    Candidate      CandidateRef
    Baseline       SnapshotRef
    EvidencePacket artifact.Ref
    BlindLabels    map[ArmID]string
    Rubric         artifact.Ref
    RequiredReviewers int
}

type ReviewRecord struct {
    Request      ReviewRequestID
    Reviewer     ActorRef
    Decision     string
    Dimensions   map[string]ReviewValue
    Comments     string
    CreatedAt    time.Time
    Digest       Digest
}

```

14.7 Promotion record

Promotion records adoption as a new campaign baseline or an exportable deployment input. It should not directly alter production.

```

type PromotionRecord struct {
    Candidate      CandidateRef
    FromSnapshot   SnapshotRef
    ToSnapshot     SnapshotRef
    Selection       SelectionDecisionRef
    Reviews        []ReviewRecordRef
    Authority       ActorRef
    Purpose        string
    CreatedAt      time.Time
    Digest         Digest
}

```

A product-specific release process may consume this record and apply the patch through its normal reviewed deployment mechanism.

15. Search strategies

15.1 The smallest stable interface

Search strategy APIs become brittle when they expose internal mutable campaign objects. OptKit should give a strategy a filtered, immutable view and require a declarative decision.

```
type Strategy interface {
    Identity() StrategyIdentity

    Next(
        ctx context.Context,
        req NextRequest,
    ) (NextDecision, error)
}

type NextRequest struct {
    Plan          PlanView
    Stage         StageView
    Baseline      SnapshotView
    SearchSpace   SearchSpaceView
    History       AuthorizedHistoryView
    Archive       ArchiveView
    Resources     ResourceSnapshot
    StrategyState *artifact.Ref
}

type NextDecision struct {
    Proposals    []CandidateDraft
    Allocations  []TrialAllocation
    Stop         *StopDecision
    Reasoning    []artifact.Ref
    NewState     *artifact.Ref
}
```

The controller validates and persists the output. The strategy never writes the journal directly.

15.2 Manual strategy

The first strategy waits for a human-authored candidate draft. This reproduces current Coinvault behavior and tests all control-plane semantics without automating diagnosis.

```
state: awaiting proposal
human submits candidate
strategy returns proposal
controller validates/materializes/evaluates
strategy waits for next proposal
```

15.3 Coordinate strategy

A coordinate strategy chooses one variable, proposes or enumerates values, evaluates them, and selects a new baseline according to policy.

For finite domains:

```
for _, variable := range orderedVariables {
    for _, value := range variable.Domain.Enumerate() {
        if value == baseline[variable] { continue }
        propose(oneAssignmentPatch(variable, value))
    }
}
```

```
}
}
```

For text domains, a proposer component supplies candidate values.

15.4 Random and grid strategies

These are deliberately boring but essential test strategies. They verify that the framework is not accidentally hard-coded to human candidates or prompts.

15.5 GEPA-like reflective strategy

A GEPA-like strategy uses rich trajectories and textual feedback to propose prompt updates, then maintains a diverse archive rather than greedily keeping only one winner.

Conceptual loop:

```
select parent candidate(s) from archive
↓
select authorized diagnostic episodes
↓
build reflection packet
↓
LLM diagnoses success/failure and assigns textual credit
↓
LLM emits structured patch proposal(s)
↓
OptKit validates patches against search space
↓
evaluate on development stage
↓
update archive
↓
repeat until budget/stop condition
```

Pseudocode:

```
func (g *GEPASStyle) Next(ctx context.Context, req NextRequest) (NextDecision, error) {
    parent := g.ParentSelector.Select(req.Archive, req.Resources)
    examples := g.ExampleSelector.Select(req.History, parent)

    packet, err := g.PacketBuilder.Build(parent, examples)
    if err != nil { return NextDecision{}, err }

    reflection, err := g.Reflector.Reflect(ctx, packet)
    if err != nil { return NextDecision{}, err }

    drafts, err := g.Parser.ParseAndValidateShape(reflection)
    if err != nil { return NextDecision{}, err }

    return NextDecision{
        Proposals: drafts,
        Reasoning: []artifact.Ref{reflection.Artifact},
        NewState:  g.UpdateState(parent, examples),
    }, nil
}
```

The strategy receives no promotion-set trajectories. Exposure policy enforces this before packet construction.

15.6 MIPRO-like and TextGrad-like strategies

A MIPRO-like strategy can propose instructions and demonstrations across multiple modules, use mini-batch evaluations, and fit a surrogate for candidate selection. A TextGrad-like strategy can represent a product run as a computation graph and propagate textual critiques toward responsible variables.

OptKit does not need to encode either algorithm into its core. It needs:

- multi-variable patches;
- trajectory and intermediate-artifact references;
- variable/module attribution metadata;
- mini-batch trial allocation;
- strategy state artifacts;
- comparable measurement records;
- archive and resource APIs.

15.7 Strategy reproducibility

A strategy identity must include:

```
algorithm/version
model and protocol identities for reflective calls
prompt/artifact digests
sampling seed or explicit nondeterminism declaration
state digest
exposure-view digest
```

The framework should record nondeterminism rather than pretending every proposal is reproducible byte-for-byte.

Part III — Architecture, persistence, and APIs

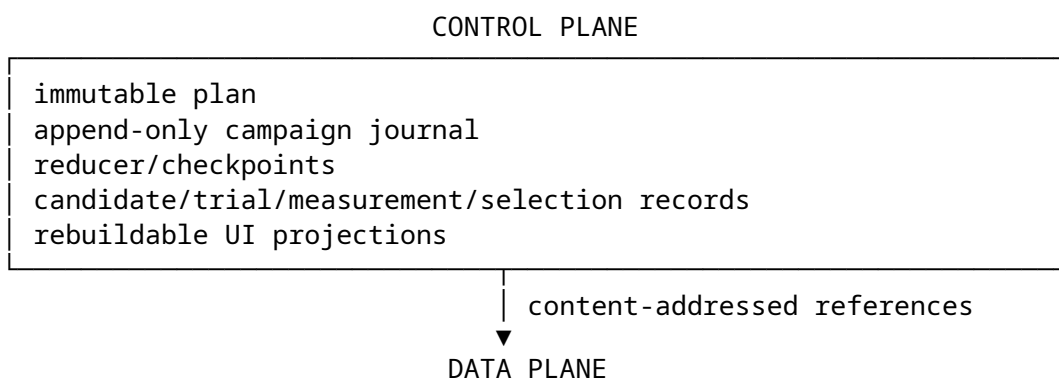
16. Component architecture

16.1 The control plane/data plane distinction

Motivation. Coinvault trajectories are large and product-specific. Campaign state is small, ordered, and decision-critical. Treating them as one storage stream either bloats control replay or strips product meaning.

Definition. OptKit uses two connected planes:

- The **control plane** stores the optimization process: plans, proposals, episode lifecycle, measurements, estimates, decisions, reviews, promotions, and resource state.
- The **data plane** stores native trajectories and other large artifacts produced by products, judges, and optimizers.

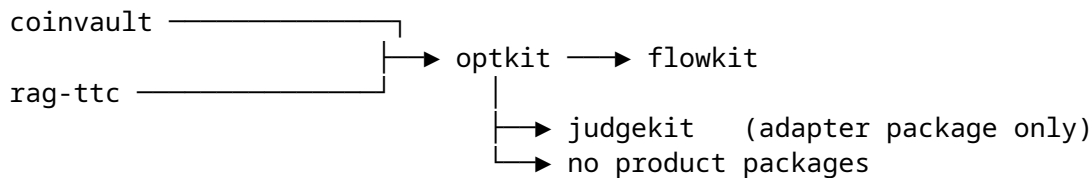


Coinvault timeline.db / turns.db / outcome.json
 RAG-TTC session records
 JudgeKit assessment reports
 reflection packets and LLM proposals
 prompt/schema/config artifacts
 optional normalized observation streams

The control journal never stores raw chain-of-thought merely to support a status screen. It records references to authorized native artifacts and bounded summaries.

16.2 Dependency direction

Recommended module/package dependencies:



ragopt → optkit + ragkit (+ judgekit adapters/presets as needed)

ragkit → flowkit

judgekit → no optkit dependency

flowkit → no optkit dependency

The dependency direction is a testable architectural rule. Add repository boundary tests to prevent optkit from importing Coinvault, RAG-TTC, RagOpt, or RagKit domain adapters.

16.3 Product-owned responsibilities

Products own:

- variable bindings and runtime materialization;
- environment validation;
- canonical episode execution;
- native trajectory artifacts;
- product constraints and intervention probes;
- case payload schemas;
- product-specific UI inspectors;
- deployment/application of approved changes.

16.4 OptKit-owned responsibilities

OptKit owns:

- generic identity and content references;
- snapshots, patches, candidates, and lineage;
- campaign plans and state transitions;
- control journal and replay;
- trial design and episode scheduling;
- typed measurement records and estimands;
- comparisons, selection, archives, and reviews;
- data exposure accounting;
- resource/accounting projections;
- generic CLI/read APIs and UI projections.

16.5 JudgeKit-owned responsibilities

JudgeKit owns:

- construct and measurement-contract definitions;
- evaluator protocol identity;
- evidence instances;
- assessment reports;
- calibration, reliability, and audit.

16.6 FlowKit-owned responsibilities

FlowKit owns:

- bounded execution and retries;
- per-item deterministic caching;
- admission, rates, and budgets;
- step/pipeline metering and reports;
- execution ledgers.

OptKit should translate FlowKit lifecycle/resource facts into campaign events, not reproduce the low-level executor.

17. Event-sourced campaign journal

17.1 Motivation

The user wants to see optimization:

- before it runs;
- live while it runs;
- after it completes;
- later, as part of the history that explains the deployed system.

Three separate reporting pipelines will drift. One immutable campaign journal can support all three as projections.

17.2 Definition: control event

A **control event** is an immutable fact accepted by the campaign controller and appended in a total order.

```
type Event struct {
    APIVersion      string
    CampaignID      CampaignID
    Sequence        uint64
    ID              EventID
    Kind            EventKind
    OccurredAt      time.Time

    Actor           ActorRef
    CorrelationID   string
    CausationID     string

    Entities        []EntityRef
    Payload         artifact.Ref
    Artifacts       []artifact.Ref

    PreviousDigest  Digest
}
```

```
    Digest          Digest
}
```

The payload is a typed, versioned artifact. The envelope remains stable even as payload schemas evolve.

17.3 Why both sequence and causality IDs exist

Sequence provides deterministic replay. CausationID and span/entity references represent causal structure. Concurrent episode completions still have one persisted order, while the UI can display their independent spans.

17.4 Hash chaining

For event e_n :

$$d_n = H(\text{canonical}(e_n \setminus \{Digest\}), d_{n-1}).$$

This makes truncation or mutation detectable. Hash chaining is not access control, but it strengthens artifact custody and diagnosis.

17.5 Stable event families

Keep the top-level vocabulary small. Recommended control events:

```
campaign.created
campaign.plan_validated
campaign.started
campaign.paused
campaign.resumed
campaign.stop_requested
campaign.completed
campaign.failed
```

```
candidate.proposed
candidate.validation_failed
candidate.validated
candidate.materialized
candidate.archived
candidate.rejected
candidate.selected
```

```
trial.planned
trial.started
trial.analysis_completed
trial.completed
trial.failed
```

```
episode.scheduled
episode.started
episode.trajectory_sealed
episode.failed
episode.completed
```

```
intervention.checked
constraint.recorded
measurement.recorded
estimate.recorded
```



```
strategy.invoked
strategy.decision_recorded
strategy.state_updated
archive.updated
```

```
selection.started
selection.decision_recorded
review.requested
review.recorded
promotion.recorded
```

```
exposure.recorded
resource.reserved
resource.consumed
resource.released
resource.exhausted
```

```
projection.checkpointed
plan.amendment_recorded
```

Do not create a new envelope kind for every product tool event. Those belong to trajectory streams or artifact summaries.

17.6 Observation stream versus control journal

High-volume live observations use a separate interface:

```
type ObservationSink interface {
    Publish(context.Context, trajectory.Observation) error
    Seal(context.Context) (trajectory.StreamRef, error)
}
```

A controller may publish selected summaries as control events, but raw token deltas, every model chunk, and every tool payload need not burden campaign replay.

The UI gateway merges:

control journal tail	durable campaign state
trajectory observation tail	live product detail
native artifact readers	completed deep inspection

17.7 Pure reducer

```
type Reducer interface {
    Apply(state State, event Event) (State, error)
}
```

Pseudocode:

```
func Replay(plan Plan, events []Event) (State, error) {
    state := NewState(plan)
    previous := Digest("")

    for index, event := range events {
        if event.Sequence != uint64(index+1) {
            return State{}, ErrSequenceGap
        }
        if event.PreviousDigest != previous {
            return State{}, ErrBrokenHashChain
        }
    }
}
```

```

    }
    if err := VerifyEvent(event); err != nil {
        return State{}, err
    }
    state, err = Reduce(state, event)
    if err != nil {
        return State{}, err
    }
    previous = event.Digest
}
return state, nil
}

```

17.8 State-transition validation

The reducer rejects impossible transitions. Examples:

```
an episode cannot complete before it starts
an unvalidated candidate cannot be materialized
a trial cannot analyze incomplete required episodes
a measurement cannot cite a nonexistent episode
a promotion cannot cite an ineligible decision
an event cannot expose promotion data to an unauthorized strategy
```

17.9 Idempotency

Workers may retry reporting after network/process errors. Commands submitted to the controller should carry stable command IDs. The controller records accepted command IDs and returns the existing event result when the same command is retried.

Exactly-once distributed delivery is not required. Idempotent acceptance plus immutable event IDs is sufficient for the first implementation.

18. Filesystem storage design

18.1 First implementation

Use a local filesystem store, preserving RagOpt's strengths: atomic writes, fsync boundaries, immutable terminal artifacts, and explicit single-writer semantics.

Recommended layout:

```

<root>/campaigns/<campaign-id>/
├── manifest.json
├── plan.json
├── plan.digest
├── status.json
├── journal/
│   ├── events.jsonl
│   ├── head.json
│   └── checkpoints/
│       ├── 00000000100.json
│       └── 00000000200.json

```

```

├── artifacts/
│   ├── sha256/
│   │   ├── ab/
│   │   │   └── cdef...
│   │   └── # canonical blob/envelope
├── entities/
│   ├── snapshots/<digest>.json
│   ├── patches/<digest>.json
│   ├── candidates/<candidate-id>.json
│   ├── trials/<trial-id>.json
│   ├── episodes/<episode-id>.json
│   ├── measurements/<measurement-id>.json
│   ├── estimates/<estimate-id>.json
│   ├── decisions/<decision-id>.json
│   └── reviews/<review-id>.json
├── native/
│   └── <episode-id>/...
│   └── # optional local native roots
├── projections/
│   ├── overview.json
│   ├── plan.json
│   ├── live-matrix.json
│   ├── lineage.json
│   ├── measurements.json
│   ├── selection.json
│   ├── resources.json
│   └── exposure.json
└── index/
    └── optkit.sqlite
    └── # disposable/rebuildable

```

The entities tree is a convenience projection/cache. The authoritative control history is `plan.json` plus the journal and verified artifact blobs.

18.2 Artifact references

```

type Ref struct {
    Digest      Digest
    MediaType   string
    SizeBytes   int64
    URI         string // optional file:, product-native:, s3:, etc.
    Schema      string
    Sensitivity string
}

```

The initial store can support local relative paths and external opaque URIs. A reference must still identify exact bytes and size where possible.

18.3 Atomic append protocol

A filesystem journal append should:

1. acquire the campaign writer lock;
2. read/verify the current head;

3. validate expected sequence and previous digest;
4. write payload/artifacts atomically and fsync them;
5. append one canonical JSON line;
6. fsync the journal file;
7. update head .json atomically and fsync its directory;
8. release the lock.

If a crash occurs between journal append and head update, recovery scans the journal and repairs the head after verifying the chain.

18.4 Single writer and workers

Only the controller writes the control journal. Episode workers write to unique native/artifact directories and submit completion commands to the controller.

```
controller
├─ schedules episode A → worker A → native/A + completion command
├─ schedules episode B → worker B → native/B + completion command
└─ serially accepts completion events
```

This avoids filesystem multi-writer races while allowing execution concurrency.

18.5 Checkpoints

Checkpoints are derived reducer states every configurable number of events. They accelerate startup but are never authoritative. A checkpoint records the event sequence and digest it follows. Replay verifies the checkpoint before applying later events.

18.6 Derived SQLite index

RAG-TTC's proposed transcript warehouse supplies the right principle:

durable native artifacts remain authoritative; SQLite is a disposable query accelerator.

optkit index build should populate normalized tables from the journal and artifact manifests. Product indexers may add trajectory tables or views.

Core tables might include:

```
campaigns
stages
candidates
patch_assignments
trials
episodes
measurements
constraints
estimates
selection_checks
reviews
promotions
resource_usage
exposures
events
artifact_refs
```

Coinvault- or RAG-TTC-specific transcript tables live in product extensions, not the OptKit core schema.

18.7 Retention and sensitivity

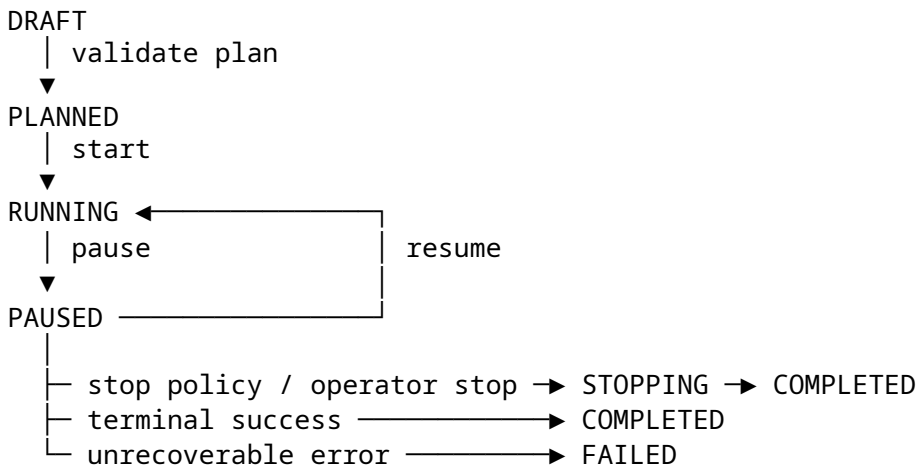
Plans should declare retention classes:

control metadata	retain indefinitely
measurements/decisions	retain indefinitely
native trajectories	policy-dependent
raw reasoning	restricted / optional
provider payloads	restricted
UI caches	disposable

Deletion of permitted native artifacts should leave a tombstone event/reference so historical decisions reveal that supporting bytes are no longer retained.

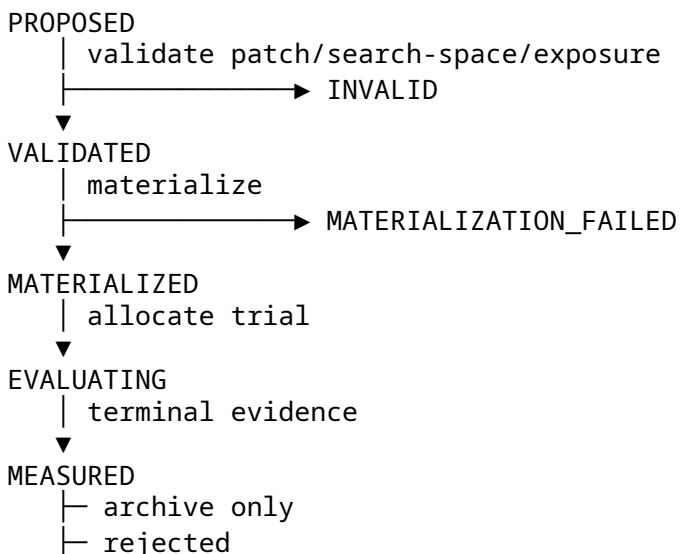
19. Campaign, candidate, trial, and episode state machines

19.1 Campaign state



A completed campaign is immutable. A continuation starts a child campaign with explicit ancestry or records a plan amendment only where the original plan permits it.

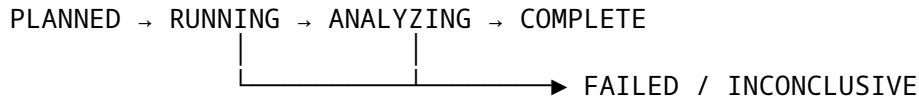
19.2 Candidate state



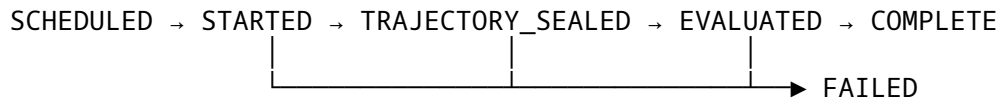
- | selected for next stage
- | reviewable / promoted

State is campaign-relative. The immutable candidate artifact does not change.

19.3 Trial state



19.4 Episode state



Separating TRAJECTORY_SEALED from EVALUATED permits delayed judging or remeasurement under a new epoch without rerunning the product episode.

Important consequence — execution and measurement are different reusable units.

Product execution can be expensive. When a judge protocol changes, OptKit should be able to remeasure an existing compatible trajectory while preserving a new measurement epoch. Conversely, a reproduction stage can forbid reuse of the product episode even when measurements are unchanged.

20. Campaign controller

20.1 Responsibilities

The controller:

- loads and validates the plan;
- replays campaign state;
- calls the search strategy using an authorized history view;
- validates patches/candidates;
- asks the product adapter to materialize snapshots;
- plans trials and episode specs;
- uses FlowKit for bounded execution/admission;
- seals trajectory references;
- invokes intervention checks, constraints, and measurements;
- computes estimates and selection decisions;
- updates archives;
- records reviews/promotions;
- emits events and projections.

20.2 Main loop pseudocode

```

func (c *Controller) Run(ctx context.Context, campaign CampaignID) error {
    state, err := c.LoadState(ctx, campaign)
    if err != nil { return err }

    for !state.Terminal() {
        if err := ctx.Err(); err != nil { return err }

        // 1. Finish or resume already-planned work before asking for more.
    }
}
  
```

```

if pending := state.PendingEpisodes(); len(pending) > 0 {
    if err := c.RunEpisodes(ctx, state, pending); err != nil {
        return c.HandleExecutionError(ctx, state, err)
    }
    state, err = c.ReloadState(ctx, campaign)
    if err != nil { return err }
    continue
}

// 2. Analyze terminal trials whose estimates are not yet produced.
if trials := state.TrialsAwaitingAnalysis(); len(trials) > 0 {
    for _, trial := range trials {
        if err := c.AnalyzeTrial(ctx, state, trial); err != nil {
            return err
        }
    }
    state, err = c.ReloadState(ctx, campaign)
    if err != nil { return err }
    continue
}

// 3. Apply stage selection and transitions.
if decisions := state.SelectionWork(); len(decisions) > 0 {
    for _, work := range decisions {
        if err := c.EvaluateSelection(ctx, state, work); err != nil {
            return err
        }
    }
    state, err = c.ReloadState(ctx, campaign)
    if err != nil { return err }
    continue
}

// 4. Ask the strategy for the next declarative decision.
authorized := c.Exposure.AuthorizeStrategyView(state)
decision, err := c.Strategy.Next(ctx, NextRequestFrom(state, authorized))
if err != nil { return err }
if err := c.RecordAndApplyStrategyDecision(ctx, state, decision); err != nil {
    return err
}

state, err = c.ReloadState(ctx, campaign)
if err != nil { return err }
}
return nil
}

```

20.3 Running episodes with FlowKit

Each episode is keyed work. FlowKit supplies bounded parallelism, retries, resource admission, reports, and optional exact caching.

```

step := flow.Step[trial.EpisodeSpec, trial.EpisodeResult]{
    Name: "optkit-episode",
    Identity: flow.Identity[trial.EpisodeSpec]{
        Kind: "optkit-episode",
        Version: "v1",
    },
}

```

```

    Key: func(spec trial.EpisodeSpec) ([]byte, error) {
        return []byte(spec.Key), nil
    },
},
Policy: flow.Policy{
    Workers: stage.Resources.Workers,
    Retry:   episodeRetryPolicy(stage),
    OnError: flow.Quarantine,
},
Do: c.executeEpisode,
}

```

The OptKit controller still records campaign events around FlowKit execution and verifies whether reuse is allowed by the stage.

20.4 Attempt semantics

An episode can have multiple execution attempts. Attempts are not separate experimental repeats. They are operational retries of one planned episode and should be represented distinctly:

```

repeat index  = statistical replication
attempt index = operational retry

```

Every billable retry consumes resources and is visible in the usage ledger.

20.5 Failure model

Failures should identify owner and phase:

```

type Failure struct {
    Class      string
    Phase      string
    Owner      string // product, optkit, flowkit, judge, environment
    Retryable  bool
    Message    string
    Diagnostics []artifact.Ref
}

```

Suggested classes:

```

environment_drift
materialization_failed
budget_exhausted
execution_timeout
product_runtime_failed
trajectory_invalid
intervention_not_exercised
constraint_failed
measurement_failed
measurement_epoch_mismatch
analysis_incomplete
selection_invalid
operator_cancelled

```

A failed episode remains a first-class observation. It is never silently dropped.

21. Exposure and authorization model

21.1 Why exposure is part of optimization semantics

Reflective optimizers learn directly from detailed failures. Once a held-out case or judge explanation is shown to the proposer, it has influenced future candidates. Calling it “held out” afterwards is mathematically false.

21.2 Actors

Declare actors such as:

```
human proposer
search strategy
reflection model
trial allocator
measurement instrument
selection authority
human reviewer
UI viewer
```

21.3 Exposure record

```
type ExposureRecord struct {
    Actor      ActorRef
    Subject     EntityRef // case, trajectory, measurement, aggregate
    Detail      DetailLevel
    Purpose     string
    Stage       StageID
    AuthorizedBy PolicyRef
    OccurredAt  time.Time
    Digest      Digest
}
```

Detail levels might be:

```
identity_only
aggregate
score_only
structured_diagnostics
redacted_trajectory
full_trajectory
native_artifact
```

21.4 Authorized history view

The controller creates a filtered view before calling a strategy. It should be impossible for a strategy implementation to reach the raw store through the request object.

```
type AuthorizedHistoryView interface {
    Candidates() []CandidateSummary
    Trials() []TrialSummary
    Episodes(selector AuthorizedSelector) []EpisodeSummary
    Measurements(selector AuthorizedSelector) []MeasurementView
    Artifacts(refs []artifact.Ref) AuthorizedArtifactReader
    Digest() Digest
}
```

The view digest is recorded in the strategy invocation event.

21.5 Worked example

development stage:

reflector sees full failed/successful trajectories and JudgeKit diagnostics

selection stage:

reflector sees no cases, trajectories, or measurements

campaign selector sees aggregate compatible estimates

promotion review:

human reviewer sees blinded answer/evidence packets

If promotion failures are later used for reflection, OptKit records that exposure and the next campaign must use a fresh promotion set.

22. Package layout

A practical Go module layout:

optkit/

└─ go.mod	
└─ model/	stable IDs, digests, actors, entity refs, status values
└─ artifact/	content-addressed refs, canonical encodings, blob store API
└─ space/	domains, variables, schemas, snapshots, patches, candidates
└─ dataset/	cases, manifests, roles, slices, exposure metadata
└─ trajectory/	manifests, observations, stream refs, generic facts
└─ measure/	measurements, instruments, epochs, constraints
└─ estimand/	estimands, contrasts, aggregations, missing-data policies
└─ trial/	arms, designs, episode specs/results, paired design
└─ stat/	paired estimates, intervals, diagnostics, comparison reports
└─ select/	lexicographic policy, checks, decisions
└─ archive/	best, Pareto, case-elites policies
└─ search/	strategy interface and built-in manual/random/coordinate
└─ campaign/	plans, stages, state machines, reducer, controller contracts
└─ event/	envelopes, payload registries, verification
└─ store/	
└─ fs/	filesystem journal, artifact store, locks, checkpoints
└─ memory/	deterministic tests
└─ projection/	read-model reducers and query interfaces
└─ review/	blind review packets and records
└─ resource/	campaign usage/admission projection and FlowKit mapping
└─ adapter/	
└─ judgekit/	JudgeKit assessment → OptKit measurement
└─ flowkit/	optional execution/report translation helpers
└─ legacy/	
└─ schema/	neutral helpers only; RagOpt-specific bridge lives in ragopt
└─ cmd/optkit/	validate, inspect, verify, replay, index, serve, tui

22.1 Dependency layering inside the module

model

↑

artifact

```

↑
space / dataset / trajectory / measure
↑
trial / estimand
↑
stat / select / archive / search
↑
campaign / event / store / projection
↑
adapters and cmd

```

Add a package-boundary test or import-lint rule. In particular:

- `model` must not depend on higher packages;
- `space` must not depend on `campaign` or product adapters;
- `projection` reads events/entities but does not execute systems;
- adapters depend inward, never the reverse.

22.2 Avoid a giant `model` package

Only universally stable identifiers and small value types belong in `model`. Domain records should live with their behavior. A giant “all structs” package produces circular concepts and weak ownership.

23. Core API sketches

The following signatures are illustrative, not a requirement to copy names verbatim. The semantics are more important than method spelling.

23.1 Artifact store

```

type BlobStore interface {
    Put(ctx context.Context, mediaType string, r io.Reader) (artifact.Ref, error)
    Open(ctx context.Context, ref artifact.Ref) (io.ReadCloser, error)
    Verify(ctx context.Context, ref artifact.Ref) error
    Exists(ctx context.Context, ref artifact.Ref) (bool, error)
}

```

23.2 Journal

```

type Journal interface {
    Head(ctx context.Context, campaign CampaignID) (event.Head, error)
    Append(ctx context.Context, req event.AppendRequest) (event.Event, error)
    Read(ctx context.Context, campaign CampaignID, after uint64, limit int) ([]event.Event, error)
    Subscribe(ctx context.Context, campaign CampaignID, after uint64) (<-chan event.Event, error)
}

```

23.3 Campaign repository

```

type Repository interface {
    Create(ctx context.Context, plan campaign.Plan) (campaign.Manifest, error)
    LoadPlan(ctx context.Context, id CampaignID) (campaign.Plan, error)
    Replay(ctx context.Context, id CampaignID) (campaign.State, error)
    List(ctx context.Context, filter campaign.Filter) ([]campaign.Summary, error)
}

```

23.4 Product system adapter

```
type SystemAdapter interface {
    System() model.SystemRef
    Schema(context.Context) (space.Schema, error)
    ValidateEnvironment(context.Context, model.EnvironmentFingerprint) error
    Materialize(context.Context, runtime.MaterializeRequest) (runtime.PreparedSystem, error)
    RunEpisode(context.Context, runtime.EpisodeRequest, trajectory.ObservationSink) (trajectory.ObservationSink, error)
}
```

23.5 Product evaluation adapter

```
type EpisodeEvaluator interface {
    InterventionChecks(context.Context, EvaluationRequest) ([]measure.ConstraintResult, error)
    DeterministicConstraints(context.Context, EvaluationRequest) ([]measure.ConstraintResult, error)
    Measurements(context.Context, EvaluationRequest) ([]measure.Measurement, error)
}
```

23.6 Projection store

```
type Reader interface {
    Overview(context.Context, CampaignID, projection.At) (projection.Overview, error)
    Plan(context.Context, CampaignID, projection.At) (projection.Plan, error)
    LiveMatrix(context.Context, CampaignID, projection.At) (projection.LiveMatrix, error)
    CandidateGraph(context.Context, CampaignID, projection.At) (projection.CandidateGraph, error)
    Episode(context.Context, EpisodeID, projection.At) (projection.Episode, error)
    Measurements(context.Context, projection.MeasurementQuery) (projection.MeasurementTable, error)
    Selection(context.Context, CandidateID, StageID) (projection.Selection, error)
    Exposure(context.Context, CampaignID) (projection.ExposureLedger, error)
    Resources(context.Context, CampaignID) (projection.ResourceLedger, error)
}
```

23.7 Product inspector extension

```
type InspectorProvider interface {
    Kinds() []string
    Inspect(context.Context, artifact.Ref, InspectorRequest) (projection.PanelSet, error)
}
```

Coinvault can render sessionstream timelines and widgets; RAG-TTC can render its current evidence/funnel/trace tabs. OptKit UIs remain useful without these extensions but gain product depth when hosted inside a product.

24. Plan schema and adaptive planning

24.1 A plan is not a fake future DAG

An adaptive optimizer cannot know every future candidate. The plan must distinguish:

- **known work**, such as baseline evaluation;
- **bounded repeated policy**, such as “up to six reflective rounds”;
- **conditional transitions**, such as “promotion only if reproduction passes”;
- **resource bounds**, such as maximum episodes or dollars.

24.2 Example plan

```
api_version: optkit.plan/v1
name: coinvault-grounding-and-routing
system:
  name: coinvault.admin-chat
  adapter_schema: coinvault.opt-adapter/v1

baseline: artifact:sha256:...
environment: artifact:sha256:...
search_space: artifact:sha256:...

strategy:
  kind: manual_then_reflective
  identity: artifact:sha256:...
  maximum_rounds: 6
  maximum_candidates_per_round: 4

stages:
- id: development
  dataset:
    manifest: artifact:sha256:...
    role: development
  design:
    kind: paired
    repeats: 1
    order: alternating_seeded
  reuse:
    system_episodes: exact
    measurements: exact_epoch
  exposure:
    strategy: full_trajectory
  selection: artifact:sha256:...
  on_pass: reproduction
  on_fail: strategy_next

- id: reproduction
  dataset:
    manifest: artifact:sha256:...
    role: selection
  design:
    kind: reproduction
    repeats: 1
  reuse:
    system_episodes: forbidden
  exposure:
    strategy: aggregate
  on_pass: promotion
  on_fail: stop

- id: promotion
  dataset:
    manifest: artifact:sha256:...
    role: promotion
  design:
    kind: paired
    repeats: 2
```

```

    exposure:
      strategy: none
      selector: aggregate
      reviewer: blinded_packet
      on_pass: human_review
      on_fail: stop

resources:
  maximum_episodes: 200
  named:
    answer_provider_calls: 1000
    embedding_calls: 500
    judge_provider_calls: 500
    provider_tokens: 5000000

stop:
  maximum_rounds: 6
  maximum_candidates: 24
  stop_when_no_archive_improvement_rounds: 2

```

24.3 Plan amendments

Some long campaigns require a budget extension or human pause. Amendments must be explicit events that state which fields may change. Never rewrite `plan.json` in place.

Safe amendment examples:

```

increase a non-hidden UI retention period
add reviewer identity
extend a budget with separate authorization
pause/resume

```

Unsafe amendment examples that should require a child campaign:

```

change measurement protocol
change search space
replace promotion dataset
alter target estimand
change baseline snapshot

```

Part IV — Visualization and interaction architecture

25. Why UI is part of the architecture

Optimization is difficult to trust when it is represented only by terminal logs and final averages. Users need to understand:

- what the campaign is allowed to change;
- what it plans to do next;
- what is currently executing;
- why a candidate was proposed;
- where a trajectory failed;
- which instrument produced a score;
- which constraints stopped promotion;
- how deployed behavior descends from prior experiments.

The UI should not implement separate business logic. It should consume projections derived from the same event journal used for replay and audit.

25.1 Three temporal modes, one data model

BEFORE plan projection	DURING live projection	AFTER history projection
mutable variables locked environment stage transitions adaptive bounds exposure rules cost upper bounds	active episodes streamed observations budget consumption provisional measurements failures/retries archive updates	candidate lineage paired estimates selection checks reviews/promotions measurement epochs deployment ancestry

25.2 UI clients

Implement projection APIs once and support several clients:

optkit tui	generic local terminal UI
optkit serve	generic read-only HTTP/SSE server
Coinvault web workspace	product-integrated web UI
RAG-TTC admin TUI	product-integrated terminal UI
notebooks/SQL	derived SQLite/read API

The core should not require a browser server to run campaigns.

26. Projection architecture

26.1 Projection definition

Every projection is computed at an event cursor:

```
type At struct {  
    Sequence uint64  
    Digest   Digest  
}
```

A UI response includes the cursor so panels can indicate whether they show one consistent snapshot.

26.2 Projection worker

```
type Projector interface {  
    Name() string  
    Apply(current any, event event.Event, artifacts artifact.Reader) (any, error)  
}
```

Projectors can run:

- synchronously after accepted events for small local campaigns;
- asynchronously from the journal for larger campaigns;
- from scratch during optkit projections rebuild.

26.3 Core projections

Campaign overview

```

type Overview struct {
    Campaign      CampaignSummary
    CurrentStage  StageSummary
    Baseline      SnapshotSummary
    ActiveCandidate *CandidateSummary
    Progress      ProgressSummary
    Resources      ResourceSummary
    LatestDecision *DecisionSummary
    Alerts         []Alert
    At             At
}

```

Plan projection Shows stage graph, mutable variables, locked identities, strategy policy, data roles, exposure rules, and resource bounds.

Live matrix Rows are cases/repeats; columns are arms/candidates. Each cell summarizes episode status, attempt, duration, constraints, measurements, and artifact availability.

Candidate graph Nodes are snapshots/candidates; edges are patches, selection, and promotion relationships.

Episode projection Combines generic lifecycle, observation spans, intervention checks, constraints, measurements, usage, and product inspector links.

Measurement table Groups values by construct and epoch, identifies missing/inapplicable measurements, and exposes assessment/calibration references.

Selection projection Displays ordered phases and checks exactly as executed. It must not recompute the decision in the browser.

Exposure ledger Shows which actors saw which data and warns when a previously hidden set has been consumed.

Resource ledger Shows reserved/consumed resources by scope, provider, model, stage, candidate, and episode.

26.4 Product-specific projections

Product projections enrich generic entities without changing core semantics.

Coinvault examples:

```

knowledge search calls and result-limit source
comparison intent/decomposition
retrieval/reranker waterfall
evidence ledger and citation graph
SQL versus knowledge routing
widget intent/render timeline
sessionstream timeline link

```

RAG-TTC examples:

search query sequence
 retrieval funnel
 hit/evidence/document panes
 agent iterations
 contract and usage panes
 native session path

27. Plan explorer

27.1 Purpose

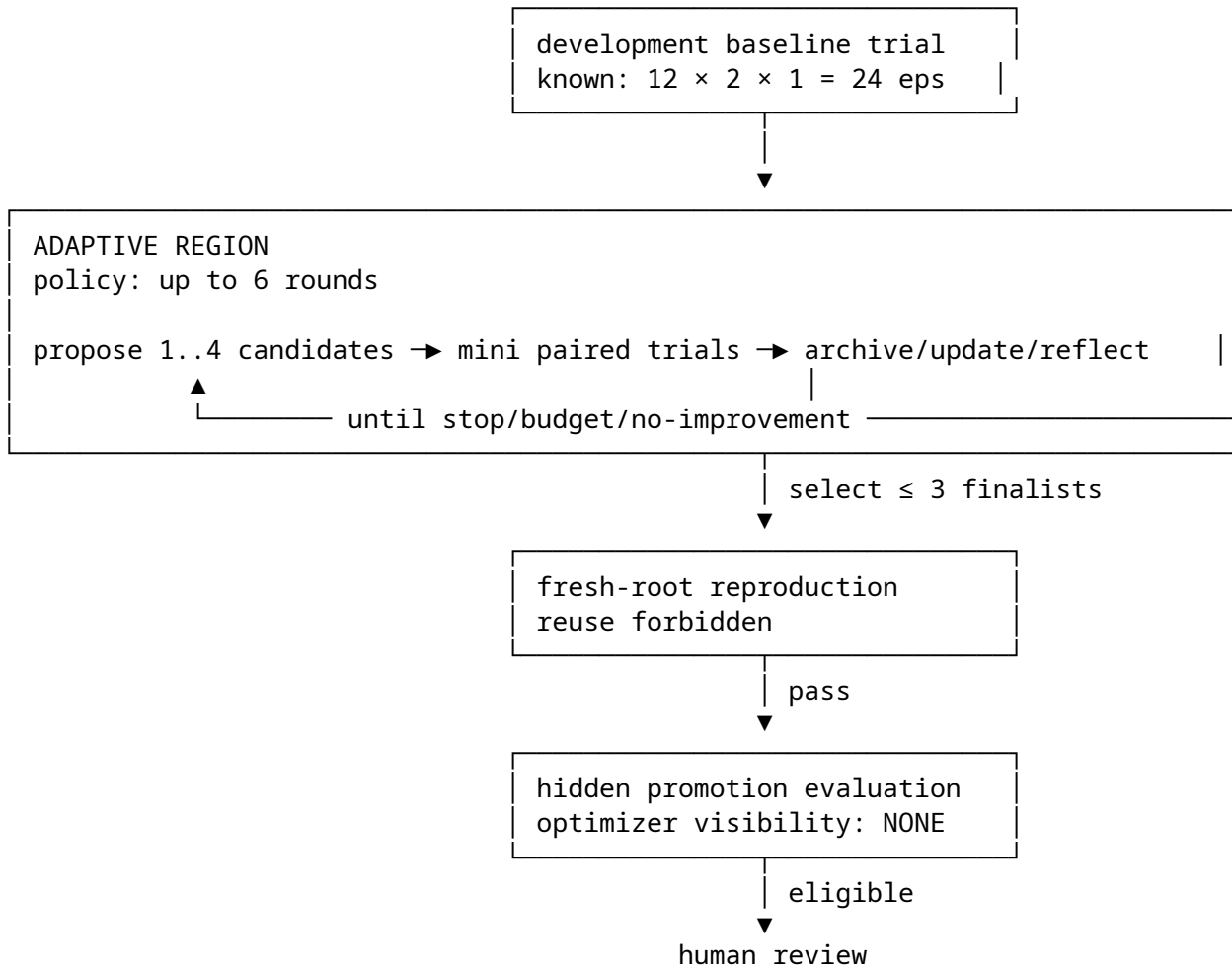
The plan explorer answers:

- What can change?
- What is frozen?
- Which data will be consumed?
- Which future steps are known versus adaptive?
- How expensive can the campaign become?
- Who will see held-out evidence?
- What can cause an early stop?

27.2 ASCII screen: plan overview

OptKit / Campaign Plan			
coinvault-grounding-and-routing		PLAN VALIDATED	sha256:91...
System	coinvault.admin-chat / coinvault.opt-adapter/v1		
Baseline	deployed-2026-08-18 sha256:4c...		
Environment	code=6a3f... bundle=rk-55be... answer=gpt-5.6-luna		
Strategy	manual → reflective-pareto max rounds=6 max candidates=24		
Mutable space			
[OK] answer.grounding_prompt	text artifact		
[OK] answer.routing_prompt	text artifact		
[OK] knowledge.tool_description	structured artifact		
[OK] knowledge.default_results	integer [1,8]		
[OK] knowledge.reranker	choice {off,qwen3-pool12}		
Candidate rule: 1 changed variable per candidate			
Locked boundary			
authorization.policy	sha256:1a...	evidence_ledger.contract	sha256:8d...
corpus/index	rk-55be...	judge promotion epoch	sha256:2f...
Stages			
[1] DEVELOPMENT →pass→ [2] FRESH REPRODUCTION →pass→ [3] PROMOTION → USER			
12 cases		no episode reuse	
full diagnostics		aggregate to strategy	
		hidden, 2 repeats	
		blinded review	
Resource envelope			
episodes ≤ 200		answer calls ≤ 1000	judge calls ≤ 500
		tokens ≤ 5.0M	
estimated lower bound: 24 episodes		upper bound under policy: 200	

27.3 ASCII screen: adaptive stage graph



The adaptive region is shown as a policy box, not fabricated future nodes.

27.4 Plan diff and validation screen

Plan validation
PASS 38 checks
Search space
PASS all variable IDs known
PASS candidate policy permits at least one move
PASS forbidden safety variables absent
Data
PASS development/promotion case IDs disjoint
PASS promotion payload not readable by strategy actor
Measurement
PASS target construct has maximize direction
PASS all required epochs pinned
Resources
PASS worst-case episodes 200 ≤ global ceiling 200
WARN price unavailable for answer-provider tokens; allowed by policy

28. Live campaign console

28.1 Purpose

The live console should answer:

- Is the campaign making progress?
- Which episodes are running or blocked?
- What is the current resource burn?
- Which candidate is being evaluated?
- Did the intervention execute?
- Which failures appeared first?
- What did the strategy just learn/propose?

28.2 ASCII screen: campaign overview

coinvault-grounding-and-routing – RUNNING – seq 184				
Stage DEVELOPMENT round 2/6 candidate gec-routing-v3				
Baseline sha256:4c... Child sha256:a7... Patch: answer.routing_prompt				
Progress				
episodes	17 complete / 24 planned	2 running	5 queued	
measures	14 complete	1 judging	2 blocked	
elapsed	00:18:42	ETA not asserted		
Resources				
answer calls	61 / 216		28%	
embeddings	33 / 192		17%	
judge calls	14 / 72		19%	
answer tokens	188k / 1M		19%	
Recent facts				
14:36:08	feedback-mixed-014/challenger	tool.call	knowledge_search	
14:36:10	feedback-mixed-014/challenger	route contract	PASS	
14:36:11	feedback-compare-gold/challenger	treatment exercised	PASS	
14:36:13	feedback-protected-003/challenger	judge skipped:	contract failure	
Alerts				
! 2 challenger episodes failed required route contract				
! promotion cannot pass unless later retry policy produces valid episodes				

The UI must not promise an ETA. It may show known progress and resource consumption.

28.3 ASCII screen: live episode matrix

Episodes: DEVELOPMENT / candidate gec-routing-v3		
case	incumbent	challenger
feedback-compare-morgan-peace	[OK] F=.46 R=.61	[OK] F=.98 R=.84
feedback-compare-gold-bars	[OK] F=.38 R=.57	J judging...
feedback-mixed-schema	[OK] valid	[X] route_contract
feedback-knowledge-only-004	[OK] valid	► tool:search
feedback-protected-003	[OK] abstain	[X] forbidden route
feedback-ambiguity-002	· queued	· queued
...		

legend [OK] complete ▶ running J measuring [X] failed · queued RETRY retry

Selecting a cell opens the episode inspector.

28.4 ASCII screen: episode trace

```

┌ Episode feedback-knowledge-only-004 / challenger / attempt 1 ───────────
│ status RUNNING      snapshot sha256:a7...      native timeline live
├ Waterfall ───────────
│ 0.00s user.message
│ 0.18s model.call #1 tokens in=2,104
│ 1.44s   └ tool.call knowledge_search
│ 1.47s       query="Morgan dollar Peace dollar differences"
│ 1.49s       requested limit=<none> effective=8 source=default
│ 1.92s       retrieval bm25=20 vector=20 fused=12 reranked=8
│ 1.94s       evidence admitted=8
│ 2.11s model.call #2 tokens in=5,876
│ 3.31s   └ message completed citations=5
├ Intervention ───────────
│ knowledge.default_results expected=8 observed=8/default          PASS
├ Constraints ───────────
│ required knowledge route PASS  citation resolution pending
└──────────────────────────────────────────────────────────────────────────

```

28.5 ASCII screen: strategy activity

```

┌ Reflector / round 2 ───────────
│ authorized view sha256:73...  6 failed + 4 successful development trajectories
├ Diagnosis summary ───────────
│ Evidence was retrieved on all comparison failures. Two answers routed to SQL
│ after the routing prompt interpreted "schema" as structured inventory data.
│ Grounding behavior remained improved when the knowledge route was selected.
├ Proposed patch ───────────
│ variable answer.routing_prompt
│ parent   sha256:5c...
│ child    sha256:9b...
│ rationale Distinguish document-schema questions from inventory-record schema
├ Validation ───────────
│ one-coordinate rule PASS  forbidden variables PASS  prompt parser PASS
└──────────────────────────────────────────────────────────────────────────

```

The reflection artifact is retained and attributable. It is not treated as ground truth.

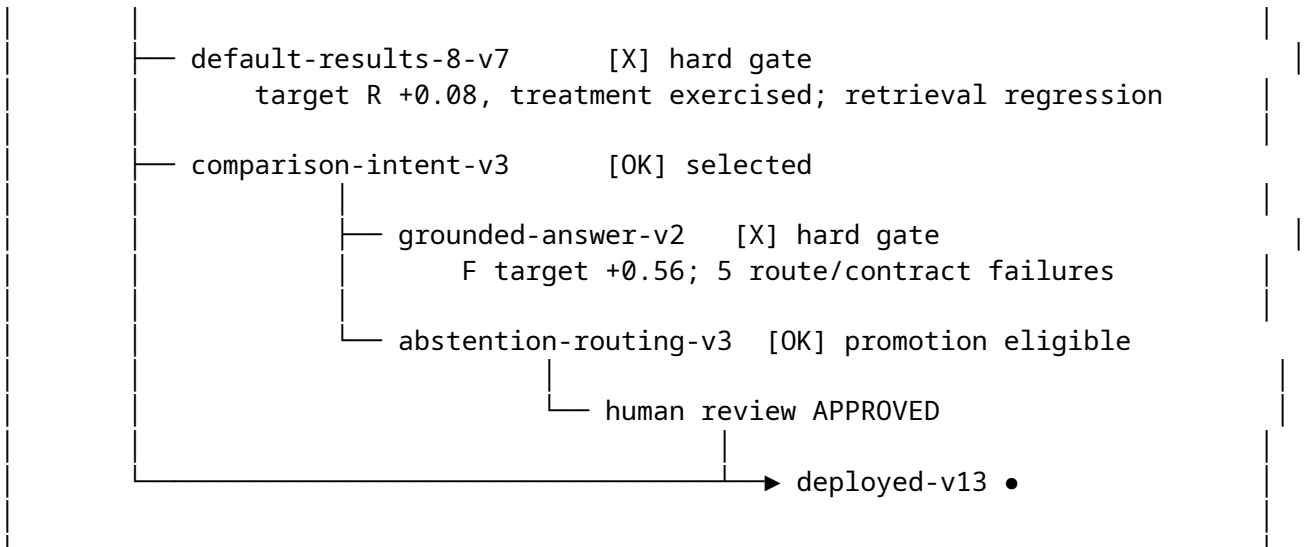
29. Results and history explorer

29.1 Candidate lineage

```

┌ Candidate lineage ───────────
│
│ deployed-v12  • sha256:4c...
└──────────────────────────────────────────────────────────────────────────

```



Selecting an edge shows the patch diff; selecting a decision shows exact checks and evidence.

29.2 Candidate comparison

grounded-answer-v2 vs parent – development					
Construct	Parent	Candidate	Δ mean	Worst Δ	Epoch
faithfulness / comparison	.419	.981	+.562	+.541	9f...
answer relevance / comp.	.590	.865	+.275	+.230	12...
faithfulness / all	.812	.846	+.034	-.180	9f...
route contract rate	1.000	.583	-.417	-1.000	det:4...
system provider calls	2.21	2.08	-.13	-2	det:1...
Selection					
IDENTITY	PASS				3/3
HARD	FAIL	route/contract validity			4/6
TARGET	NOT EVALUATED				
REGRESSIONS	NOT EVALUATED				
COST	NOT EVALUATED				
Final	INELIGIBLE				

29.3 Measurement epoch warning

Measurement compatibility warning

faithfulness values cannot be combined

candidate A: epoch sha256:judge-v8... contract=3f... protocol=91...

candidate B: epoch sha256:judge-v10... contract=7a... protocol=b2...

Available actions

- remeasure both trajectories under one epoch
- attach a validated epoch bridge
- display descriptively without a selection comparison

29.4 Pareto archive

Search archive / objective Pareto				
candidate	faithfulness Δ	relevance Δ	calls Δ	eligibility
grounded-answer-v2	+.562	+.275	-.13	ineligible
routing-v3	+.081	+.140	-.05	eligible
reranker-pool12-v1	+.110	+.032	+.77	eligible
tool-description-v2	+.040	+.190	-.22	eligible
frontier members: routing-v3, reranker-pool12-v1, tool-description-v2				
note: grounded-answer-v2 retained as diagnostic evidence, not frontier				

For more than two objectives, a table plus filters is often more useful than a scatter plot.

29.5 Exposure ledger

Data exposure			
dataset/stage	actor	detail	status
development	reflector-v2	full trajectory	expected
development	human proposer	native artifact	expected
reproduction	reflector-v2	aggregate	expected
promotion-2026q3	reflector-v2	none	protected
promotion-2026q3	selector	aggregate	expected
promotion-2026q3	reviewer:alice	blinded packet	expected
hidden-set integrity: INTACT			

29.6 Deployment ancestry

A deployment view should answer “why is this setting in production?”

production variable: coinvault.answer.routing_prompt

current value digest: sha256:9b...

introduced by: candidate abstention-routing-v3

promotion: 2026-08-19 / reviewer approval 2-of-2

supporting campaigns:

development campaign C-184

fresh reproduction C-185

promotion evaluation C-186

measurement epochs:

route contract det:v4

faithfulness judge epoch 9f...

parent value: sha256:5c...

30. Time travel and replay UI

Because projections are event-derived, the UI can render campaign state at any sequence.

Campaign replay	
cursor: seq 142 / 311	[◀] [▶] [play]

```
14:12:08 candidate.materialized gec-routing-v3
at this point:
  1 candidate active, 0 episodes scheduled, resource usage 31%
next event: trial.planned
```

This is useful for debugging reducers, explaining decisions, and demonstrating that later knowledge did not affect earlier proposals.

31. HTTP, SSE, and WebSocket interfaces

31.1 Read API

A minimal read-only server:

```
GET /api/optkit/campaigns
GET /api/optkit/campaigns/{id}
GET /api/optkit/campaigns/{id}/plan
GET /api/optkit/campaigns/{id}/events?after=184
GET /api/optkit/campaigns/{id}/projections/overview
GET /api/optkit/campaigns/{id}/projections/live-matrix
GET /api/optkit/campaigns/{id}/projections/lineage
GET /api/optkit/candidates/{id}
GET /api/optkit/episodes/{id}
GET /api/optkit/artifacts/{digest}/metadata
```

Use content negotiation or explicit artifact endpoints for bytes. Do not expose restricted native artifacts solely because a caller can guess a digest.

31.2 Live updates

SSE is sufficient for durable control events and projection invalidations:

```
GET /api/optkit/campaigns/{id}/stream?after=184
```

WebSocket is appropriate when the host product already has a bidirectional session transport or when live observation streams require richer subscriptions.

31.3 Coinvault integration

Coinvault already has canonical sessionstream transport and web projection logic. The recommended integration is:

OptKit control API/SSE	→ campaign/status/lineage panels
Coinvault sessionstream	→ selected live episode details/widgets
artifact inspector adapter	→ completed timeline/turn/evidence panels

Do not force OptKit events into Coinvault's user-chat event schema. Either add a separate OptKit protobuf namespace on the same transport or expose a dedicated endpoint and merge client-side.

31.4 RAG-TTC integration

The existing Bubble Tea experiment browser should stop parsing arbitrary results.json and instead use projection.Reader. Product inspector tabs can continue reading native RAG-TTC outcomes and sessions.

A useful improvement follows from typed constructs: the current browser marks the numerically largest metric as best. The new UI should use each construct's direction, so lower latency/error/cost values are correctly preferred.

31.5 Command API versus UI mutations

The UI may expose pause, resume, stop, review, or proposal submission, but these actions should send commands to the controller. The UI never edits files or projections directly.

```
UI command
  ↓ authenticated/authorized
controller validates against current state
  ↓
control event appended
  ↓
projection updates
```

32. Accessibility, security, and failure rendering

32.1 Accessibility

- Do not encode pass/fail only through color.
- Every graph must have a table representation.
- Keyboard navigation is required for TUI and web.
- Tooltips cannot be the only source of protocol/epoch identity.
- Time series and Pareto views need textual summaries.

32.2 Security

- Artifact reads require authorization by sensitivity and campaign role.
- Promotion-set case payloads must not be embedded in generic projections.
- Raw reasoning and tool payloads may need stricter permissions than aggregate measurements.
- UI links use opaque authorized IDs, not unchecked filesystem paths.
- Every state-changing command records actor identity.

32.3 Failure presentation

The UI must distinguish:

```
product failed
intervention not exercised
hard constraint failed
judge failed
measurement not applicable
measurement epoch mismatch
budget exhausted
campaign cancelled
```

Displaying all of these as a red "0" destroys the semantics OptKit is designed to preserve.

Part V — Porting the existing systems

33. Migration strategy: preserve evidence before improving abstractions

The migration should not begin by rewriting Coinvault’s evaluator or deleting RagOpt. The safe sequence is:

1. import and reproduce current identities/artifacts
2. run the same experiment through OptKit compatibility adapters
3. prove result parity
4. split product responsibilities into the new interfaces
5. introduce typed measurements and campaign stages
6. rewire UIs to projections
7. only then add automated search

Each product must have a golden compatibility campaign before legacy code is retired.

33.1 Recommended implementation order

1. Implement the OptKit storage/model kernel.
2. Build the RagOpt bridge and import fixtures.
3. Port RAG-TTC first because its adapter is smaller.
4. Port Coinvault using the proven interfaces.
5. Add generic/UI projections.
6. Add manual and coordinate strategies.
7. Add a GEPA-like reflective strategy only after exposure and measurement-epoch rules are enforced.

Coinvault should still contribute fixtures and interface requirements from the start.

34. Evolving RagOpt into an OptKit-based RAG layer

34.1 Current RagOpt responsibility map

Current package	Current responsibility	OptKit destination
pkg/candidate	immutable snapshots, exactly-one mutation	space, candidate; one-mutation becomes plan policy
pkg/eval	suites, two arms, cells, resume	dataset, trial/paired, campaign/controller
pkg/compare pkg/policy	strict pairs and means gate schema	estimand, stat/paired select/lexicographic policy schema
pkg/gate pkg/runstore	ordered gate evaluation durable local run	select store/fs, generalized campaign journal/artifacts
pkg/report pkg/review	Markdown/JSON reports blind review	projection, report renderers review

34.2 Do not create a dependency cycle

OptKit must not import RagOpt. The compatibility bridge should live in RagOpt:

```
ragopt/pkg/optkitbridge
  imports ragopt legacy types
```

```
imports optkit types
converts legacy candidates/runs/policies
```

RagOpt can then depend on OptKit. OptKit remains product- and RAG-independent.

34.3 Legacy candidate mapping

RagOpt v1	OptKit
Snapshot.LockedAssets	snapshot values + campaign-locked variables/context
Snapshot.MutableAssets	snapshot values + search-space mutable variables
Mutation	one-assignment Patch
MutationDeclaration	candidate hypothesis/targets/risks
Candidate.Parent/Child	snapshot refs
candidate digest	imported legacy identity and new bridge digest

Preserve the original candidate bytes and digest as an artifact. Do not pretend the new canonical digest is historically identical.

```
type ImportedIdentity struct {
    SourceSchema string
    SourceDigest Digest
    SourceBytes artifact.Ref
}
```

34.4 Legacy run mapping

One RagOpt run becomes one OptKit campaign with:

```
one plan
one stage
one paired trial
one baseline arm
one candidate arm
N × repeats × 2 episodes
legacy Outcome maps to measurements/constraints/usage
legacy comparison maps to imported estimate artifacts
legacy gate maps to selection decision
native artifacts remain linked
```

34.5 Legacy metric adapter

During compatibility:

```
type LegacyMetricAdapter struct {
    ConstructRegistry map[string]measure.ConstructRef
    Epoch             measure.EpochID
    Instrument         measure.InstrumentRef
}
```

Every legacy metric name must be explicitly registered with unit and direction. Unknown names remain descriptive or cause import failure; do not guess.

34.6 Legacy gate translation

The bridge maps current phases directly:

identity checks
hard gates
one target estimand
per-case regressions
group-mean regressions
cost tie-breakers

The translated selection decision should be compared to the legacy gate . Evaluate output in golden tests.

34.7 Future RagOpt role

After product ports, RagOpt should contain:

```
ragopt/  
├─ preset/  
│   ├── retrieval/      common RAG constructs and estimands  
│   ├── evidence/      evidence/citation constraints  
│   ├── paired/        common paired campaign builders  
│   └─ promotion/      conservative RAG selection presets  
├─ adapter/  
│   └─ ragkit/          trajectory/evidence mappings where genuinely generic  
├─ optkitbridge/       legacy import/export during migration  
└─ legacy/             frozen compatibility APIs, deprecated
```

RagOpt should not own product prompts, product runtime construction, data, or deployment.

34.8 Compatibility milestones

- Import every checked-in Coinvault and RAG-TTC candidate bundle.
- Import a deterministic RagOpt fixture run.
- Reproduce pair keys, complete/missing-pair handling, mean deltas, wins/ties/losses, failure rates, and gate result.
- Verify old native artifact digests.
- Render an OptKit projection from the imported run.
- Keep old CLI functional until both products have parity campaigns.

35. Porting RAG-TTC

RAG-TTC is the recommended first live port because its current adapter is compact and already separates most product behavior from RagOpt.

35.1 Current flow

cmd/rag-ttc/cmds/tooleval/ragopt.go currently:

```
load frozen candidate and suite  
↓  
validate locked budgets/model/profile/source/index/corpus identities  
↓  
resolve answer and two judge profiles  
↓  
create RagOpt incumbent/challenger arms  
↓  
for each cell:  
    materialize tool config
```

```

create chat runtime
submit question
project native TurnRecord
judge answer
write native outcome.json
return float metrics + cost projection

```

35.2 Target package layout in RAG-TTC

```

rag-ttc/internal/optimization/
├─ adapter.go           implements optkit runtime/evaluation interfaces
├─ schema.go           product system schema and variable registry
├─ materializer.go     snapshot → prepared `tool-qa.yaml` tree
├─ runner.go           canonical chat episode execution
├─ trajectory.go       native session manifest and generic facts
├─ constraints.go      runtime/answer/tool contracts
├─ measurements.go     answerquality/JudgeKit adapter
├─ environment.go      source/profile/index/corpus validation
├─ campaigns.go        I5 feedback/validation plan builders
├─ projections.go      TUI product-inspector projections
└─ fixtures_test.go    parity and golden tests

```

Keep files reasonably sized; the names are responsibility boundaries, not a requirement for one type per file.

35.3 Product system schema

The initial search space can be intentionally small:

```

func Schema() space.Schema {
    return space.MustSchema(
        model.SystemRef{
            Name:           "rag-ttc.admin-tool-loop",
            AdapterSchema: "rag-ttc.opt-adapter/v1",
        },
        space.TextArtifactVariable(
            "rag_ttc.tool.search.description",
            "Instructions that guide the search tool's first-query behavior.",
            mediaTypeYAML,
        ),
        // Declared but locked in the I5 campaign:
        space.TextArtifactVariable("rag_ttc.agent.orchestration_prompt", ...),
        space.JSONArtifactVariable("rag_ttc.answer.schema", ...),
        space.StructuredVariable("rag_ttc.runtime.contract", ...),
    )
}

```

The I5 campaign marks only `rag_ttc.tool.search.description` mutable.

35.4 Snapshot conversion

The existing parent/candidate assets map naturally:

```

orchestration_prompt → rag_ttc.agent.orchestration_prompt
answer_schema       → rag_ttc.answer.schema
search_description   → rag_ttc.tool.search.description

```

runtime_contract → rag_ttc.runtime.contract
judge_contract → measurement-plan artifact, not a system variable

The judge contract does not affect product execution and belongs in the measurement plan. Preserve it in imported snapshots during compatibility, then move it at the new campaign boundary.

35.5 Environment validator

Move validateI5Environment into an implementation of:

```
type EnvironmentValidator struct {  
    RepositoryRoot string  
    IndexBundle    string  
    Providers      *raggepetto.Bundle  
}  
  
func (v *EnvironmentValidator) Validate(  
    ctx context.Context,  
    expected model.EnvironmentFingerprint,  
) error
```

The fingerprint includes:

answer model/profile and selected profile digest
embedding provider/model/dimensions
evaluator adapter identity
suite partition identity
tool safety identity
source digests for evaluation, judge, native adapter, OptKit adapter, runner types
index manifest digest
corpus digest

Keep the fail-closed behavior.

35.6 Materializer

Extract materializeToolConfig into deterministic materialization:

```
type Materializer struct{}  
  
func (m Materializer) Materialize(  
    ctx context.Context,  
    req runtime.MaterializeRequest,  
) (runtime.PreparedSystem, error) {  
    cfg := BuildModelFromSnapshot(req.Snapshot)  
  
    root := req.OutputDirectory  
    writeVerified(root+"/orchestration.txt", cfg.OrchestrationPrompt)  
    writeVerified(root+"/answer-schema.json", cfg.AnswerSchema)  
    writeVerified(root+"/search-description.yaml", cfg.SearchDescription)  
    writeVerified(root+"/tool-qa.yaml", RenderToolQA(cfg))  
  
    manifest := PreparedManifest{  
        ToolQAConfigDigest: digest.File(root+"/tool-qa.yaml"),  
        SnapshotDigest:     req.Snapshot.Digest,  
        EnvironmentDigest:  req.Environment.Digest,  
    }  
    return runtime.PreparedSystem{Root: root, Manifest: Put(manifest)}, nil  
}
```

No provider or database call occurs here.

35.7 Episode runner

Extract the runtime portion of `ragoptCellExecutor.run`:

```
type Runner struct {
    IndexBundle      string
    CacheDirectory   string
    Providers         *raggepetto.Bundle
    EmbeddingBudget  int
    GenerationBudget int
}

func (r *Runner) RunEpisode(
    ctx context.Context,
    req runtime.EpisodeRequest,
    sink trajectory.ObservationSink,
) (trajectory.Manifest, error) {
    input := DecodeCase(req.Case)
    rt := chatpkg.NewRuntime(... prepared tool-qa.yaml ...)
    record, submitErr := rt.Submit(ctx, input.Question, "", answering.RetrievalConfig{})
    status := rt.Status()
    closeErr := rt.Close(context.WithoutCancel(ctx))

    projected := ProjectChatRecord(req.Arm.Role, input, status.SessionDir, record)
    EmitGenericObservations(sink, record, projected)
    return SealNativeTrajectory(req, status.SessionDir, projected, submitErr, closeErr)
}
```

The runner does not call the judge. This permits remeasurement and keeps system/evaluation costs separate.

35.8 Trajectory projector

Move `projectChatRecord` into a product projector. Preserve all existing facts:

answer text and abstention
contract validity
citation IDs
evidence IDs
tool call counts/errors
provider calls
token usage
reasoning-token usage
failure and duration
native session path

Expose common facts through `TrajectoryManifest.SummaryFacts`, but retain the full `session.TurnRecord` artifacts.

35.9 Constraints and intervention probe

The initial patch changes a tool description. The intervention probe should verify the prepared runtime consumed the expected description digest/identity. If the runtime does not currently expose that identity in its session record, add a bounded runtime fact to the native outcome.

Deterministic constraints retain current `projected.ContractValid` semantics and can later split into named checks rather than one Boolean.

35.10 Measurement adapter

Phase 1 may wrap the existing `answerquality.JudgeToolLoop` as an `OptKit` instrument while retaining its native result. Phase 2 should express its construct/protocol through `JudgeKit`.

```
type AnswerQualityInstrument struct {  
    Statements *raggepetto.Bundle  
    Verdicts   *raggepetto.Bundle  
    Budget     int  
    Epoch      measure.EpochID  
}
```

Produce typed faithfulness and `answer_relevance` measurements. Record statement/verdict provider calls and tokens under `UsageEvaluation`.

35.11 Campaign plan

The checked-in I5 configuration becomes a plan preset:

```
func I5Plan(inputs I5Inputs) (campaign.Plan, error) {  
    return campaign.NewPlan(  
        campaign.WithBaseline(inputs.ParentSnapshot),  
        campaign.WithSearchSpace(OneVariable("rag_ttc.tool.search.description")),  
        campaign.WithStage(FeedbackStage(  
            inputs.FeedbackSuite, 1,  
            PairedFixedOrder(),  
            inputs.I5SelectionPolicy,  
        )),  
        campaign.WithStage(ValidationStage(  
            inputs.ValidationSuite, 2,  
            PairedCounterbalanced(),  
            inputs.I5SelectionPolicy,  
        )),  
        campaign.WithStrategy(search.Manual()),  
        campaign.WithHumanPromotion(),  
    )  
}
```

For exact compatibility, use the current fixed order first. Counterbalancing can be a later campaign-version change.

35.12 Existing feedback and validation sizes

The current configuration declares three feedback cases and seven disjoint validation cases. With one feedback repeat:

$$3 \times 2 \times 1 = 6$$

episodes. With two validation repeats:

$$7 \times 2 \times 2 = 28$$

episodes.

The plan UI should show these exact counts before execution.

35.13 TUI migration

Current files:

```
internal/admin/tui/runstorebrowser.go
internal/admin/tui/expbrowser.go
```

Recommended changes:

1. Replace direct directory scanning and `results.json` flattening with `projection.Reader`.
2. Keep current Bubble Tea navigation and styling.
3. Add campaign/stage/status columns instead of only run name/status.
4. Add plan, live matrix, lineage, selection, and measurement-epoch views.
5. Preserve native inspector tabs by registering a RAG-TTC InspectorProvider.
6. Use construct direction to mark best values.
7. Continue tolerating unreadable/corrupt campaigns, but expose verification errors explicitly.

35.14 Transcript warehouse integration

The earlier GEPA-inspired JSONL-to-SQLite design becomes a product indexer:

optkit control index	generic campaign tables
RAG-TTC trajectory indexer	sessions/turns/iterations/tool_calls/evidence
SQL recipes	diagnostic views
reflection packet builder	authorized queries over those views

Source JSONL/native sessions remain authoritative; SQLite remains rebuildable.

35.15 RAG-TTC parity test

Create a deterministic/fake-provider fixture and one controlled live fixture. Verify:

```
same candidate parent/child bytes
same materialized tool-qa.yaml
same episode count and order
same source/profile/index/corpus validation
same native session artifacts
same contract-valid/abstention values
same faithfulness/relevance projections
same system cost accounting
same paired deltas and gate decision
resume does not rerun completed episodes
```

Only after this test passes should the legacy `tooleval/ragopt.go` path be deprecated.

36. Porting Coinvault

Coinvault is the requirements-driving port. It exercises multi-turn agent behavior, multiple variable kinds, treatment applicability, rich trajectories, canonical WebSocket observation, source locks, shared budgets, deterministic contracts, protected abstention, and UI/widget infrastructure.

36.1 Current responsibility map

Current file	Current responsibility	Target responsibility
knowledge_ragopt.go	CLI, preflight, budgets, runner construction, cell execution, judge, outcome projection	product command + OptKit adapter/controller wiring
knowledge_ragopt_case.go	case payload decoding/validation	dataset codec and product case projector
knowledge_ragopt_contract.go	answer/route/evidence/citation checks	named deterministic constraints
knowledge_ragopt_trace.go	canonical event collection and runtime facts	native trajectory collector and generic fact projector
knowledge_ragopt_treatment.go	mechanism schema, materialization expectations, exercise checks	variable registry, bindings, expected effects, probes
knowledge_ragopt_reranker.go	reranker asset/runtime verification	reranker variable binding
knowledge_ragopt_suite_lock.go	suite identity/source constraints	dataset/environment validation
knowledge_ragopt_gate.go	terminal comparison/gate/review/plan artifacts	selection preset + review/promotion projections

36.2 Target package layout in Coinvault

```

coinvault/internal/optimization/
├─ adapter.go
├─ schema.go
├─ snapshot.go
├─ variables/
│   └─ result_depth.go
│   └─ comparison.go
│   └─ prompts.go
│   └─ reranker.go
│   └─ tool_description.go
├─ materializer.go
├─ environment.go
├─ runner.go
├─ trajectory.go
├─ intervention.go
├─ constraints.go
├─ measurements.go
├─ campaigns.go
├─ legacy_import.go
├─ projections.go
└─ parity_test.go

```

A flat package with these files is also acceptable initially. Avoid separate Go packages for every variable unless dependency boundaries justify them.

36.3 Coinvault system schema

Initial variable registry:

```

func CoinvaultSchema() space.Schema {
    return space.MustSchema(
        model.SystemRef{

```

```

        Name: "coinvault.admin-chat",
        AdapterSchema: "coinvault.opt-adapter/v1",
    },

    IntVariable(
        "coinvault.knowledge.default_results",
        1, knowledge.DefaultToolConfig().MaxResults,
    ),
    OptionalIntVariable(
        "coinvault.knowledge.forced_results",
        1, knowledge.DefaultToolConfig().MaxResults,
    ),
    StructuredArtifactVariable(
        "coinvault.knowledge.comparison_plans",
        "coinvault.comparison-plans/v1",
    ),
    BoolVariable("coinvault.knowledge.comparison_queries_enabled"),
    BoolVariable("coinvault.knowledge.comparison_intents_enabled"),

    TextArtifactVariable("coinvault.answer.grounding_prompt"),
    TextArtifactVariable("coinvault.answer.routing_prompt"),
    TextArtifactVariable("coinvault.answer.policy_prompt"),

    StructuredArtifactVariable(
        "coinvault.knowledge.reranker",
        "coinvault.reranker-config/v1",
    ),
    StructuredArtifactVariable(
        "coinvault.knowledge.tool_description",
        "coinvault.knowledge-tool-description/v1",
    ),
)
}

```

Some current candidate assets combine multiple booleans or plans. Preserve exact imported values, then normalize only in a new campaign schema version. Migration must not reinterpret historical bytes.

36.4 Locked system variables and environment

Variables that affect product behavior but are normally locked should still be representable in snapshots:

```

answer model/profile
embedding model/profile
SQL and knowledge tool policy
maximum provider/tool iterations
answer schema
query-transform identity
evidence-ledger contract
retrieval-policy identity
authorization policy
widget projection policy

```

Code/source revisions, index/bundle digests, provider serving revisions, and adapter versions belong in the environment fingerprint unless the campaign deliberately makes them search variables.

36.5 Replacing the mechanism switch

Current logic branches on strings such as:

```

knowledge_comparison_decomposition
knowledge_comparison_intent
answer_grounding_prompt
answer_routing_prompt
answer_policy_prompt
knowledge_tool_description
knowledge_reranker
knowledge_tool_default_results
knowledge_tool_forced_results

```

The target materializer starts from a typed build model and applies all snapshot values through registered bindings:

```

type CoinvaultBuild struct {
    RunnerOptions evalchat.ConfiguredRunnerOptions

    ExpectedRuntime struct {
        PromptSuffixDigests map[space.VariableID]model.Digest
        RerankerIdentity      string
        ToolDescriptionID     string
        DefaultResults        int
        ForcedResults         int
        ComparisonQueries     bool
        ComparisonIntents     bool
        QueryTransformID      string
        RetrievalPolicyID     string
        EvidenceLedgerID      string
    }
}

func (m *Materializer) Materialize(
    ctx context.Context,
    req runtime.MaterializeRequest,
) (runtime.PreparedSystem, error) {
    build := NewCoinvaultBuild(m.BaseSettings)

    for _, variable := range req.Schema.CanonicalVariables() {
        value := req.Snapshot.Values[variable.ID]
        binding := m.Bindings.MustGet(variable.ID)
        if err := binding.Apply(ctx, value, &build); err != nil {
            return runtime.PreparedSystem{}, err
        }
    }

    expected := m.Bindings.ExpectedEffects(build, req.Snapshot)
    manifest := SealPreparedCoinvault(build, expected, req)
    return runtime.PreparedSystem{Manifest: manifest.Ref, Opaque: build}, nil
}

```

There is no “which treatment mechanism is this?” branch. The patch records which variables changed; the materializer always builds the complete system.

36.6 Variable binding examples

Default result depth

```

bindings.Register(IntBinding{
    ID: "coinvault.knowledge.default_results",

```

```

ApplyInt: func(build *CoinvaultBuild, value int) error {
    build.RunnerOptions.KnowledgeDefaultResults = value
    build.ExpectedRuntime.DefaultResults = value
    return nil
},
Probe: DefaultResultsProbe{},
})

```

Grounding prompt

```

bindings.Register(TextArtifactBinding{
    ID: "coinvault.answer.grounding_prompt",
    ApplyText: func(build *CoinvaultBuild, text string, ref artifact.Ref) error {
        build.RunnerOptions.SystemPromptSuffix = JoinPromptSuffix(
            build.RunnerOptions.SystemPromptSuffix,
            text,
        )
        build.ExpectedRuntime.PromptSuffixDigests[
            "coinvault.answer.grounding_prompt"
        ] = ref.Digest
        return nil
    },
    Probe: RuntimePromptDigestProbe{},
})

```

Reranker

```

bindings.Register(StructuredBinding[RerankerValue]{
    ID: "coinvault.knowledge.reranker",
    ApplyValue: func(build *CoinvaultBuild, value RerankerValue) error {
        reranker, runtimeConfig, err := ResolveAndVerifyReranker(value)
        if err != nil { return err }
        build.RunnerOptions.KnowledgeReranker = reranker
        build.RunnerOptions.KnowledgeRerankerConfig = runtimeConfig
        build.ExpectedRuntime.RerankerIdentity = value.RuntimeIdentity
        return nil
    },
    Probe: RerankerConfiguredAndAppliedProbe{},
})

```

Tool description

```

bindings.Register(ArtifactBinding{
    ID: "coinvault.knowledge.tool_description",
    ApplyRef: func(build *CoinvaultBuild, ref artifact.Ref) error {
        description := knowledge.LoadToolDescription(ref.LocalPath())
        identity := description.RuntimeIdentity()
        build.RunnerOptions.KnowledgeDescription = ref.LocalPath()
        build.ExpectedRuntime.ToolDescriptionID = identity
        return nil
    },
    Probe: ToolDescriptionIdentityProbe{},
})

```

36.7 Applicability

Current treatment contracts decide whether a mechanism applies to a case. Preserve this concept at the variable/probe level:

```
type ApplicabilityPolicy interface {
    Applies(
        variable space.VariableID,
        c dataset.CaseView,
        trajectory *trajectory.View,
    ) (measure.Applicability, string)
}
```

Examples:

- prompt variables apply to all answer episodes;
- comparison intent applies to declared comparison cases;
- result-depth variables apply when a knowledge call uses the relevant fallback/forced path;
- reranker application requires a retrieval call with enough candidates;
- tool-description effects may be applicable to cases where the model must decide whether to invoke knowledge search.

Applicability may use case metadata before execution and trajectory facts after execution.

36.8 Prepared system manifest

The materializer should seal a product-specific manifest:

```
type PreparedManifest struct {
    APIVersion      string
    SnapshotDigest  model.Digest
    EnvironmentDigest model.Digest
    RunnerOptionsDigest model.Digest
    ExpectedEffects []runtime.ExpectedEffect
    AssetRefs       []artifact.Ref
    ProductFacts    map[string]string
    Digest          model.Digest
}
```

This replaces the need for a separate locked per-candidate treatment contract in new campaigns. The expected effects derive from the exact snapshot and registered bindings. Legacy imports retain their treatment contract as evidence.

36.9 Environment validator

Move validateGECRagoptEnvironment, source locks, model/profile checks, bundle checks, and budget identity checks into product preflight:

```
type CoinvaultEnvironmentValidator struct {
    RepositoryRoot string
    IndexBundle    string
    Profiles       ProfileSettings
    AnswerIdentity resolvedInferenceIdentity
    JudgeIdentity  resolvedInferenceIdentity
}
```

Validation should return structured checks suitable for the plan UI:

```
type EnvironmentCheck struct {
    Name      string
    Expected string
}
```

```

    Observed string
    Passed bool
    Artifact *artifact.Ref
}

```

A failed preflight creates no provider calls and no episodes.

36.10 Episode runner

Split current gecRagoptCellExecutor.Run after materialization:

```

type CoinvaultRunner struct {
    Parsed *values.Values
    Settings KnowledgeOptimizationSettings
    Profiles ProfileSettings
    Timeout time.Duration
    Budget *ExecutionBudgetAdapter
}

func (r *CoinvaultRunner) RunEpisode(
    ctx context.Context,
    req runtime.EpisodeRequest,
    sink trajectory.ObservationSink,
) (trajectory.Manifest, error) {
    input := DecodeCoinvaultCase(req.Case)
    prepared := req.Prepared.Opaque.(CoinvaultBuild)

    runner, err := evalchat.NewConfiguredRunner(ctx, r.Parsed, prepared.RunnerOptions)
    if err != nil { return trajectory.Manifest{}, err }

    collector := NewCoinvaultTraceCollector(prepared.ExpectedRuntime)
    observed, runErr := runner.RunPrompt(
        episodeTimeout(ctx, r.Timeout),
        input.Question,
        CorrelationKey(req),
        func(ev evalchat.Event) error {
            collector.Observe(ev)
            EmitCoinvaultObservation(sink, ev)
            return r.Budget.Observe(ev)
        },
    )

    closeErr := runner.Close()
    return SealCoinvaultTrajectory(req, collector, observed, runErr, closeErr)
}

```

This runner returns after the product trajectory is sealed. It does not run the judge.

36.11 Native artifacts

Preserve the current native artifact's valuable fields:

```

arm/case/repeat/candidate/snapshot/bundle
answer and judge runtime identities
session ID
trace
embedding/provider usage
treatment/intervention report

```

answer contract
contract issues
termination ownership/accounting

In the target architecture, judge results, intervention reports, and constraints may be separate content-addressed artifacts linked from the episode. A compatibility outcome.json can continue to aggregate them for operators while migration is in progress.

36.12 Canonical observation and UI integration

Coinvault already observes the canonical HTTP/WebSocket path through evalchat.ConfiguredRunner. Continue doing so. Do not create a special local-only optimization runtime that bypasses production composition.

Emit normalized observation summaries from the same callback used by the existing trace collector. The Coinvault web UI can subscribe to:

OptKit campaign events	candidate/trial/measurement/decision lifecycle
Coinvault sessionstream	detailed selected episode timeline and widgets

36.13 Trajectory facts

The product projector should expose bounded generic facts such as:

terminal event/status/error
message ID and session ID
answer present / abstained
knowledge call count
SQL call count
requested/effective result limits and source
query-transform/retrieval-policy/evidence-ledger identities
comparison intent/decomposition applied
reranker configured/applied/identity
evidence admitted/omitted
citation IDs and resolution counts
provider calls/tokens/duration
widget intent/render counts and errors

Raw queries, answers, evidence, SQL, and tool payloads remain in restricted native artifacts.

36.14 Intervention probes

Replace evaluateGECRagoptDefaultResultsTreatment and mechanism-specific branches with independent probes. The patch determines which probes must pass.

```
func CheckPatchExercise(
    ctx context.Context,
    patch space.Patch,
    prepared PreparedManifest,
    tr trajectory.View,
    registry ProbeRegistry,
) ([]measure.ConstraintResult, error) {
    results := []measure.ConstraintResult{}
    for _, assignment := range patch.Assignments {
        probe := registry.MustGet(assignment.Variable)
        expected := prepared.ExpectedEffect(assignment.Variable)
        report, err := probe.Check(ctx, expected, tr)
        if err != nil { return nil, err }
        results = append(results, ReportAsConstraints(report)...)
    }
}
```

```

    }
    return results, nil
}

```

36.15 Deterministic constraints

Refactor the answer contract into individually named constraints:

```

runtime.terminal_success
answer.present_or_valid_abstention
route.required_tool_used
route.forbidden_tool_not_used
evidence.required_groups_present
evidence.admitted_identity_valid
citation.ids_resolve
citation.knowledge_evidence_cited
projection.message_completed
protected.correct_abstention

```

The existing aggregate answer-contract report can be rendered from these constraint results during migration.

36.16 Measurement pipeline

The target pipeline:

```

sealed trajectory
  ↓
intervention probes
  ↓
deterministic constraints
  ↓ blocking pass?
  ├── no → record missing/skipped judge measurements with reason
  └── yes
      ↓
      build JudgeKit instance from answer + admitted evidence + case
      ↓
      run JudgeKit suite
      ↓
      record typed faithfulness/relevance/unsupported-claim measurements

```

The current `knowledge.JudgeAnswer` implementation can initially be wrapped as an instrument. A later migration can move its contracts/protocols fully into `JudgeKit` without changing `OptKit` campaign semantics.

36.17 Budget mapping

Current fixed ceilings:

maximum answer calls	216
maximum embeddings	192
maximum judge calls	72
maximum answer tokens	1,000,000

Represent them as named campaign resources with explicit scopes. `FlowKit` handles attempt admission where possible; the existing event observer continues to reconcile actual provider calls/tokens from canonical events.

On resume:

1. replay resource-consumption events;

2. verify them against native artifacts;
3. reconcile discrepancies fail-closed;
4. initialize shared FlowKit budgets with remaining capacity;
5. schedule only missing episode keys.

36.18 Stage plan

Current Coinvault behavior becomes explicit:

```
func KnowledgeOptimizationPlan(inputs Inputs) campaign.Plan {
    return campaign.Plan{
        Baseline:    inputs.Baseline,
        SearchSpace: inputs.SearchSpace,
        Strategy:    search.Manual(),
        Stages: []campaign.StagePlan{
            FeedbackStage(
                inputs.Feedback12,
                repeats(1),
                strategyVisibility(FullDiagnostics),
                onPass("fresh-reproduction"),
            ),
            FreshReproductionStage(
                inputs.Feedback12,
                noSystemEpisodeReuse(),
                strategyVisibility(AggregateOnly),
                onPass("promotion"),
            ),
            ClosedPromotionStage(
                inputs.PromotionDataset,
                repeats(2),
                strategyVisibility(None),
                selectorVisibility(AggregateOnly),
                onPass(HumanReview),
            ),
        },
    }
}
```

If the promotion corpus is not yet represented in repository artifacts, the stage remains declared but unavailable, with a controlled external data provider. The UI should display CLOSED, not silently omit the stage.

36.19 Existing candidate import

Import all current directories, including:

```
default-results-8-v1...v7
forced-results-8-v7
comparison-decomposition-v1
comparison-intent-v1...v3
grounded-answer-v1/v2
abstention-routing-v1...v3
schema-routing-prompt-v1
answer-policy-routing-grounding-v1
source-role-routing-v1/v2
qwen3-reranker-pool12-v1
canonical-seed-stack-v1
```

For each:

1. preserve the entire candidate directory as an artifact;
2. load through current RagOpt validation;
3. map parent/child assets to schema variables;
4. construct one-assignment imported patches where possible;
5. retain treatment-contract bytes as a legacy intervention instrument;
6. preserve proposer identity and hypotheses;
7. attach evaluator/protocol revisions as historical metadata;
8. do not aggregate incompatible historical judge epochs.

36.20 Separating evaluator revisions from candidate lineage

Names such as ...evaluator-v8 in proposer identities reveal that measurement changes were interleaved with candidate iterations. In OptKit:

candidate identity	changes only when system configuration changes
measurement epoch	changes when evaluator contract/protocol changes
trial/campaign identity	changes when the candidate is remeasured

A candidate can therefore have results:

```

candidate grounded-answer-v2
├─ measured under epoch judge-v8
└─ remeasured under epoch judge-v10

```

without inventing a new system candidate.

36.21 Coinvault web workspace

Add an Optimization route/workspace. Recommended navigation:

```

Optimization
├─ Campaigns
├─ Plan
├─ Live
├─ Candidates
├─ Episodes
├─ Measurements
├─ Selection
├─ Data exposure
└─ Deployment history

```

When an episode is selected, reuse existing sessionstream timeline and widget components through an inspector adapter. The optimization UI should not duplicate message/widget rendering.

36.22 Coinvault parity campaign

Use one checked-in candidate such as grounded-answer-v2 and a fake/deterministic runtime fixture before a controlled live proof. Verify:

```

same 12-case suite and 24 episode plan
same incumbent/challenger snapshot asset bytes
same canonical runtime path
same requested/effective result-limit facts
same prompt/reranker/tool-description identities
same treatment-exercise result
same answer-contract issues and precedence
same judge invocation/skipping behavior
same numeric metric projection

```

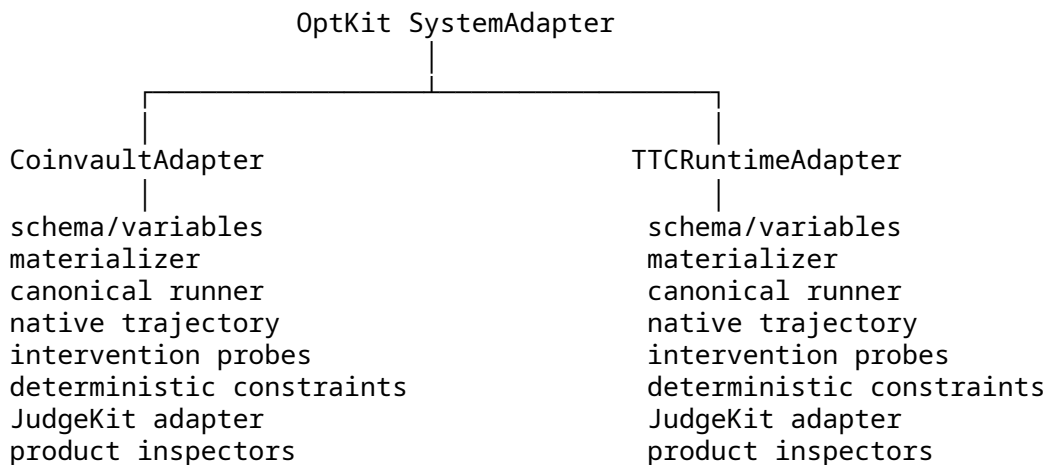
same provider/embedding/judge accounting
same paired comparison and gate decision
same native outcome bytes where feasible, otherwise field parity
same resume behavior and no duplicate completed episodes

36.23 Deprecation sequence

1. Keep coinvault knowledge ragopt unchanged.
 2. Add coinvault optimize import-ragopt and a compatibility campaign runner.
 3. Run parity fixtures in CI.
 4. Add coinvault optimize plan/run/resume/status backed by OptKit.
 5. Move one candidate family—preferably prompt or search-description style—onto variable bindings.
 6. Move all mechanisms.
 7. Rewire gate/review artifacts.
 8. Add web projections.
 9. Freeze old command as legacy read/import path.
 10. Remove only after historical runs and candidate bundles remain inspectable.
-

37. Shared product adapter pattern

After both ports, the product adapters should look structurally similar without sharing product semantics.



Only generic interfaces and small helper libraries should be shared. Do not create a “universal chat adapter” by intersecting Coinvault and RAG-TTC until a third real product demonstrates the same stable need.

Part VI — Implementation tools and work plan

38. Command-line tools

OptKit should be library-first. Generic commands can inspect and validate artifacts, but product execution commands remain statically linked in product binaries because they require product adapters and credentials.

38.1 Generic optkit command

```
optkit plan validate <plan.yaml>
optkit plan explain <plan.yaml>
optkit plan graph <plan.yaml>
```

```

optkit campaign list --root <root>
optkit campaign inspect <campaign>
optkit campaign verify <campaign>
optkit campaign replay <campaign> [--at-sequence N]
optkit campaign export <campaign> --format evidence-bundle

optkit candidate inspect <candidate>
optkit candidate diff <candidate>
optkit candidate verify <candidate>

optkit measurements list <campaign>
optkit measurements epochs <campaign>
optkit measurements compare <candidate-a> <candidate-b>

optkit selection explain <decision>
optkit exposure inspect <campaign>
optkit resources inspect <campaign>

optkit projections build <campaign>
optkit projections verify <campaign>
optkit index build <campaign>
optkit index query <campaign> --sql <file-or-query>

optkit artifacts verify <campaign>
optkit artifacts inspect <digest>

optkit tui --root <root>
optkit serve --root <root> --read-only

```

38.2 Product commands

Coinvault:

```

coinvault optimize plan --preset knowledge-feedback --candidate <bundle>
coinvault optimize preflight --plan <plan>
coinvault optimize run --plan <plan>
coinvault optimize resume --campaign <id>
coinvault optimize status --campaign <id>
coinvault optimize propose --campaign <id> --patch <patch.yaml>
coinvault optimize review --campaign <id> --candidate <id>
coinvault optimize import-ragopt --run <legacy-run>

```

RAG-TTC:

```

rag-ttc optimize plan --preset i5 --candidate <bundle>
rag-ttc optimize run --plan <plan>
rag-ttc optimize resume --campaign <id>
rag-ttc optimize status --campaign <id>
rag-ttc optimize import-ragopt --run <legacy-run>

```

38.3 Why generic execution is not the first goal

A command such as `optkit run --plugin coinvault.so` would introduce dynamic loading, version skew, credential ambiguity, and product-runtime ownership problems. Static product commands invoking the OptKit library are simpler and safer for v0.

38.4 Command output contract

Commands should support human rows and machine JSON. A successful run command reports durable identity and location, not an ephemeral in-memory object:

```
{
  "campaign_id": "cmp-20260820-...",
  "campaign_directory": "experiments/optkit/cmp-...",
  "plan_digest": "sha256:...",
  "state": "running",
  "journal_sequence": 12,
  "projection_root": ".../projections"
}
```

39. Libraries and tools to implement

This section is the intern's implementation inventory. Each item states purpose, minimum API, and acceptance criteria.

39.1 Canonical identity library

Purpose. Ensure every semantic record has stable canonical bytes and digest.

Implement.

- bounded identifiers;
- SHA-256 digest type with strict parsing;
- canonical JSON encoding;
- semantic-vs-byte digest helpers;
- golden fixtures;
- path-independent artifact identity.

Acceptance. Same semantic object produces identical bytes/digest across map insertion orders and repeated runs. Unknown schema versions fail closed.

39.2 Content-addressed artifact store

Purpose. Store prompts, plans, reports, and event payloads once and refer to exact bytes.

Implement.

- atomic put/open/verify;
- digest and size validation;
- media type/schema metadata;
- sensitivity labels;
- local URI and external-reference support;
- corruption tests.

Acceptance. Existing invalid blobs never silently recompute or overwrite. Concurrent identical puts converge safely.

39.3 Event journal

Purpose. Authoritative campaign control history.

Implement.

- append with expected head;
- canonical event envelope;

- sequence/hash-chain verification;
- fsync durability boundary;
- idempotent command IDs;
- read/tail;
- crash recovery and head repair;
- single-writer lock.

Acceptance. Property tests cover truncation, duplicate append, sequence gap, broken digest, crash between append/head update, and replay after repair.

39.4 Campaign reducer and checkpoints

Purpose. Reconstruct state and reject impossible transitions.

Implement.

- plan-based initial state;
- pure reducers per event family;
- state-machine validation;
- checkpoint schema and verification;
- replay --at-sequence support.

Acceptance. Full replay and checkpoint replay produce byte-identical state projections.

39.5 Space/snapshot/patch/candidate packages

Purpose. Represent heterogeneous system configurations and interventions.

Implement.

- domain types;
- variable schemas;
- inline/artifact values;
- total snapshots;
- stale-safe patches;
- candidate policy;
- candidate lineage;
- actual-diff validation independent of declarations.

Acceptance. One-coordinate policy reproduces RagOpt's exactly-one mutation behavior. Multi-coordinate fixtures work when policy permits them.

39.6 Dataset and exposure packages

Purpose. Preserve case identity, roles, slices, and information leakage.

Implement.

- opaque case payload refs;
- dataset manifests and split validation;
- group/slice selectors;
- actor/detail exposure policy;
- exposure events/ledger;
- authorized history view.

Acceptance. A strategy cannot retrieve promotion payloads in tests. Showing promotion diagnostics records exposure and invalidates hidden-set integrity as configured.

39.7 Trial and paired design

Purpose. Generalize RagOpt cells/pairs while preserving exact compatibility.

Implement.

- arms;
- episode keys/specs;
- paired planning;
- fixed/alternating/seeded order policies;
- repeats versus attempts;
- exact reuse policy;
- strict pair construction;
- missing-pair diagnostics.

Acceptance. Compatibility mode emits the same (case , repeat , arm) sequence and pair keys as current RagOpt.

39.8 Runtime controller and FlowKit integration

Purpose. Execute planned episodes under bounded resources and persist lifecycle facts.

Implement.

- product adapter registry supplied by host binary;
- materialization lifecycle;
- FlowKit step construction;
- resource mapping;
- event translation;
- cancellation/pause/stop;
- resume scheduling;
- attempt accounting.

Acceptance. Completed episodes are not rerun on resume; exact-reuse policy is honored; reproduction-stage no-reuse is honored; every retry consumes declared resources.

39.9 Trajectory and observation interfaces

Purpose. Preserve product-native artifacts while enabling generic live views.

Implement.

- trajectory manifest;
- stream references;
- optional observation sink;
- terminal sealing;
- bounded summary facts;
- native artifact verification;
- product inspector interface.

Acceptance. The framework can run a product with no normalized stream and still produce useful control projections; a streamed product yields live spans without changing native authority.

39.10 Measurement and constraint packages

Purpose. Replace float maps with instrumented observations.

Implement.

- construct references/directions/units;
- instrument and epoch identity;
- scalar/label/vector value kinds;

- applicability/status;
- uncertainty;
- evidence/assessment refs;
- typed constraint results;
- legacy metric adapter.

Acceptance. Zero, NA, missing, and failed remain distinct. Cross-epoch aggregation fails by default.

39.11 JudgeKit adapter

Purpose. Preserve JudgeKit semantics in optimization evidence.

Implement.

- instance builder boundary;
- suite execution adapter;
- assessment-to-measurement mapping;
- epoch calculation;
- calibration/audit refs;
- usage-scope records.

Acceptance. Measurement record can trace to exact contract, protocol, evidence instance, assessment report, and calibration artifact.

39.12 Estimand and statistics packages

Purpose. Make the scientific question explicit and compute transparent estimates.

Implement.

- paired contrast;
- group/slice population;
- exact epoch compatibility;
- missingness policies;
- mean/wins/ties/losses/worst delta;
- failure-rate and usage estimates;
- interval extension point;
- deterministic report digests.

Acceptance. Golden fixtures reproduce current RagOpt comparison reports.

39.13 Selection package

Purpose. Generalize lexicographic gates without scalarizing hard constraints.

Implement.

- ordered phases;
- check registry;
- stop-on-failure;
- eligibility states;
- target/regression/cost checks;
- exact decision input refs;
- legacy policy translator.

Acceptance. Grounded-answer and RAG-TTC I5 gate fixtures match legacy decisions and messages semantically.

39.14 Archive package

Purpose. Retain useful candidates for search.

Implement.

- best-by-estimand;
- objective Pareto;
- all-evidence;
- deterministic dominance/tie handling;
- policy identity;
- archive update artifacts.

Acceptance. Order-independent insertion yields the same archive. Ineligible candidates are retained or excluded according to explicit policy, never implicitly.

39.15 Search strategies

Purpose. Prove the abstraction before reflective automation.

Implement first.

- manual;
- finite coordinate enumerator;
- random seeded sampler;
- stop/budget strategy.

Implement later.

- GEPA-style reflector and case-elites archive;
- surrogate-assisted multi-component strategy;
- bandit/sequential allocator.

Acceptance. Strategy only sees authorized history; all decisions and strategy state are artifacts; invalid proposals are rejected by core validation.

39.16 Projection engine

Purpose. Power plan/live/history UIs without raw-file parsing in clients.

Implement.

- overview;
- plan/stage graph;
- live matrix;
- candidate lineage;
- episode summary;
- measurements/epochs;
- selection;
- exposure;
- resources;
- replay cursor;
- product panel hooks.

Acceptance. Rebuilding from scratch matches incrementally maintained projections.

39.17 Generic TUI and read-only server

Purpose. Make the system inspectable before product UI integration.

Implement.

- campaign list;

- plan view;
- live matrix;
- lineage;
- episode/measurement/selection views;
- event tail;
- read-only HTTP/SSE API;
- access checks for artifacts.

Acceptance. TUI can inspect imported legacy campaigns and live local campaigns. Server never permits filesystem traversal through artifact URIs.

39.18 RagOpt bridge

Purpose. Preserve current candidate/run evidence and reduce migration risk.

Implement.

- candidate importer;
- suite/case importer;
- outcome/metric importer;
- comparison/gate importer;
- runstore importer;
- legacy verification report;
- optional export of OptKit one-pair campaign to legacy report during transition.

Acceptance. All checked-in product candidates import. Golden legacy run produces matching OptKit estimates and selection.

40. Suggested implementation phases and pull requests

Phase 0 — Architecture decision records and fixtures

Goal. Freeze semantics before code spreads.

Deliverables:

- ADR: OptKit scope and dependency direction;
- ADR: event-sourced control plane/native artifact data plane;
- ADR: snapshot versus environment;
- ADR: typed measurements and epochs;
- ADR: search versus selection versus promotion;
- copied/verified RagOpt fixture run;
- one Coinvault candidate fixture;
- one RAG-TTC candidate fixture.

Exit criteria:

- maintainers approve terminology and laws;
- every proposed package has a stated owner/non-goal;
- fixtures can be validated by existing code.

Phase 1 — Identity, artifacts, journal, replay

Pull requests:

1. model and canonical encoding.
2. artifact filesystem store.
3. event envelope and filesystem journal.
4. reducer/checkpoint/memory store tests.

Exit criteria:

- append/replay/crash tests pass under race detector;
- generic TUI can show a synthetic event timeline.

Phase 2 — Space and candidate model

Pull requests:

1. domains/variables/snapshots;
2. patches/candidate policy;
3. candidate/lineage artifacts;
4. RagOpt candidate importer.

Exit criteria:

- all checked-in RAG-TTC and Coinvault candidate bundles import;
- exactly-one mutation parity is demonstrated.

Phase 3 — Trials, episodes, and paired analysis

Pull requests:

1. dataset manifests/roles;
2. arms/episode keys/paired planner;
3. episode lifecycle events;
4. legacy outcome adapter;
5. paired estimands/statistics;
6. legacy gate translator.

Exit criteria:

- synthetic paired campaign reproduces RagOpt golden results;
- missing and incompatible measurements fail closed.

Phase 4 — Runtime controller and FlowKit

Pull requests:

1. SystemAdapter/PreparedSystem APIs;
2. controller scheduling/resume;
3. FlowKit resource integration;
4. native trajectory manifests;
5. observation sink.

Exit criteria:

- fake product campaign survives interruption and resumes without duplicate completed episodes;
- resource conservation property tests pass.

Phase 5 — RAG-TTC compatibility port

Pull requests:

1. RAG-TTC schema/environment/materializer;
2. episode runner/trajectory projector;
3. answer-quality measurement adapter;
4. I5 plan and parity test;
5. TUI projection reader.

Exit criteria:

- 15 feedback and validation fixture parity;
- legacy and OptKit paths can be run side-by-side;
- existing TUI can browse OptKit campaign data.

Phase 6 — Coinvault compatibility port

Pull requests:

1. Coinvault schema and legacy snapshot importer;
2. variable bindings and prepared manifest;
3. canonical episode runner/trajectory projector;
4. intervention probes;
5. named answer constraints;
6. judge/measurement adapter;
7. budget/resume reconciliation;
8. feedback/reproduction/promotion plan;
9. grounded-answer parity campaign.

Exit criteria:

- same 24-episode plan and terminal gate semantics;
- all existing treatment mechanisms represented without a central mechanism switch;
- native traces remain authoritative.

Phase 7 — Projection and UI layer

Pull requests:

1. core projections and SQLite index;
2. generic TUI;
3. read-only API/SSE;
4. RAG-TTC TUI views;
5. Coinvault optimization workspace;
6. product inspector integrations.

Exit criteria:

- plan/live/history views all derive from journal/projections;
- users can drill from decision to estimate to measurement to episode to native artifact.

Phase 8 — Search strategies

Pull requests:

1. manual strategy and proposal command;
2. coordinate/random strategies;
3. objective Pareto archive;
4. exposure-filtered reflection packet builder;
5. GEPA-style strategy behind an experimental package/flag;
6. case-elites archive and strategy-state artifacts.

Exit criteria:

- reflective strategy cannot access promotion data;
- every proposal cites authorized history and reasoning artifact;
- manual strategy remains fully supported.

Phase 9 — Deprecation and documentation

Deliverables:

- frozen RagOpt legacy compatibility support;
 - migration documentation;
 - operator runbooks;
 - incident/recovery guide;
 - API stability policy;
 - v0.x release and upgrade fixtures.
-

41. Intern work protocol

41.1 Reading order

1. This document through Part III.
2. ragopt/pkg/candidate, eval, compare, policy, gate, runstore.
3. Coinvault knowledge_ragopt_* files.
4. RAG-TTC toolevel/ragopt.go and TUI browser files.
5. JudgeKit construct/contract/protocol/report/suite packages.
6. FlowKit developer guide and report/ledger APIs.
7. Existing optimization design documents listed in the source map.

41.2 Work rhythm

For each pull request:

1. write the invariant or state transition first;
2. add a failing fixture/property test;
3. implement the narrowest package;
4. add golden artifact bytes when identity is involved;
5. run unit, race, vet, and boundary tests;
6. rebuild projections from scratch and compare;
7. document migration effect and any schema version change.

41.3 Questions for every review

What exact semantic identity is introduced or changed?
 What remains authoritative after this projection/helper is added?
 Can this code expose hidden data to a proposer?
 Can a missing/failed value become a zero?
 Can measurements from different epochs mix?
 Can a retry escape resource admission?
 Can a rejected candidate mutate the baseline?
 Can the UI derive a different decision than the controller?
 Can this artifact be replayed and verified after interruption?
 Does this generic package contain product semantics?

41.4 Definition of done for one feature

A feature is not complete until it has:

- schema/API documentation;
- validation and fail-closed behavior;
- deterministic identity tests;
- interruption/resume tests where applicable;
- projection/UI representation;
- migration notes;
- security/exposure analysis;
- no broken dependency boundaries.

Part VII — Worked examples

42. Worked example A: porting Coinvault’s grounded-answer experiment

This example follows one current Coinvault candidate end to end and shows how each OptKit abstraction is used.

42.1 Baseline and search space

The baseline snapshot contains all Coinvault configuration values. The campaign permits one variable:

```
search_space:
  variables:
    - coinvault.answer.grounding_prompt
  candidate_policy:
    minimum_changed_variables: 1
    maximum_changed_variables: 1
```

All retrieval, routing, model, evidence-ledger, authorizer, comparison-plan, and judge-promotion identities are locked by the plan/environment.

42.2 Patch and candidate

```
patch:
  id: patch-grounded-answer-v2
  base: snapshot:parent
  assignments:
    - variable: coinvault.answer.grounding_prompt
      before: artifact:sha256:old
      after: artifact:sha256:new
```

Candidate hypothesis:

With comparison-intent retrieval held fixed, direct clause-level entailment and adjacent citations will reduce unsupported comparison claims without degrading answer relevance.

42.3 Materialization

The materializer:

1. verifies base snapshot and environment;
2. initializes canonical Coinvault runner options;
3. applies every snapshot binding;
4. appends/places the grounding prompt exactly once;
5. records the expected runtime prompt digest;
6. seals a prepared-system manifest.

No model call occurs.

42.4 Trial plan

Feedback suite: 12 cases, one repeat, two arms.

$$N_{episodes} = 12 \times 1 \times 2 = 24.$$

The plan projection lists every episode before execution:

case	repeat	arm	status
feedback-compare-morgan-peace	0	incumbent	scheduled
feedback-compare-morgan-peace	0	challenger	scheduled
feedback-compare-gold-coins-bars	0	incumbent	scheduled
feedback-compare-gold-coins-bars	0	challenger	scheduled
...			

42.5 Execution and native trajectory

Each episode uses the canonical admin-chat runner. The challenger trajectory records the prompt digest and all runtime events. Native artifacts are sealed before evaluation.

42.6 Intervention check

For every challenger episode:

```
expected grounding prompt digest = sha256:new
observed runtime suffix digest   = sha256:new
```

The variable applies to all answer episodes. Mismatch is a blocking intervention failure.

42.7 Deterministic constraints

For each episode:

```
terminal success
answer/projection valid
required route
forbidden route absent
required evidence groups
citation resolution
protected abstention behavior
```

If any blocking check fails, judge measurements are recorded as missing/skipped with the causal constraint reference.

42.8 JudgeKit measurements

Valid episodes produce faithfulness, answer_relevance, and optionally claim-level/unsupported-claim measurements under exact epochs.

42.9 Paired estimates

For the two target comparison cases, suppose the values reproduce current evidence:

Morgan versus Peace:

$$\Delta F = 1.0000 - 0.4595 = 0.5405.$$

Gold coins versus bars:

$$\Delta F = 0.9615 - 0.3778 = 0.5837.$$

Target mean:

$$\widehat{\Delta F} = \frac{0.5405 + 0.5837}{2} \approx 0.5621.$$

42.10 Selection

Despite the large target gain, assume five other challenger episodes have route/retrieval/contract failures. The hard feasibility phase fails before target evaluation is allowed to promote the candidate.

identity	PASS
hard	FAIL
completed	PASS
contract	FAIL
failure rate	FAIL
floor	not fully satisfied
target	NOT EVALUATED FOR ELIGIBILITY
regressions	NOT EVALUATED
cost	NOT EVALUATED

The comparison estimate remains available for diagnosis and archive policy, but the candidate is ineligible for promotion.

42.11 What the strategy learns

The history can support a textual conclusion:

The grounding intervention appears effective when the knowledge route succeeds.
The next recoverable failure is routing/abstention, not grounding.

A human or reflective strategy may propose a one-coordinate routing-prompt patch while preserving the grounding lesson as diagnostic evidence. The production baseline does not change unless a full staged promotion succeeds.

42.12 UI path

```

lineage node grounded-answer-v2
↓
selection HARD FAIL
↓
failed route constraint
↓
episode feedback-protected-003/challenger
↓
Coinvault native timeline and tool call
↓
compare against successful target episode
↓
reflection/proposal artifact for routing-v3

```

This drill path is one of the principal reasons to retain event/entity references rather than only a Markdown summary.

43. Worked example B: porting RAG-TTC I5

43.1 Variable and patch

Mutable variable:

```
rag_ttc.tool.search.description
```

Hypothesis:

A combined first search containing every exact comparison subject and requested attribute will preserve quality while reducing redundant provider/search calls.

43.2 Environment

The environment validator pins:

```
answer profile/model
embedding model/dimensions
selected profile digest
evaluator identity
tool safety policy
source digests
index manifest digest
corpus digest
```

A drifted source file produces `environment_drift` before any episode starts.

43.3 Feedback trial

Three feedback cases, one repeat, two arms:

$$3 \times 1 \times 2 = 6$$

episodes.

The materializer writes:

```
orchestration.txt
answer-schema.json
search-description.yaml
tool-qa.yaml
prepared-manifest.json
```

43.4 Native execution

The episode runner invokes `chatpkg.Runtime.Submit`, closes the runtime without losing terminal artifacts, and seals the native session. `ProjectChatRecord` becomes the trajectory projector.

43.5 Measurements and cost

System usage:

```
provider calls
tool calls
input/output/reasoning tokens
duration
```

Evaluation usage:

statement-judge calls/tokens
verdict-judge calls/tokens

Quality measurements:

faithfulness
answer relevance

43.6 Selection policy

Current rules translate directly:

faithfulness floor	0.80
comparison answer-relevance mean delta	≥ 0
worst faithfulness delta	≥ -0.20
worst answer relevance delta	≥ -0.50
overall mean faithfulness delta	≥ -0.05
overall mean relevance delta	≥ -0.05
cost tie-breakers: provider calls, tool calls, tokens, duration	

43.7 Validation stage

Seven validation cases, two repeats, two arms:

$$7 \times 2 \times 2 = 28$$

episodes.

The strategy should not receive validation trajectories unless the plan explicitly classifies the stage as development. A passing result produces a human review request, not automatic replacement of the checked-in description.

43.8 TUI path

campaign list
↓
I5 campaign plan: 6 feedback + conditional 28 validation episodes
↓
feedback live matrix
↓
candidate comparison table
↓
search-call/session inspector
↓
validation stage and selection checks

44. Worked example C: a future GEPA-style Coinvault campaign

This example demonstrates that the same core supports more than one manually authored candidate.

44.1 Search space

answer.grounding_prompt
answer.routing_prompt
knowledge.tool_description

Candidate policy permits one or two prompt/description variables but forbids authorization, corpus, judge, and evidence-ledger changes.

44.2 Initial population

baseline
human seed A: grounding clarification
human seed B: routing clarification
human seed C: tool-description source-role clarification

44.3 Development rollouts

The strategy allocates a small diagnostic batch to each seed. Trajectories include full tool calls, outputs, constraints, and judge diagnostics.

44.4 Reflection packet

```
parent_candidate: seed-B
selected_examples:
  failures:
    - case: feedback-protected-003
      trajectory: artifact:sha256:...
      constraints:
        - forbidden_sql_route
      measurements: []
    - case: feedback-mixed-schema
      trajectory: artifact:sha256:...
      constraints:
        - wrong_source_role
  successes:
    - case: feedback-knowledge-only-004
      trajectory: artifact:sha256:...
      measurements:
        faithfulness: 0.94
        answer_relevance: 0.87
search_space:
  allowed_variables:
    - answer.routing_prompt
    - knowledge.tool_description
forbidden_information:
  promotion_cases: not exposed
```

44.5 Reflector output

The reflector emits structured patches and rationale, not arbitrary filesystem edits:

```
proposals:
  - hypothesis: >-
    Separate "document schema" from live inventory-record schema and describe
    knowledge_search as authoritative for historical/educational content.
  assignments:
    - variable: answer.routing_prompt
      value: artifact:sha256:...
  - hypothesis: >-
    Clarify source roles in the tool description without altering routing prompt.
  assignments:
```

```
- variable: knowledge.tool_description
  value: artifact.sha256:...
```

OptKit independently validates both.

44.6 Archive update

Suppose one candidate is best on protected routing and another is best on comparison relevance. A case-elites or Pareto archive retains both. The next reflection can select one parent or create a permitted merged patch.

44.7 Promotion remains separate

Even if the development archive finds a high-scoring candidate, fresh reproduction and closed promotion stages use the independent selection policy. The reflector never sees promotion details.

45. Worked example D: optimizing widget generation

OptKit should support product behavior beyond text and retrieval.

45.1 Variables

```
coinvault.ui.presentation_policy
coinvault.ui.widget_selection_prompt
coinvault.ui.table_threshold
```

45.2 Trajectory events

```
model emits widget intent
projection validates intent schema
frontend renders widget
user task result or synthetic interaction test
```

Separate:

```
agent action = widget intent
```

from

```
environment result = rendered widget artifact.
```

45.3 Measurements and constraints

Hard constraints:

```
widget schema valid
render completes
no restricted data exposed
keyboard-accessibility checks pass
```

Objectives:

```
task completion
answer comprehension
appropriate visualization choice
```

latency
render cost

A visually attractive widget cannot compensate for rendering failure or data leakage.

45.4 UI inspector

The episode inspector can show intent JSON beside rendered output and accessibility diagnostics:

Widget evaluation		
intent: inventory_table	schema PASS	render PASS
rows: 18	truncated: yes	accessibility PASS
[rendered preview or product-native link]		
task completion .91	presentation appropriateness .88	latency 74ms

This illustrates why trajectory, measurement, and product inspector abstractions must not assume “final answer text” is the only output.

46. Worked example E: remeasurement after a judge update

46.1 Problem

A new faithfulness protocol improves claim extraction. Existing candidate trajectories remain valid, but old and new values are not automatically comparable.

46.2 OptKit procedure

- 1. Register new measurement epoch η_2 .
- 2. Create a remeasurement trial over selected sealed trajectories.
- 3. Run only the JudgeKit suite; do not rerun product episodes.
- 4. Record new measurements linked to the same episodes.
- 5. Compare candidates only within η_2 .
- 6. Optionally run a calibration bridge between η_1 and η_2 .

46.3 History

episode E-101

- └ faithfulness .82 epoch η_1
- └ faithfulness .77 epoch η_2

candidate C-7

- └ selection decision under η_1 (historical, unchanged)
- └ new descriptive/recomputed decision under η_2

Historical decisions remain historically accurate; OptKit does not rewrite them after the evaluator changes.

Part VIII — Testing, safety, and operations

47. Test strategy

47.1 Unit tests

Each package needs strict validation tests, canonical digest goldens, unknown-version rejection, nil/empty handling, and boundary cases.

47.2 Property tests

High-value properties:

```
canonicalization independent of map insertion order
patch application order independent after canonical sorting
archive result independent of insertion order
replay(full journal) == replay(checkpoint + suffix)
projection rebuild == incremental projection
paired analysis independent of episode completion order
resource consumed ≤ admitted
hidden view contains no forbidden refs
```

47.3 State-machine tests

Generate invalid event sequences and verify reducer rejection:

```
complete before start
measurement before trajectory seal
selection before estimates
promotion without review/eligible decision
candidate child digest not matching patch application
exposure not authorized by plan
```

47.4 Crash tests

Simulate process death:

```
after artifact write, before event append
after event append, before head update
after episode native seal, before completion command
after measurement artifact, before measurement event
during projection update
during checkpoint write
```

Recovery should either adopt verified orphan artifacts through explicit repair or leave them unreferenced; it must never infer a false completion.

47.5 Race tests

Run `go test -race` around:

```
parallel episode workers
journal command submission
observation sinks
projection subscribers
artifact puts
FlowKit resource sharing
pause/stop during execution
```

47.6 Golden legacy tests

Maintain frozen inputs/outputs for:

RagOpt candidate digest/diff

RagOpt paired schedule

RagOpt comparison report

RagOpt gate decisions: pass, hard fail, target fail, regression fail, tie-break

RAG-TTC I5 configuration

Coinvault grounded-answer treatment/contract/gate

47.7 Integration tests

Use fake providers for deterministic CI. Controlled live integration tests should be opt-in and source/profile locked.

47.8 UI tests

- projection JSON goldens;
 - event-cursor consistency;
 - browser/TUI rendering snapshots where stable;
 - artifact authorization tests;
 - no color-only meaning;
 - corrupt campaign rendering;
 - late/out-of-order observation stream handling;
 - epoch/missingness warnings.
-

48. Safety model

48.1 No automatic production mutation

OptKit may export an approved patch or promotion bundle. It does not edit deployed prompts, configs, databases, or model checkpoints.

48.2 Locked safety variables

Campaign search spaces should exclude authorization, data-access policy, evidence-ledger semantics, production secrets, and judge/promotion controls by default. Making one mutable requires a specialized campaign and independent review.

48.3 Judge exploitation

Mitigations:

separate development and promotion instruments

measurement epochs

calibration/audit requirements

hidden promotion data

human review

trajectory/constraint evidence, not score alone

no simultaneous uncontrolled optimizer+judge mutation

48.4 Prompt injection and reflective optimizers

Trajectory content may contain adversarial text from corpora, tools, or users. Reflection packet builders should:

- mark untrusted content boundaries;
- redact secrets;
- restrict artifact tools;
- require structured patch output;
- validate all values against domains;
- prevent the reflector from editing plan/search/exposure policy;
- record model/protocol identity and raw response artifact.

48.5 Data privacy

Case payloads and native trajectories need sensitivity labels. Generic projections should default to bounded summaries. Product inspectors enforce product authorization.

48.6 Resource denial and runaway adaptation

Plans require hard maxima for rounds, candidates, episodes, provider calls, tokens, and optional money. Strategy decisions cannot increase these ceilings.

48.7 Cancellation semantics

Operator cancellation should:

1. record a stop request;
 2. stop admitting new work;
 3. allow configured commit-after-cancellation behavior for successful expensive work;
 4. seal available native artifacts;
 5. mark unscheduled work cancelled;
 6. reconcile resources;
 7. produce an inspectable terminal/incomplete campaign state.
-

49. Operational runbooks

49.1 Preflight

```
validate plan/schema versions
verify all artifact digests
verify baseline snapshot and patch
validate environment/source/model/corpus identities
validate data-role disjointness and exposure policy
validate measurement epochs
calculate exact known episodes and worst-case adaptive bound
preflight resources/prices
render plan projection
make zero provider calls
```

49.2 Start

```
create campaign directory
write immutable plan/manifest
append campaign.created and plan_validated
acquire controller writer lock
append campaign.started
schedule first trial/strategy action
```


49.3 Resume

- verify plan and journal chain
- load/check latest checkpoint
- replay suffix
- verify referenced terminal artifacts
- reconcile resource ledger with native facts
- repair head/projections if derived state is stale
- reschedule only pending exact episode specs

49.4 Pause

A pause stops new admission after recording the event. Running episodes follow stage policy: allow to finish or cancel. Resume is a new attributable event.

49.5 Judge outage

Because trajectories seal before measurement:

- product episodes remain complete
- measurement tasks remain pending/failed
- campaign can resume judging without rerunning product episodes
- selection remains inconclusive

49.6 Environment drift mid-campaign

If a worker detects a changed model/code/index identity:

- stop admitting affected episodes
- record `environment_drift`
- fail or pause campaign according to plan
- never mix new-environment episodes into existing trial
- start child campaign with new environment if continuation is desired

49.7 Corrupt artifact

Fail closed, preserve the corrupt bytes for diagnosis, and record verification failure. Do not silently recompute expensive work under the same identity.

49.8 Promotion handoff

- selection decision eligible
 - ↓
- freeze review packet
 - ↓
- collect required human reviews
 - ↓
- record promotion
 - ↓
- export promotion bundle:
 - parent snapshot
 - patch
 - child snapshot
 - evidence/decision/review refs
 - ↓
- normal product deployment process applies it

↓
record deployed version in a new external/product event or follow-up campaign

50. Observability for OptKit itself

OptKit should expose operational metrics distinct from product objectives:

journal append latency/errors
projection lag
artifact verification failures
controller loop latency
queued/running episodes
FlowKit retries/quarantines
strategy invocation latency/failure
measurement backlog
resource reconciliation discrepancies
subscriber lag

These can be emitted through existing logging/metrics conventions. Do not treat them as campaign measurements unless explicitly imported.

Part IX — Design decisions and alternatives

51. Architecture decisions

Decision 1: OptKit is a separate domain-neutral module

Chosen. A standalone optkit module owns optimization process semantics.

Why. FlowKit is too low-level; JudgeKit measures but does not optimize; RagKit is RAG-specific; product repositories cannot supply a reusable control plane; RagOpt's current API is fixed around one mutation and one pair.

Rejected alternative. Expand RagOpt directly into every optimization use case. This would make non-RAG users depend on a RAG-named module and make backward compatibility constrain the new model.

Decision 2: RagOpt becomes a RAG preset/compatibility layer

Chosen. RagOpt imports OptKit, preserves legacy import, and supplies RAG-specific conveniences.

Why. Existing users and artifacts remain valuable, but candidate/trial/measurement abstractions are broader than RAG.

Decision 3: event-sourced control plane, native artifact data plane

Chosen. Campaign history is an append-only journal; large trajectories remain product-native and content-addressed.

Why. This supports plan/live/history views, replay, custody, and product fidelity.

Rejected alternative. Copy every trajectory into one global warehouse as the authority. This would duplicate data, create schema pressure, and weaken product semantics.

Decision 4: filesystem first

Chosen. Local filesystem store with one writer per campaign.

Why. It matches current RagOpt operational constraints, is inspectable, and keeps the first implementation bounded.

Deferred. SQL/object-store/distributed controller implementations can satisfy the same interfaces later.

Decision 5: snapshots are total configurations; mutability is plan-relative

Chosen. A snapshot does not permanently label assets mutable/locked.

Why. The same variable may be mutable in one campaign and locked in another. This separates semantic state from experimental permission.

Decision 6: patches, not treatment mechanism enums

Chosen. Generic variable assignments plus product bindings/probes.

Why. This removes the central switch while preserving exact runtime exercise checks.

Decision 7: materialization and execution are separate

Chosen. Provider-free deterministic preparation precedes stochastic episodes.

Why. It enables preflight, clear identity, reuse, UI diffs, and failure attribution.

Decision 8: native trajectories remain authoritative

Chosen. OptKit stores a manifest and refs, not a lossy universal transcript.

Why. Coinvault and RAG-TTC have rich but different runtime representations.

Decision 9: typed measurements and explicit epochs

Chosen. Measurement is authoritative; float maps are compatibility projections.

Why. Optimization without instrument identity invites invalid aggregation and judge exploitation.

Decision 10: estimands precede estimates

Chosen. Campaign plans state population, contrast, construct, epoch, aggregation, and missingness.

Why. This makes statistical questions stable even when estimator implementation evolves.

Decision 11: search, selection, review, and promotion are separate

Chosen. Search proposes/allocates; selection computes eligibility; review records human judgment; promotion records adoption.

Why. An optimizer should not certify or deploy its own winner.

Decision 12: lexicographic hard constraints remain first-class

Chosen. No universal weighted reward in the core.

Why. Product safety, authorization, route validity, and catastrophic regressions must not be compensable.

Decision 13: data exposure is persisted

Chosen. The campaign records which actor saw which case/trajectory/detail.

Why. Reflective optimization makes hidden-set leakage especially consequential.

Decision 14: paired trials are built-in but not universal

Chosen. Preserve current paired rigor through a trial-design interface.

Why. Future multi-arm, sequential, shadow, and reproduction designs need the same episode/measurement substrate.

Decision 15: static product adapters for v0

Chosen. Product binaries link their adapters and invoke the OptKit library.

Why. Dynamic plugin loading adds complexity without solving a current requirement.

Decision 16: no automatic deployment

Chosen. OptKit exports promotion evidence/patches only.

Why. Deployment authorization belongs to product release processes.

Decision 17: UI logic is projection logic

Chosen. UIs consume stable read models; they do not scan arbitrary files or recompute selection.

Why. This keeps TUI/web/history views consistent and testable.

Decision 18: RAG-TTC is the first complete port

Chosen. Prove interfaces on the smaller adapter, with Coinvault fixtures guiding requirements.

Why. This lowers implementation risk without treating Coinvault as an afterthought.

52. Alternatives rejected or deferred**52.1 Put optimization state into FlowKit**

Rejected. FlowKit's strength is bounded deterministic execution with no persisted adaptive control state. Adding campaign search/selection would blur that contract.

52.2 Make JudgeKit own promotion

Rejected. Measurement instruments should not decide product deployment. JudgeKit correctly treats evaluators as instruments, not authorities.

52.3 Represent every objective as scalar reward

Rejected. It hides feasibility constraints, metric direction, applicability, and instrument identity.

52.4 Keep exactly one mutation forever

Rejected as a universal rule; retained as a campaign policy. One-coordinate experiments are excellent for attribution, but some search algorithms require multi-component proposals.

52.5 Make the SQLite warehouse authoritative

Rejected. It is a useful derived analysis/index format but should be rebuildable from journals/native artifacts.

52.6 Let the optimizer read all campaign files

Rejected. The authorized history view is required to enforce hidden-set and privacy boundaries.

52.7 Rejudge historical results in place

Rejected. New measurements are appended under a new epoch; historical decisions remain immutable.

52.8 Automatically merge “good parts” of failed candidates

Deferred to search strategies. The control plane should preserve evidence, but semantic merging requires product/algorithm knowledge and a new validated patch.

52.9 Build distributed workers first

Deferred. Interfaces allow it later; local one-controller campaigns are sufficient to validate semantics and ports.

52.10 Implement official GEPA as the first strategy

Rejected. Manual/coordinate/random strategies and exposure/measurement correctness must work first. A reflective strategy is valuable only when its evidence substrate is trustworthy.

53. Final recommendation

Build OptKit as a **versioned experimental control system for optimizing stochastic compound programs**.

The complete control loop can be summarized as

$$\begin{aligned} H_n &\xrightarrow{\text{authorized optimizer}} \delta_n \\ \theta'_n &= \theta_n \oplus \delta_n \\ \tau_{i,r}^a &\sim K_{\theta_n^a}(x_i) \\ I_{i,r} &= \text{Intervention}(\delta_n, \tau_{i,r}^1) \\ C_{i,r}^a &= \text{Contracts}(x_i, \tau_{i,r}^a) \\ M_{i,r}^a &= \text{Instruments}_\psi(x_i, \tau_{i,r}^a) \\ \Delta_{i,r} &= M_{i,r}^1 - M_{i,r}^0 \\ A_{n+1} &= \text{Archive}(A_n, \Delta, C, I) \\ D_n &= \text{Selection}(A_{n+1}, \Delta, C, I, B) \\ H_{n+1} &= H_n \cup \{\delta_n, \tau, M, \Delta, D_n\}. \end{aligned}$$

The append-only history H yields three consistent temporal views:

before execution: plan projection
while executing: live projection
after execution: lineage, evidence, and decision projections

Its core object is not a prompt and not a scalar reward. It is the persistent relation among:

configuration → patch → trajectory → measurement → estimate → decision → lineage

The first release should reproduce today's strongest behavior exactly:

- immutable candidate identity;
- one-coordinate interventions where campaigns require them;
- paired deterministic scheduling;
- native trajectory custody;
- treatment/intervention exercise checks;
- deterministic product constraints before judges;
- typed, versioned measurements;
- lexicographic hard gates;
- bounded resources and resumability;
- human-controlled promotion.

Once those semantics are stable, manual Coinvault optimization, RAG-TTC I5 experiments, GEPA-like reflection, multi-component search, widget-policy optimization, and future RL-generated policies become different strategies and adapters over the same compositional core.

The implementation should be judged by a simple standard: an engineer must be able to answer, from durable evidence,

What changed?

Why was it proposed?

Did the change actually execute?

What trajectories resulted?

What exactly was measured, by which instrument?

What population-level claim was estimated?

Which constraints passed or failed?

Why was the candidate retained, rejected, reviewed, or promoted?

What data did the optimizer see?

What production configuration descends from this decision?

If OptKit makes those questions easy before, during, and after every campaign, it will be a sound reusable foundation rather than another prompt-optimization wrapper.

Appendices

Appendix A. Proposed schema sketches

These schemas are abbreviated. Production schemas need strict unknown-field rejection, identifier bounds, version validation, canonicalization rules, and cross-reference checks.

A.1 Snapshot

```
api_version: optkit.snapshot/v1
id: snapshot:coinvault-2026-08-18
system:
  name: coinvault.admin-chat
  adapter_schema: coinvault.opt-adapter/v1
values:
  coinvault.knowledge.default_results:
    kind: integer
    inline: 5
    digest: sha256:...
```

```
coinvault.answer.grounding_prompt:
  kind: text_artifact
  artifact:
    digest: sha256:...
    media_type: text/plain
    size_bytes: 3812
labels:
  release: deployed-v12
digest: sha256:...
```

A.2 Environment

```
api_version: optkit.environment/v1
system:
  name: coinvault.admin-chat
  adapter_schema: coinvault.opt-adapter/v1
code_revision: 6a3f...
dimensions:
  answer_runtime: lunaroute/gpt-5.6-luna/revision-x
  judge_runtime: lunaroute/gpt-5.6-luna/revision-y
  index_bundle: rk-55be57...
  evaluator: gec-canonical-admin-chat-evaluator/v1
artifacts:
  - digest: sha256:...
    schema: coinvault.source-lock/v1
digest: sha256:...
```

A.3 Patch

```
api_version: optkit.patch/v1
id: patch:grounded-answer-v2
base_snapshot: snapshot:coinvault-2026-08-18
base_digest: sha256:...
assignments:
  - variable: coinvault.answer.grounding_prompt
    before:
      digest: sha256:old
      artifact: {digest: sha256:old, media_type: text/plain}
    after:
      digest: sha256:new
      artifact: {digest: sha256:new, media_type: text/plain}
digest: sha256:...
```

A.4 Candidate

```
api_version: optkit.candidate/v1
id: candidate:gec-grounded-answer-v2
parent: {id: snapshot:..., digest: sha256:...}
patch: {id: patch:..., digest: sha256:...}
child: {id: snapshot:..., digest: sha256:...}
proposer:
  kind: human
  identity: ragopt-gec-phase5-evidence-only-answer
hypothesis: >-
  Direct clause-level entailment and adjacent citations will improve
  faithfulness on comparison cases.
```

```
targets:
  - estimand: estimand:comparison-faithfulness-delta
regression_risks:
  - overly terse answers
  - omitted synthesis
evidence:
  - digest: sha256:...
    schema: coinvault.diagnostic-manifest/v1
created_at: 2026-08-20T00:00:00Z
digest: sha256:...
```

A.5 Measurement

```
api_version: optkit.measurement/v1
id: measurement:...
subject:
  kind: episode
  id: episode:...
construct:
  id: faithfulness
  definition_digest: sha256:...
instrument:
  kind: judgekit
  contract_digest: sha256:...
  protocol_digest: sha256:...
  adapter_version: optkit-judgekit/v1
epoch: sha256:...
status: observed
applicability: applicable
value:
  kind: scalar
  number: 0.9615
uncertainty: null
evidence:
  - digest: sha256:...
assessment:
  digest: sha256:...
produced_at: 2026-08-20T00:00:00Z
digest: sha256:...
```

A.6 Constraint result

```
api_version: optkit.constraint-result/v1
constraint_id: coinvault.route.required_knowledge
subject: {kind: episode, id: episode:...}
status: fail
severity: hard
instrument:
  kind: deterministic
  identity: coinvault-answer-contract/v5
expected:
  route: knowledge_search
observed:
  route: sql_doc
evidence:
  - digest: sha256:...
```



```
message: Required knowledge route was not used.  
digest: sha256:...
```

A.7 Estimand

```
api_version: optkit.estimand/v1  
id: comparison-faithfulness-delta  
construct: faithfulness  
epoch_policy:  
  mode: exact  
population:  
  groups: [feedback, comparison]  
contrast:  
  kind: paired_difference  
  baseline_role: incumbent  
  candidate_role: challenger  
aggregation:  
  method: mean  
missing_policy: fail_closed  
direction: maximize  
digest: sha256:...
```

A.8 Selection policy

```
api_version: optkit.selection-policy/v1  
name: gec-grounded-answer-v2  
phases:  
  - id: identity  
    stop_on_failure: true  
    checks:  
      - kind: complete_pairing  
      - kind: exact_plan_identity  
  - id: hard  
    stop_on_failure: true  
    checks:  
      - kind: all_episodes_completed  
      - kind: all_hard_constraints_pass  
      - kind: maximum_failure_rate  
        maximum: 0  
      - kind: measurement_floor  
        construct: faithfulness  
        minimum: 0.80  
  - id: target  
    stop_on_failure: true  
    checks:  
      - kind: minimum_estimate  
        estimand: comparison-faithfulness-delta  
        minimum: 0  
  - id: regression  
    stop_on_failure: true  
    checks:  
      - kind: minimum_pair_delta  
        construct: faithfulness  
        minimum: -0.20  
      - kind: minimum_pair_delta  
        construct: answer_relevance
```

```

    minimum: -0.30
- id: cost
  stop_on_failure: false
  checks:
    - kind: ordered_usage
      resources: [provider_calls, tool_calls, system_tokens, duration]
digest: sha256:...

```

A.9 Event envelope

```

{
  "api_version": "optkit.event/v1",
  "campaign_id": "campaign:...",
  "sequence": 184,
  "id": "event:...",
  "kind": "measurement.recorded",
  "occurred_at": "2026-08-20T14:36:13Z",
  "actor": {"kind": "instrument", "identity": "judgekit-suite:..."},
  "correlation_id": "trial:...",
  "causation_id": "event:episode-completed-...",
  "entities": [
    {"kind": "episode", "id": "episode:..."},
    {"kind": "measurement", "id": "measurement:..."}
  ],
  "payload": {"digest": "sha256:...", "schema": "optkit.measurement-recorded/v1"},
  "artifacts": [{"digest": "sha256:...", "schema": "judgekit.assessment/v1"}],
  "previous_digest": "sha256:...",
  "digest": "sha256:..."
}

```

Appendix B. Event catalog and reducer effects

Event	Required prior state	Reducer effect
campaign.created	no campaign	initializes manifest
campaign.plan_validated	draft	pins plan/environment validation
campaign.started	planned/paused	sets running
candidate.proposed	running stage accepts proposals	adds proposed entity
candidate.validated	proposed	records patch/child validity
candidate.materialized	validated	links prepared manifest
trial.planned	materialized arms/cases valid	adds trial and episode specs
episode.started	scheduled	marks attempt active
episode.trajectory_sealed	started	links native trajectory
constraint.recorded	trajectory sealed	adds typed constraint
measurement.recorded	trajectory sealed and allowed	adds measurement
episode.completed	required evaluation terminal	marks complete
trial.analysis_completed	required episodes terminal	links estimates/report
selection.decision_recorded	estimates/constraints available	records eligibility
archive.updated	candidate evidence available	replaces archive projection state
review.requested	reviewable/eligible	creates frozen packet
review.recorded	open request	adds reviewer annotation
promotion.recorded	required decision/reviews	advances baseline lineage
exposure.recorded	authorization check passed	updates leakage ledger
campaign.completed	stop/terminal policy satisfied	seals campaign

Every event payload has its own versioned schema. Reducers must reject unknown semantic versions rather than guessing.

Appendix C. Candidate and episode identity equations

C.1 Snapshot

$$D_{\text{snapshot}} = H(\text{schema}, \text{system}, \text{sort}_{\text{variable}}(\text{variable}, \text{value_digest})).$$

C.2 Patch

$$D_{\text{patch}} = H(\text{schema}, D_{\text{base}}, \text{sort}_{\text{variable}}(\text{variable}, \text{before}, \text{after})).$$

C.3 Candidate

$$D_{\text{candidate}} = H(D_{\text{parent}}, D_{\text{patch}}, D_{\text{child}}, \text{proposer}, \text{hypothesis}, \text{targets}, \text{risks}, \text{evidence refs}).$$

C.4 Prepared system

$$D_{\text{prepared}} = H(D_{\text{snapshot}}, D_{\text{environment}}, D_{\text{adapter}}, D_{\text{prepared artifacts}}, D_{\text{expected effects}}).$$

C.5 Episode key

$$K_e = H(D_{\text{plan}}, \text{stage}, D_{\text{trial design}}, D_{\text{case}}, D_{\text{prepared}}, \text{repeat}, D_{\text{execution protocol}}, \text{reuse epoch}).$$

C.6 Measurement epoch

$$D_{\text{epoch}} = H(D_{\text{construct/contract}}, D_{\text{protocol}}, D_{\text{adapter}}, D_{\text{aggregation}}, D_{\text{calibration policy}}).$$

Appendix D. Minimal end-to-end pseudocode

```
func RunManualPairedCampaign(ctx context.Context, deps Dependencies, plan campaign.Plan, draft campaign.Draft) error {
    // Create and validate the campaign.
    manifest, err := deps.Repository.Create(ctx, plan)
    if err != nil { return err }
    ctl := campaign.NewController(deps, manifest.ID)

    // Submit one human proposal. The controller records the command/event.
    if err := ctl.SubmitCandidate(ctx, draft); err != nil { return err }

    // The controller loop performs the remaining work:
    // validate patch → materialize → plan paired trial → execute episodes →
    // intervention/constraints → measure → estimate → select.
    if err := ctl.Run(ctx); err != nil { return err }

    state, err := deps.Repository.Replay(ctx, manifest.ID)
    if err != nil { return err }

    decision := state.LatestSelection()
    fmt.Printf("candidate=%s status=%s evidence=%s\n",
        decision.Candidate, decision.Status, decision.Digest)
    return nil
}
```

Controller episode execution:

```
func (c *Controller) executeEpisode(ctx context.Context, spec trial.EpisodeSpec) (trial.EpisodeResult, error) {
    if reusable, result := c.Reuse.FindExact(spec.Key); reusable {
        c.Events.Append(EpisodeReused(spec, result))
        return result, nil
    }

    prepared := c.State.PreparedSystem(spec.Arm.Snapshot)
    c.Events.Append(EpisodeStarted(spec))

    sink := c.Observations.NewSink(spec.ID)
    tr, runErr := c.Adapter.RunEpisode(ctx, runtime.EpisodeRequest{
        Spec: spec, Prepared: prepared, Case: c.State.Case(spec.Case),
    }, sink)
    stream, sealErr := sink.Seal(context.WithoutCancel(ctx))

    if runErr != nil || sealErr != nil {
        return c.SealEpisodeFailure(spec, tr, stream, runErr, sealErr)
    }
    c.Events.Append(TrajectorySealed(spec, tr, stream))

    constraints, err := c.Evaluator.InterventionChecks(ctx, EvalRequest(spec, tr))
    if err != nil { return trial.EpisodeResult{}, err }
    constraints2, err := c.Evaluator.DeterministicConstraints(ctx, EvalRequest(spec, tr))
    if err != nil { return trial.EpisodeResult{}, err }
    constraints = append(constraints, constraints2...)
    c.RecordConstraints(constraints)

    var measurements []measure.Measurement
    if !measure.AnyBlockingFailure(constraints) {
        measurements, err = c.Evaluator.Measurements(ctx, EvalRequest(spec, tr))
        if err != nil {
            c.RecordMeasurementFailure(spec, err)
        } else {
            c.RecordMeasurements(measurements)
        }
    }

    result := BuildEpisodeResult(spec, tr, constraints, measurements)
    c.Events.Append(EpisodeCompleted(result))
    return result, nil
}
```

Appendix E. Intern review checklist

Architecture

- ☐ Does the package belong in OptKit rather than a product, JudgeKit, RagKit, or FlowKit?
- ☐ Is dependency direction enforced by a test?
- ☐ Is native product data still authoritative?

Identity

- ☐ Are semantic inputs complete?

- ☐ Are canonical bytes deterministic?
- ☐ Are schema versions strict?
- ☐ Are environment and snapshot identities both present where needed?

Candidate

- ☐ Does actual diff validation ignore the proposer's claim and compute changes independently?
- ☐ Does patch application reject stale Before values?
- ☐ Is mutability defined by the campaign, not permanently by the snapshot?

Execution

- ☐ Is materialization provider-free?
- ☐ Is repeat distinct from retry attempt?
- ☐ Does exact episode reuse include every semantic input?
- ☐ Are retries admitted and metered?
- ☐ Can interruption resume without duplicate completed work?

Measurement

- ☐ Is the construct defined?
- ☐ Is the instrument/protocol pinned?
- ☐ Is the epoch explicit?
- ☐ Are NA, missing, failed, invalid, and zero distinct?
- ☐ Can every value drill down to evidence/assessment?

Statistics and selection

- ☐ Is the estimand stated before computing the estimate?
- ☐ Is missingness policy explicit?
- ☐ Are hard constraints non-compensable?
- ☐ Does the decision cite exact inputs?
- ☐ Is search separated from promotion?

Exposure

- ☐ Can the strategy access only its authorized view?
- ☐ Are hidden-set disclosures recorded?
- ☐ Are sensitive artifacts protected independently of digest knowledge?

UI

- ☐ Is the panel a projection, not a second implementation of business logic?
- ☐ Does it show event cursor/epoch identity where relevant?
- ☐ Does it avoid color-only meaning?
- ☐ Can users drill to native evidence?
- ☐ Does it distinguish failure classes and missingness?

Migration

- ☐ Are legacy bytes/digests preserved?
- ☐ Is parity tested before deleting old code?
- ☐ Are evaluator changes represented as epochs rather than fake candidate changes?

Appendix F. Glossary

Actor — an attributable human, strategy, instrument, reviewer, controller, or product component participating in a campaign.

Applicability — whether a measurement or intervention check meaningfully applies to a subject. Distinct from pass/fail and missingness.

Archive — a policy-maintained collection of candidates retained for search or analysis.

Arm — a named snapshot/environment pairing participating in a trial.

Artifact — immutable bytes identified by digest, media type, schema, size, and sensitivity.

Authorized view — the exact history and artifact subset an access/exposure policy permits an actor to inspect.

Baseline — the snapshot used as the current comparison/root within a campaign stage. It is not necessarily the deployed production snapshot unless declared so.

Budget — an admitted upper bound on a named resource within a declared scope.

Campaign — the persistent optimization process containing plan, history, candidates, trials, measurements, decisions, and lineage.

Candidate — an immutable proposed child snapshot plus patch, hypothesis, proposer, targets, risks, and evidence.

Case — one stable evaluation input with identity, payload, groups, and data-role metadata.

Constraint — a typed predicate affecting feasibility or selection.

Construct — the abstract property an evaluation intends to measure, operationalized by a measurement contract.

Command — a request to change campaign state; unlike a control event, it may be rejected and is not itself a durable fact.

Control event — an immutable ordered fact in the campaign journal.

Control plane — the campaign process state and decisions, as opposed to large product-native artifacts.

Data plane — native trajectories, assessment reports, reflection packets, prompts, and other referenced bytes.

Dataset role — declared use such as diagnostic, development, calibration, selection, promotion, or shadow.

Domain — the admissible values and canonical encoding for a variable.

Environment fingerprint — content-addressed external execution semantics such as code, model, corpus, index, and adapter identities.

Episode — one planned execution of one arm on one case/repeat under one execution protocol.

Episode key — exact digest determining whether a completed episode is semantically reusable.

Entity reference — a typed identifier for a campaign domain object, distinct from a reference to artifact bytes.

Estimate — a statistical result computed from observed measurements for an estimand.

Estimand — the population-level quantity a campaign seeks to learn.

Exposure — recorded information made visible to an actor, including detail level and purpose.

Execution protocol — the semantic rule for expanding trials into episodes, including arm order, repeats, seeds, and reuse.

Failure — a typed, attributable unsuccessful outcome with phase, cause, retryability, and evidence references.

Instrument — the procedure that produces a measurement, including deterministic checkers or JudgeKit protocols.

Intervention check — evidence that a patch's changed variable actually affected runtime behavior as expected.

Journal — append-only sequence of campaign control events.

Measurement — one typed observation of a construct for a subject, with instrument, epoch, applicability, status, value, and evidence.

Measurement epoch — compatibility identity controlling which measurements may be aggregated.

Metric — a convenient named numeric projection; not a substitute for a typed measurement.

Native artifact — the product-owned authoritative execution artifact, such as a Coinvault timeline or RAG-TTC session.

Objective — an estimand or ordered set of estimands the search process seeks to improve.

Observation stream — optional high-volume trajectory events used for live inspection and generic summaries.

Pareto frontier — candidates not dominated across a declared objective vector.

Patch — a stale-safe set of variable assignments applied to a base snapshot.

Plan — immutable declaration of permitted search, stages, data, measurement, resources, exposure, and selection.

Prepared system — deterministic materialization of a snapshot/environment into product runtime configuration and expected-effect metadata.

Projection — rebuildable read model derived from plan, journal, and artifacts.

Promotion — authorized adoption of a candidate as a new baseline or deployment input, recorded separately from selection.

Proposal — strategy/human output requesting creation of one or more candidates or trial allocations.

Resource — a named consumable such as provider calls, tokens, dollars, or execution slots.

Reuse policy — rules stating which prior episodes, trajectories, or measurements may satisfy requested work under exact identity checks.

Repeat — statistical replication index. Distinct from an operational retry attempt.

Review — attributable human assessment over a frozen evidence packet.

Search space — campaign-declared mutable variables, domains, and candidate structural rules.

Search strategy — component that consumes authorized history and proposes declarative next work.

Selection policy — ordered rules that classify a candidate's evidence as eligible, ineligible, inconclusive, invalid, or reviewable.

Snapshot — immutable total assignment of system variables to canonical values.

Stage — policy-bounded campaign phase with data, trial, measurement, resource, exposure, reuse, selection, and transition rules.

System adapter — product-owned implementation that validates environment, materializes snapshots, and executes episodes.

Trajectory — complete product execution artifact and optional normalized streams produced by an episode.

Trial — concrete set of arms, cases, repeats, episode specs, and analysis protocol generated by a trial design.

Trial design — rule that plans episodes and analyzes them, such as paired A/B evaluation.

Trial allocation — a declarative strategy request assigning candidates, case slices, repeats, instruments, and resources to future trials.

Usage scope — attribution of resource cost to system execution, evaluation, optimizer, or control work.

Value — canonical assignment for one variable, inline or artifact-backed.

Variable — named coordinate of system configuration with a domain and product binding.

Appendix G. Source map from the studied repositories

RagOpt

- `ragopt/pkg/candidate/types.go` — current snapshot/candidate/mutation model.
- `ragopt/pkg/eval/types.go` and `runner/resume` files — paired cells and product-arm boundary.
- `ragopt/pkg/compare/types.go`, `build.go` — strict paired comparisons and aggregates.
- `ragopt/pkg/policy/policy.go` — current gate-policy schema.
- `ragopt/pkg/gate/evaluate.go` — lexicographic identity/hard/target/regression/cost evaluation.
- `ragopt/pkg/runstore/*` — local atomic run storage and lifecycle.
- `ragopt/pkg/review/*` — blind-review support.
- `ragopt/http/2026/08/06/RAGOPT-001--reusable-reproducible-self-optimization-harness/design-doc/01-ragopt-intern-guide-to-a-reusable-evidence-gated-optimization-harness.md` — original scope, ownership, and decisions.

Coinvault

- `coinvault/cmd/coinvault/cmds/knowledge_ragopt.go` — canonical product execution, budgets, native artifacts, judge flow, and RagOpt adapter.
- `knowledge_ragopt_treatment.go` — current mechanism contracts and treatment-exercise checks.
- `knowledge_ragopt_contract.go` — answer/route/evidence/citation contracts.
- `knowledge_ragopt_trace.go` — canonical event observation and runtime facts.
- `knowledge_ragopt_gate.go` — comparison/gate/review/promotion artifacts.
- `coinvault/configs/ragopt/*` — historical candidate bundles and hypotheses.
- `coinvault/pkg/doc/tutorials/03-efficient-rag-optimization-experiment.md` — operator workflow and source-lock/treatment observability discipline.
- `coinvault/internal/webchat/sessionstream/*` and `coinvault/web/src/ws/*` — existing live session transport and projection architecture.

RAG-TTC

- `rag-ttc/cmd/rag-ttc/cmds/tooleval/ragopt.go` — I5 adapter, source locks, materialization, native session projection, and judge invocation.
- `rag-ttc/assets/configs/ragopt/i5-combined-comparison-v1/*` — candidate, suites, contracts, and gate policy.
- `rag-ttc/internal/admin/tui/runstorebrowser.go` and `expbrowser.go` — current completed-run TUI.
- `rag-ttc/http/2026/08/02/RAG-TTC-GEPA-OPT-001--*/design-doc/01-intern-guide-to-a-pragmatic-gepa-inspired-self-optimization-loop.md` — derived SQLite warehouse, reflection packet, dataset partition, and future search design.
- `rag-ttc/http/2026/07/30/RAG-TTC-EXP-BROWSER-001--*/design-doc/01-design-experiment-results-browser.md` — experiment-browser architecture.

JudgeKit

- `judgekit/spec/construct.go`, `contract.go` — constructs and measurement contracts.
- `judgekit/protocol/protocol.go` — complete evaluator protocol identity.

- `judgekit/eval/*` — instances/evidence/artifacts.
- `judgekit/assessment/report.go` — sealed assessment reports.
- `judgekit/suite/suite.go` — evaluator DAG execution.
- `judgekit/audit/*`, `calibration/*` — reliability and calibration.

RagKit

- `ragkit/rag/*` — reusable RAG data/evidence/usage/ordering contracts.
- `ragkit/rag/answering/*`, `ragkit/rag/indexbundle/*` — product-adjacent RAG execution components.
- `ragkit/boundary_test.go` — dependency-boundary precedent.

FlowKit

- FlowKit README and developer guide — bounded execution, content-addressed cache identity, retries, budgets, metering, reports, ledgers, and the explicit non-goal of persistent workflow control state.

Appendix H. Research context

The architecture is compatible with, but not limited to, several recent optimization approaches:

- **GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning**, Agrawal et al., arXiv:2507.19457. Relevant ideas: system-level trajectories, natural-language reflection, prompt proposals, and Pareto-style exploration.
- **Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs**, Opsahl-Ong et al., arXiv:2406.11695. Relevant ideas: multi-module prompt optimization, program/data-aware proposals, credit assignment, mini-batch evaluation, and surrogate-assisted search.
- **TextGrad: Automatic “Differentiation” via Text**, Yuksekogonul et al., arXiv:2406.07496. Relevant ideas: computation-graph variables and textual feedback propagated toward optimizable components.
- **Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge**, Shi et al., arXiv:2406.07791. Relevant implication: evaluator outputs require explicit protocol identity, audit, and reliability treatment rather than being treated as truth.

OptKit should implement the stable substrate these methods need. It should not hard-code one paper’s optimizer as the framework’s core model.
